UNIVERZITA TOMÁŠE BATI VE ZLÍNĚ
Fakulta technologická
Institut informačních technologií

ZÁKLADY POČÍTAČOVÉ GRAFIKY

Ing. Pavel Pokorný, Ph.D.

2004

OBSAH

ÚVOD 5
1. ZÁKLADY ROVINNÉ GRAFIKY 6
1.1 Rasterizace jednoduchých objektů 6
1.1.1 Úsečka 6
1.1.2 Kružnice a elipsa 9
1.1.3 Křivky 13
1.1.4 Oblasti 17
1.2 Barvy a barevné modely 22
1.2.1 Barevné modely 22
1.3 Zpracování obrazu 25
1.3.1 Digitalizace 26
1.3.2 Alias a antialiasing 28
1.3.3 Reprezentace rastrového obrazu 29
1.3.4 Komprese rastrových obrazů 31
1.3.5 Transformace barev 35
1.4 Základní grafické a animační formáty 41
1.4.1 PCX 41
1.4.2 BMP 42
1.4.3 TIFF 43
1.4.4 GIF 43
1.4.5 PNG 44
1.4.6 JPG (JPEG) 45
1.4.7 DXF 46
1.4.8 WMF 46
1.4.9 FLI/FLC 47
1.4.10 MPEG 48
2. ZÁKLADY PROSTOROVÉ GRAFIKY 49
2.1 Promítání 49
2.1.1 Rovnoběžné promítání 49
2.1.2 Perspektivní promítání 53
2.2 Modelování 3D Objektů 55
2.2.1 Hraniční reprezentace 57
2.2.2 Šablonování 59
2.2.3 Obsazenost prostoru a oktálové stromy 60
2.2.4 Konstruktivní geometrie těles 62

2.3 Kódování objektů a zobrazování ve 3D 62
2.3.1 VRML 62
2.3.2 DirectX 63
2.3.3 OpenGL 65
2.3.4 Porovnání DirectX a OpenGL 66
2.4 Viditelnost objektů 67
2.4.1 Předzpracování dat 68
2.4.2 Liniové algoritmy viditelnosti 69
2.4.3 Rastrové algoritmy viditelnosti 69
2.5 Zobrazovací modely a stínování 72
2.5.1 Zobrazovací techniky a stínování 73
2.5.2 Světelné zdroje 76
2.6 Globální zobrazovací metody 78
2.6.1 Částicový pohled na světlo 78
2.6.2 Vlnový pohled na světlo 81
2.7 Textury 83
2.7.1 Mapování textur 84
2.7.2 Promítané textury 86
2.7.3 Hrbolaté textury 86
2.8 Fraktály 87
2.8.1 Nekonečně členité útvary 88
2.8.2 Hausdorffova dimenze a soběpodobnost 89
2.8.3 Typy fraktálů 91
2.9 Animace 94
2.9.1 Nízkoúrovňová animace 95
2.9.2 Vysokoúrovňová animace 97
3. GRAFICKÝ HARDWARE 101
3.1 Vstupní grafická zařízení 101
3.1.1 Myš 101
3.1.2 Scanner 102
3.1.3 Optické pero 103
3.1.4 Trackball 103
3.1.5 Tablet 103
3.1.6 Joystick 103
3.2 Výstupní grafická zařízení 104
3.2.1 Tiskárna 104
3.2.2 Plotter 104

3.2.3 Monitor 105
3.2.4 LCD 106
3.2.5 Grafická karta 107
4. MATEMATIKA V POČÍTAČOVÉ GRAFICE 110
4.1 Vektory 110
4.1.1 Velikost vektoru 110
4.1.2 Sčítání a odčítání vektorů, opačný vektor 110
4.2 Matice 111
4.2.1 Operace s maticemi 112
4.3 Transformace 113
4.3.1 Homogenní souřadnice 113
4.3.2 Dvojrozměrné geometrické transformace 113
4.3.3 Trojrozměrné geometrické transformace 117
SEZNAM POUŽITÉ LITERATURY 120

ÚVOD

Skripta „Základy počítačové grafiky“ jsou určeny zejména studentům FT UTB ve Zlíně a obsahují ucelený přehled látky předmětu Počítačová grafika, jehož výuku zajišťuje Institut informačních technologií. Oblast počítačové grafiky je velice rozsáhlá a v omezeném rozsahu těchto skript nemůže být uvedena celá problematika. Proto bych případné zájemce o podrobnější informace odkázal na použitou literaturu, jejíž seznam je uveden na konci této publikace.

Zároveň si jsem vědom, že se počítačová grafika, stejně jako jiné oblasti počítačové techniky, neustále rozvíjí a stále se objevují další možnosti jejího uplatnění. Nelze se tedy při studiu upnout pouze na tento zdroj. Při hodinách výuky bude probíraná látka neustále doplňována a aktualizována.

Tato skripta jsou napsána pouze po zkušenostech z prvního roku výuky. Jelikož stejně jako celý obor Informační technologie prochází tento předmět ještě vývojem, může se v dalších letech lišit rozsah a náplň tohoto předmětu. Vzhledem k těmto skutečnostem nemusí celý obsah těchto skript aktuálně odpovídat rozsahům výuky. Aby mohlo být dosaženo co nejvyšší kvality výuky, není možné přistupovat k náplni pouze ze strany vyučujícího, ale je nutná i spolupráce se studenty. Proto uvítám veškeré připomínky, dotazy a náměty nejen k těmto učebním textům, ale i předmětu samotnému.

Celá skripta jsou rozdělena do čtyř kapitol. První z nich pojednává o základech rovinné grafiky. Obsahuje popis rasterizace základních dvojrozměrných objektů (úsečka, kružnice, elipsa, křivky a plochy). Dále se zabývá transformací některých objektů z pohledu geometrie a barev a jsou zde charakterizovány nejčastěji používané grafické soubory. Druhá část je zaměřena na prostorovou grafiku. Zde jsou popsány druhy reprezentace trojrozměrných objektů a stručný popis způsobu programování práce s 3D objekty. Kromě jiného zde student nalezne také porovnání různých způsobů kódování objektů ve 3D, rozebranou problematiku řešení viditelnosti a popis globálních zobrazovacích metod. Třetí část se zabývá hardwarem souvisejícím s grafikou a poslední (čtvrtá) shrnuje aplikace matematiky v počítačové grafice – vektory, matice a základní transformace ve 2D a 3D.

Jelikož je možné se v počítačové grafice často setkat s originálními pojmy v angličtině, zpravidla jsou kromě českých výrazů uvedeny i původní názvy. Aby byly odlišeny od okolního textu, jsou většinou uzavřeny v závorkách a psány kurzívou. Stejným způsobem (tedy kurzívou) jsou značeny jednotlivé proměnné a veličiny, které se používají v uvedených vzorcích.

Přeji všem studentům hodně úspěchů při studování těchto skript.

Ing. Pavel Pokorný, Ph.D.
Ve Zlíně, 10.9.2004

1. ZÁKLADY ROVINNÉ GRAFIKY

1.1 Rasterizace jednoduchých objektů

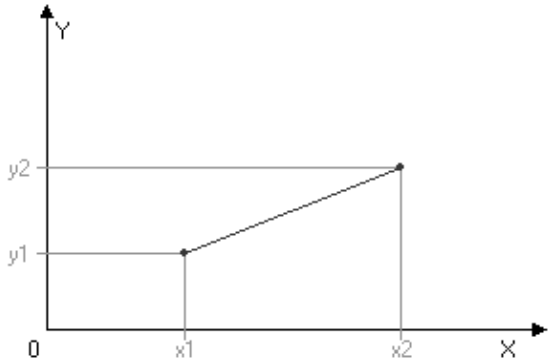
Rasterizací se nazývá transformace základních grafických objektů (úsečka, kružnice, elipsa, křivky, oblasti…) do posloupnosti obrazových bodů. Při práci se používají výstupní zařízení, které většinou pracují s rastrem. Základním stavebním prvkem rastru je právě bod, ze kterého jsou sestaveny všechny grafické prvky. Příkladem může být monitor, který se skládá z rastrové obrazovky. Množství těchto bodů (pixelů) je dáno rozlišením monitoru (např. při rozlišení 800x600 je na obrazovce celkem 480 000 pixelů).

Při rasterizaci objektů vzniká určitá chyba, která vzniká skutečností, že bod je nejmenší zobrazovací jednotka a souřadnice zobrazovaných bodů, které nemají celočíselnou hodnotu, se musí zaokrouhlovat.

1.1.1 Úsečka

Úsečka patří k základním a nejjednodušším grafickým prvkům. Složitější obrazce bývají často nahrazeny právě posloupností navazujících úseček, proto by vykreslování úseček mělo být prováděno co nejefektivněji.

Úsečka se nejčastěji zadává pomocí koncových bodů $[x_1, y_1]$ a $[x_2, y_2]$, případně jedním bodem $[x_1, y_1]$ a vektorem rozdílu souřadnic $(\Delta x, \Delta y) = (x_2 - x_1, y_2 - y_1)$ – viz obr. 1. Pro jednodušší rasterizaci se zpravidla dodržuje zásada, že první koncový bod leží vlevo od druhého bodu, případně nad ním.



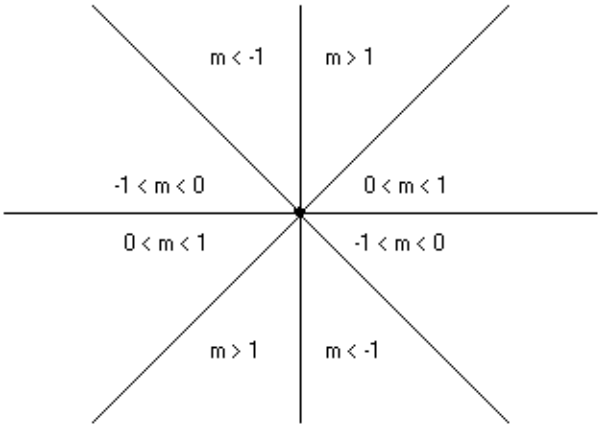
Obr. 1. Úsečka daná koncovými body

Popis úsečky, případně dalších grafických prvků bývá doplněn **atributy**, které definují jejich vzhled. U úsečky jde o tloušťku a způsob kresby čáry (plná, čerchovaná…).

Úsečka je vzorkována s konstantním krokem vždy v jedné z os *x* a *y* v závislosti na sklonu, který udává **směrnice** *m*. Ta je dána podílem rozdílů koncových a počátečních souřadnic úsečky

$$m = \frac{\Delta y}{\Delta x} = \frac{y_2 - y_1}{x_2 - x_1}. \tag{1}$$

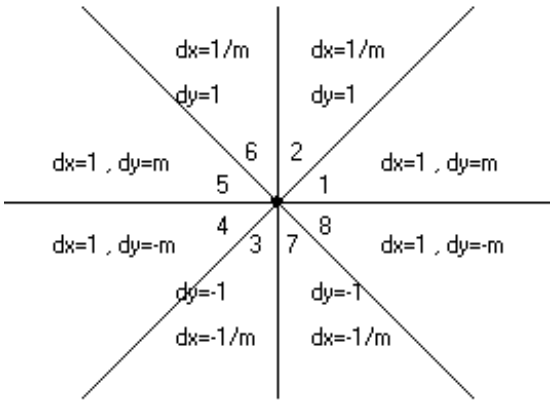
Hodnoty směrnice *m* pro různé sklony úsečky jsou patrné na obr. 2. Pokud je $|m|<1$, úsečka má k ose *x* sklon menší než 45°, proto se provádí vzorkování podle osy *x*. V případě, že je $|m|>1$, vzorkuje se podle osy *y*. Má-li úsečka směrnici 1, nazývá se diagonální a vzorkuje se podle libovolné osy. Osa, podle které je prováděno vzorkování, se nazývá **hlavní** nebo **řídicí**. Zbylá osa se nazývá **vedlejší**.



Obr. 2. Hodnoty směrnice *m* pro různé sklony úsečky

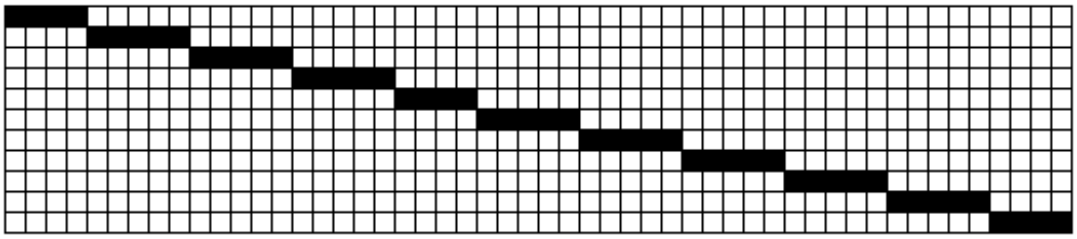
1.1.1.1 DDA algoritmus

DDA (*Digital Differential Analyzer*) algoritmus bývá také často nazýván jako **jednoduchý přírůstkový algoritmus**. Je založený na pravidelném přírůstku *Δx* pro souřadnici *x a Δ*y pro souřadnici *y*. Začíná se v počátečním bodu úsečky a přírůstek na řídicí ose je rovný hodnotě 1 (jednomu bodu).



Obr. 3. Přírůstky pro souřadnice *x* a *y*

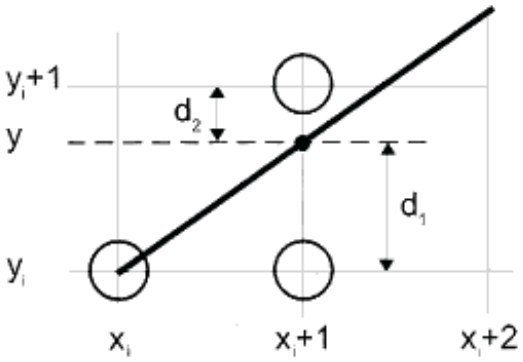
Řešení rasterizace DDA algoritmem je následující: v jednom cyklu se postupně připočítává k souřadnici na řídicí ose jednička a k souřadnici na vedlejší ose vypočítaný přírůstek. V případě, že je hlavní osa *x*, má hodnotu směrnice *m*, pokud je hlavní osa *y*, přírůstek je 1/*m* (obr. 3). Tyto hodnoty jsou reálné a při vykreslování se zaokrouhlují. Při výpočtu dalších souřadnic DDA algoritmus využívá hodnoty předcházejícího bodu. Tento typ algoritmu se označuje jako **iterační** (přírůstkový).



Obr. 4. Příklad rasterizace úsečky DDA algoritmem

1.1.1.2 Bresenhamův algoritmus

J. E. Bresenham vyvinul algoritmus rasterizace úsečky, jehož efektivita spočívá ve výpočtu za použití celočíselné aritmetiky. Následující odvození bude vycházet z předpokladu, že řídicí osa je *x* a jsou známy souřadnice bodu [*x*, *y*]. Úkolem je zjistit, zda následující bod úsečky má souřadnice [*x*+1, *y*] nebo [*x*+1, *y*+1] (viz obr. 5).



Obr. 5. Výběr pixelů úsečky

Obecná rovnice přímky je dána $y = mx + b$, kde *m* je směrnice (viz rovnice (1)) a *b* posun na ose *y*. Souřadnice následujícího bodu přímky jsou dány vztahem $y = m(x_i + 1) + b$. Pak lze odvodit

$$d_1 = y - y_i = m(x_i + 1) + b - y_i \tag{2}$$

$$d_2 = y_i + 1 - y = y_i + 1 - m(x_i + 1) - b\,. \tag{3}$$

Rozdíl těchto dvou vzdáleností lze vyjádřit jako

$$\Delta d = d_1 - d_2 = 2m(x_i + 1) - 2y_i + 2b - 1\,. \tag{4}$$

Podle proměnné Δd lze snadno určit, který ze dvou možných bodů leží blíže skutečné úsečce. Záporná hodnota znamená, že blíže leží pixel o souřadnici y_i, kladná hodnota určuje jako bližší pixel o souřadnici y_i+1. Pro stanovení, který z bodů se má vykreslit, není důležitá skutečná hodnota proměnné Δd, ale jen její znaménko. Jedinou neceločíselnou proměnnou je směrnice m, která je daná jako podíl Δy a Δx. Vynásobením celé rovnice hodnotou Δx je možné získat vztah

$$p_i = \Delta d \Delta x = 2\Delta y x_i - 2\Delta x y_i + 2\Delta y + \Delta x(2b-1), \tag{5}$$

kde proměnná p_i představuje tzv. **predikci**. Její hodnota v následujícím kroku bude

$$p_{i+1} = 2\Delta y x_{i+1} - 2\Delta x y_{i+1} + 2\Delta y + \Delta x(2b-1). \tag{6}$$

Vzájemným odečtením rovnic (6) a (5) vznikne rovnice

$$p_{i+1} = p_i + 2\Delta y - 2\Delta x(y_{i+1} - y_i). \tag{7}$$

Pro $x_{i+1} = x_i + 1$ lze počítat hodnoty každé další proměnné z předchozí hodnoty. Znaménko určuje způsob její aktualizace:

pro $p_i \le 0$ platí $p_{i+1} = p_i + 2\Delta y$,

pro $p_i > 0$ platí $p_{i+1} = p_i + 2\Delta y - 2\Delta x$. (8)

Počáteční hodnota se vypočte jako

$$p_1 = 2\Delta y - \Delta x. \tag{9}$$

Snadnost výpočtu (a tím i rychlost algoritmu) dokazuje aktualizace predikce, která je dána pouze sčítáním. Pokud je její hodnota záporná, souřadnice y následujícího bodu se nemění, při kladném znaménku má následující pixel souřadnici y o jedničku vyšší.

Tento postup platí pro úsečky, jejíž směrnice je kladná a menší než 1 (obr. 2). Analogicky by platily vzorce pro úsečky orientované ve zbylých třech směrech.

1.1.2 Kružnice a elipsa

1.1.2.1 Kružnice

Kružnice bývá jednoznačně definována svým středem a poloměrem nebo souřadnicemi tří různých bodů. Implicitně ji lze vyjádřit rovnicí

$$(x - x_c)^2 + (y - y_c)^2 - r^2 = 0, \tag{10}$$

nebo parametricky

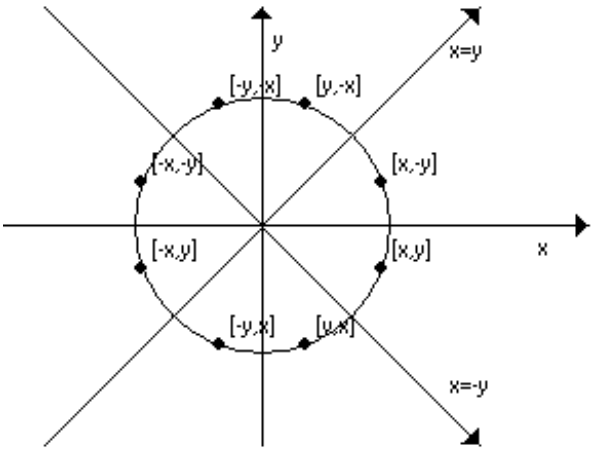
$$x = x_c + r\cos\alpha$$
$$y = y_c + r\sin\alpha \,, \qquad (11)$$

kde $\alpha \in \langle 0, 2\pi \rangle$.

Jedním cyklem lze tak získat posloupnost pravidelně rozmístěných bodů na kružnici. Jejich spojením vznikne posloupnost úseček nahrazující kružnici. Při velikosti přírůstku α o velikost $1/r$, dochází ke generování bodů na kružnici, jejichž souřadnice se liší přibližně o jeden pixel. Tento postup počítání jednotlivých bodů na kružnici je pomalý, protože se pro každý bod musí vyhodnocovat dvě goniometrické funkce. Výhodnější tak bývá využít transformaci otáčení (viz kap. 4.3.2.2), při které jsou souřadnice jednoho bodu na kružnici opakovaně podrobovány otočení kolem středu kružnice. Výpočet hodnot $\sin\alpha$ a $\cos\alpha$ nutných k transformaci otáčení, stačí provést pouze jednou. Nevýhodou takového algoritmu je však jeho iteračnost, protože se chyba vzniklá výpočtem kumuluje a u velkých kružnic se pak poslední bod kružnice nemusí setkat s prvním. Místo kružnice takto vznikne spirála.

Bresenhamův algoritmus

Při pohledu na kružnici je vidět, že je symetrická k osám *x*, *y* a i na osy kvadrantů (funkce $x = y$ a $x = -y$). Z toho vyplývá, že leží–li na kružnici se středem v počátku souřadnicového systému bod (x, y), potom i (y, x), $(y, -x)$, $(x, -y)$, $(-x, -y)$, $(-y, -x)$, $(-y, x)$ a $(-x, y)$ jsou body této kružnice (obr. 6). Proto na určení všech bodů kružnice stačí vypočítat jen její 1/8 (oktant), např. oblouk 0 - 45°. Bresenhamův algoritmus vychází z tohoto zjednodušení a stejně jako u úsečky pracuje pouze s celočíselnou aritmetikou.



Obr. 6. Body ležící symetricky na kružnici

Opět se předpokládá řídicí osa *x*, která má konstantní krok výpočtu. Dále platí, že střed kružnice je v počátku souřadnicového systému a platí implicitní rovnice, která se zapíše jako funkce

$$F(x, y) : x^2 + y^2 - r^2 = 0 \,. \qquad (12)$$

Dle obr. 7 má z bodu [x_i, y_i] odvozený následující bod souřadnice [x_i +1, y_i] nebo [x_i +1, y_i -1]. Uprostřed mezi těmito body leží tzv. **středový bod** rozhodující o tom, který ze zmíněných bodů se má zobrazit. Pokud se dosadí souřadnice středového bodu [x_i +1, y_i –1/2] do implicitní rovnice (12), vznikne vztah

$$p_i = F(x_i + 1, y_i - \frac{1}{2}) = (x_i + 1)^2 + (y_i - \frac{1}{2})^2 - r^2, \tag{13}$$

kde p_i, stejně jako u úsečky, značí **predikci**. Pokud je její znaménko záporné, bude pro další kresbu vybrán bod se stejnou souřadnicí y, v opačném případě se vybere bod ležící o pixel níže. Hodnota predikce se opět určuje iteračně z předcházející hodnoty

$$p_{i+1} = p_i + 2x_i + 3 + (y_i - \frac{1}{2})^2 + (y_{i+1} - \frac{1}{2})^2. \tag{14}$$

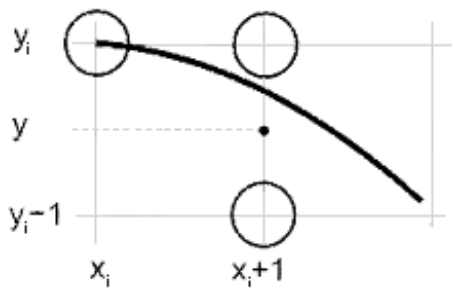
Podle znaménka lze tento vzorec vyčíslit následovně:

pro $p_i \le 0$ platí $p_{i+1} = p_i + 2x_i + 3$,

pro $p_i > 0$ platí $p_{i+1} = p_i + 2x_i + 5 - 2y_i$. (15)

Počáteční hodnota se počítá ze vztahu

$$p_1 = 1 - r. \tag{16}$$



Obr. 7. Část kružnice v rastru

1.1.2.2 Elipsa

Elipsa je charakterizována svým středem a délkami dvou poloos a, b (toto je případ osové elipsy, jejíž rasterizace je jednodušší). Pokud její poloosy nejsou rovnoběžné s osami souřadnicové soustavy, není toto určení dostačující. V tomto případě se definuje svým středem a dvojicí vrcholů ležících na hlavních poloosách. Implicitně je možné elipsu vyjádřit rovnicí

$$\frac{(x - x_c)^2}{a^2} + \frac{(y - y_c)^2}{b^2} - 1 = 0 \tag{17}$$

nebo parametricky

$$x = x_c + a\cos\alpha$$
$$y = y_c + b\sin\alpha \quad , \tag{18}$$

kde $\alpha \in \langle 0, 2\pi \rangle$. Stejně jako kružnici lze elipsu vykreslit řešením těchto rovnic. Vzhledem k nutnosti počítání dvou goniometrických funkcí v každém kroku, také v tomto případě lze využít transformaci otáčení nebo Bresenhamův algoritmus.

Bresenhamův algoritmus

Pro osově orientovanou elipsu je možné poměrně snadno nalézt body na jejím obvodu pomocí celočíselné aritmetiky. Na rozdíl od kružnice je elipsa symetrická pouze podle souřadnicových os, lze tedy z jednoho bodu určit souřadnice tří dalších bodů. Proto je třeba provést rasterizaci v celém jednom kvadrantu. Elipsa se středem v počátku bývá implicitně vyjádřena funkcí

$$F(x, y) = b^2x^2 + a^2y^2 - a^2b^2 = 0 . \tag{19}$$

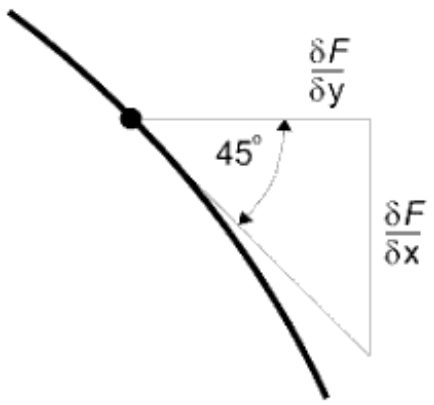
Z této rovnice lze pomocí středového bodu odvodit predikci podobně jako u kružnice. Při vzorkování elipsy dochází ke změně řídicí osy (obr. 8). Pokud se generují body v prvním kvadrantu zleva doprava, je řídicí osou nejprve *x* a posléze *y*. Bod, ve kterém dochází ke změně řídicí osy, má směrnici –1 (tečna má sklon –45 °). Jeho souřadnice lze nalézt prostřednictvím parciálních derivací funkce *F*. Pro derivace platí

$$\frac{\delta F(x, y)}{\delta x} = 2b^2x, \quad \frac{\delta F(x, y)}{\delta y} = 2a^2y . \tag{20}$$

V části s řídicí osou *x* se pro výpočet predikce používají pravidla

$$\begin{aligned} p_i \le 0 \quad & p_{i+1} = p_i + b^2(2x_i + 1) \\ p_i > 0 \quad & p_{i+1} = p_i + b^2(2x_i + 1) - 2a^2y_i \end{aligned} \tag{21}$$

Podobné vztahy lze odvodit, pokud je řídicí osou *y*.



Obr. 8. Změna řídicí osy u elipsy

1.1.3 Křivky

Křivky se v počítačové grafice hojně používají pro modelování ve dvou i třech dimenzích, při určování dráhy pohybujících se objektů, v počítačové animaci nebo definici fontů. Obecně ji lze vyjádřit třemi způsoby:

- implicitně - $F(x, y) = 0$,
- explicitně – $y = f(x)$,
- parametricky – $x = x(t)$, $y = y(t)$.

Křivka se fyzikálně chápe jako dráha pohybujícího se bodu. Tento bod lze pak vyjádřit rovnicí křivky

$$Q(t) = [x(t), y(t), z(t)], \tag{22}$$

nebo tzv. polohovým vektorem

$$\vec{q}(t) = (x(t), y(t), z(t)), \tag{23}$$

jehož velikost je rovna vzdálenosti bodu od počátku. Výhodou tohoto parametrického zápisu je závislost na jediném parametru t, jehož fyzikální interpretací je čas.

Derivace parametricky vyjádřené křivky se provádí po jejích složkách. Tečný vektor (bývá nazýván též **směrový**) je v počátečním bodě určen derivacemi ve tvaru

$$\vec{q}'(t_0) = (\frac{\partial x(t_0)}{\partial t}, \frac{\partial y(t_0)}{\partial t}, \frac{\partial z(t_0)}{\partial t}). \tag{24}$$

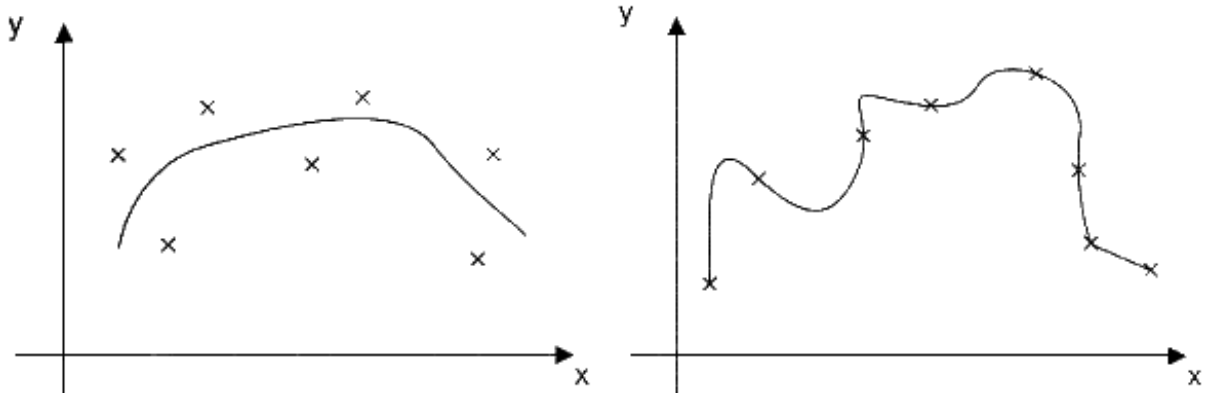
Z tohoto vztahu je patrné, že zásadní výhodou parametrické reprezentace křivek je snadné vyjádření tečny ke křivce. Této skutečnosti se využívá zejména při navazování křivek a skládání složitých tvarů z jednodušších částí.

Při modelování křivek mají největší význam křivky polynomiální udávané ve tvaru

$$p_n(t) = a_0 + a_1 t + ... + a_n t^n . \tag{25}$$

Polynomiální křivky lze rychle počítat a jsou snadno diferencovatelné. Nejčastěji používané jsou křivky třetího stupně (tzv. **kubiky**), které poskytují dostatečně širokou škálu tvarů a lze s nimi snadno manipulovat.

Modelování probíhá obvykle tak, že je definováno několik řídicích bodů a matematický aparát určuje průběh křivky z jejich vzájemné polohy. Existují dva základní způsoby interpretace řídicích bodů – aproximace a interpolace (obr. 9). Při aproximaci je tvar křivky sice určen řídicími body, ovšem sama křivka jimi procházet nemusí. U interpolace generovaná křivka všemi danými body přímo prochází.

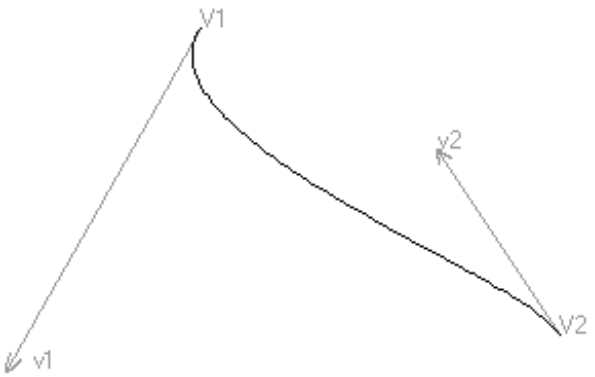


Obr. 9. Aproximační (vlevo) a interpolační křivka s řídicími body

1.1.3.1 Interpolační křivky

Fergusonovy kubiky

Nejznámější interpolační křivky užívané v počítačové grafice jsou Fergusonovy kubiky (obr. 10). Jsou určeny dvěma řídicími body *V1* a *V2* a hodnotami vektorů v těchto bodech *v1* a *v2*. Křivka začíná v bodě *V1* a končí v bodě *V2*. Tvar křivky je určen směrem a velikostí vektorů *v1* a *v2*. Budou-li oba vektory nulové, výsledná křivka bude úsečka. Velkou výhodou těchto křivek je snadná návaznost (stejné směrové vektory v navazujícím uzlu).



Obr. 10. Příklad Fergusonovy kubiky

Fergusonova kubika má tvar

$$Q(t) = T\begin{bmatrix} 2 & -2 & 1 & 1 \\ -3 & 3 & -2 & -1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}\begin{bmatrix} V_1 \\ V_2 \\ V_3 \\ V_4 \end{bmatrix}, \quad T = [t^3\ t^2\ t\ 1], \quad t \in \langle 0, 1 \rangle. \tag{26}$$

Její úpravou lze získat vyjádření

$$Q(t) = V1F_1(t) + V2F_2(t) + v1F_3(t) + v2F_4(t)$$
$$F_1(t) = 2t^3 - 3t^2 + 1, F_2(t) = 2t^3 + 3t^2, F_3(t) = t^3 - 2t^2 + t, F_4(t) = t^3 - t^2 \quad (27)$$

Dosazením $t = 0$ nebo $t = 1$ se lze přesvědčit, že křivka skutečně prochází body V1 a V2.

1.1.3.2 Aproximační křivky

V průběhu rozvoje výpočetní techniky se ukázalo, že prokládání křivek body pomocí interpolačních metod je málo pružné. Z tohoto důvodu se přešlo k aproximačním křivkám, kde se místo s body pracuje s lomenou čárou (tzv. řídicím polygonem), která určuje vytvářenou křivku.

Beziérovy křivky

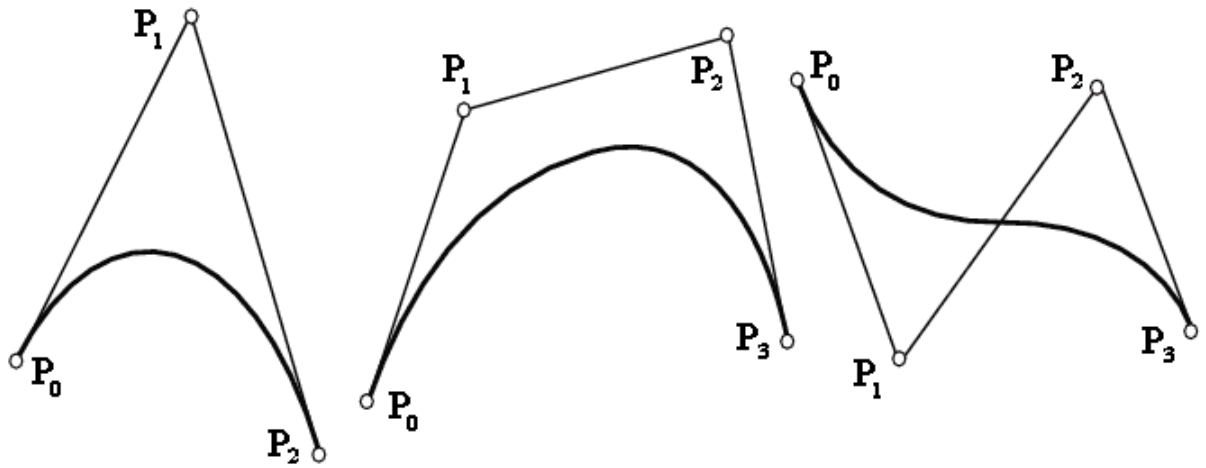
Beziérovy křivky patří mezi nejčastěji používané křivky. Své uplatnění nacházejí pro definici dvojrozměrných i trojrozměrných objektů nebo při definici fontů. Beziérova křivka stupně n je určena rovnicí

$$Q(t) = \sum_{i=0}^{n} P_i B_i^n(t), \quad (28)$$

kde P_i jsou polohové vektory vrcholů řídicího polygonu (tzv. Beziérovy body) a $B_i^n(t)$ jsou Bernsteinovy polynomy, pro které platí

$$B_i^n(t) = \binom{n}{i} t^i (1-t)^{n-i}, t \in <0,1>, i = 0,1..... \quad (29)$$

Beziérova křivka stupně n je dána řídicím polygonem s n+1 body P_0, … P_n. Několik příkladů Beziérových křivek je na obr. 11.



Obr. 11. Příklady Beziérových křivek druhého a třetího stupně

Pokud se do vztahu (28) dosadí $t = 0$ nebo $t = 1$, lze se snadno přesvědčit, že křivka prochází prvním a posledním bodem řídicího polygonu. Nevýhoda takto určené křivky je, že při změně jednoho řídicího bodu dojde ke změně celé křivky. Tato vlastnost vede k rozdělení složitějších křivek na menší segmenty, které se postupně navazují.

B-spline křivky

Obecná B-spline křivka stupně k je určena rovnicí

$$Q(t) = \sum_{i=0}^{n} P_i B_i^k(t), \tag{30}$$

kde P_i jsou vrcholy řídicího polynomu a $B_i^k(t)$ jsou normalizované bázové funkce stupně k, pro které platí následující vlastnosti:

- nezáporná hodnota $B_i^k(t) \geq 0$ pro všechny hodnoty i, n, t,
- jednotkový součet $\sum_{i=0}^{n} B_i^k(t) = 0$ pro $t \in \langle t_0, t_m \rangle$,
- rekurentní vztah $B_i^k(t) = \dfrac{t - t_i}{t_{i+k} - t_i} B_i^{k-1}(t) + \dfrac{t_{i+k+1} - t_i}{t_{i+k+1} - t_{i+1}} B_{i+1}^{k-1}(t)$,

kde $B_i^0(t) = 1$ pro $t \in \langle t_i, t_{i+1} \rangle$ a v ostatních případech je $B_i^0(t) = 0$.

V počítačové grafice se používají nejčastěji B-spline kubiky, které patří mezi aproximační. Jednou z nich je **Coonsova kubika**, která je definovaná svými čtyřmi řídicími body P_0, P_1, P_2, P_3 (obr. 12) a vztahem

$$Q(t) = \frac{1}{6} T \begin{bmatrix} -1 & 3 & -3 & 1 \\ 3 & -6 & 3 & 0 \\ -3 & 3 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} P_0 \\ P_1 \\ P_2 \\ P_3 \end{bmatrix}. \tag{31}$$

Obr. 12. Coonsova kubika

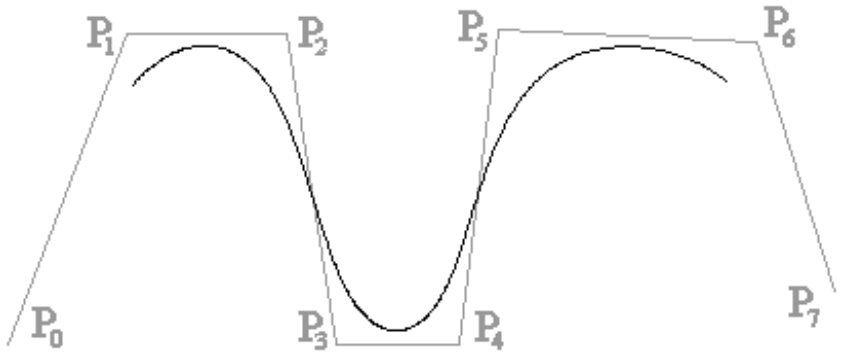
Charakteristické pro tuto křivku je, že neprochází krajními body svého řídicího polygonu. Dosazením za $t = 0$ a $t = 1$ do (31) lze vypočítat, že křivka začíná a končí v bodech

$$\begin{aligned} Q(0) &= \frac{P_0 + 4P_1 + P_2}{6} \\ Q(1) &= \frac{P_1 + 4P_2 + P_3}{6} \end{aligned} . \tag{32}$$

Tzv. uniformní neracionální kubický B-spline (Coonsův kubický B-spline) se dostane postupným navazováním Coonsových kubik (31). Jednotlivé Coonsovy kubiky mají řídicí body

$$\begin{aligned} &Q_i : P_{i-3}, P_{i-2}, P_{i-1}, P_i \\ &Q_{i+1} : P_{i-2}, P_{i-1}, P_i, P_{i+1} \\ &Q_{i+2} : P_{i-1}, P_i, P_{i+1}, P_{i+2} \\ &\ldots\ldots \end{aligned} . \tag{33}$$

Zatímco Coonsova plocha je určena čtyřmi body, Coonsův kubický B-spline určený $n \geq 4$ řídicími body se skládá z n-3 segmentů.



Obr. 13. Coonsův kubický B-spline určený 8 řídicími body

1.1.4 Oblasti

Dříve uvedené grafické objekty měly liniový charakter. Oblast představuje uzavřenou plochu, jejíž vnitřek lze vyplnit různými způsoby. Definice oblasti tak souvisí zejména s popisem její hranice. Rozlišují se dva způsoby popisu

- **hranice nakreslená v rastru** – její tvar může být libovolný, je třeba znát barvu hranice nebo barvu vnitřních (případně vnějších) bodů oblasti,
- **geometricky určená hranice** – se nejčastěji zadává posloupností bodů definujících mnohoúhelník. Hranice se uzavírá tak, že poslední bod se spojuje s prvním. Kromě přímkových úseků lze definovat hranice i pomocí jedné nebo více křivek.

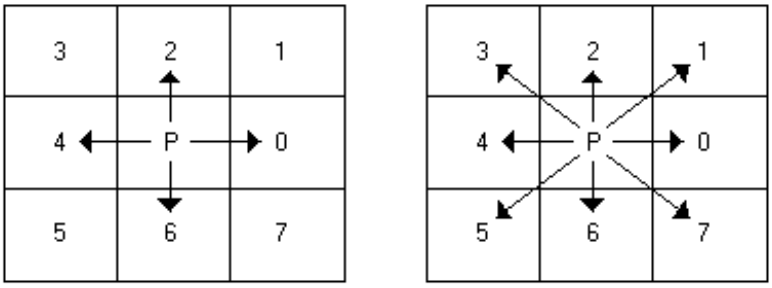
Oblast mívá i další charakteristické vlastnosti, které určují způsob jejího vyplnění. Pro vyplnění může být použitá jedna barva, šrafování nebo lze nanášet opakovaně vzorek složený z více barevných bodů. Tyto vlastnosti se snadněji aplikují na oblasti, které mají geometricky určené hranice.

Úkolem algoritmů pro rasterizaci oblasti je nalézt všechny body uvnitř oblasti a nastavit jejich barvu.

1.1.4.1 Vyplňování oblasti, jejíž hranice je vykreslená v rastru

Pro vyplňování oblasti mající definované hranice v rastru se používá tzv. metoda **semínkového vyplňování**. Její podstata spočívá v definici polohy semínka (vybraný vnitřní bod oblasti), od kterého se postupně rozšiřuje prohledávání okolních bodů. Těm pixelům, které spadají do oblasti, se nastaví nová barva.

Každému rastrovému bodu *P* připadá v rastrovém obraze osm sousedních bodů, které se dají očíslovat 0,...,7. Sousední body 0, 2, 4 a 6 se nazývají přímí sousedé bodu *P*, protože jimi reprezentované obdélníky mají s výchozím bodem společnou hranu. Sousední body 1, 3, 5 a 7 se nazývají nepřímí sousedé bodu *P*. Podle směru prohledávání a vyplňování se oblasti rozlišují na **4-spojité** (4-souvislé) a **8-spojité** (8-souvislé). U 4-spojité oblasti (resp. 8-spojité) lze každé dva body spojit postupností 4 (resp. 8) sousedních bodů z této oblasti – viz obr. 14.



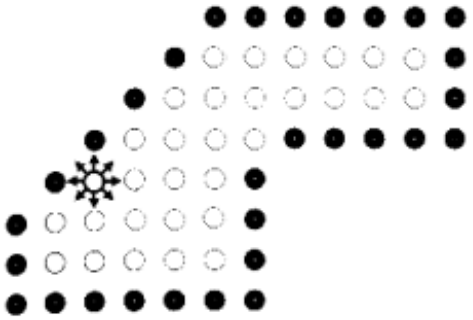
Obr. 14. 4-spojitý a 8-spojitý způsob vyplňování oblastí

Použitý způsob vyplňování oblasti přímo závisí na jejím typu. Pokud bude oblast ohraničena pouze vodorovnými a svislými hranicemi, lze použít i 8-spojité vytváření posloupnosti pixelů. Pokud však budou hranice určeny i diagonální hranicí a k vyplnění se použije 8-spojité vytváření posloupnosti pixelů, algoritmus může vyplnit i rastrové body mimo zvolenou hranici (obr. 15).

Při semínkovém vyplňování se postupuje od zadaného semínka a zkoumá se, zda jeho sousední bod patří dovnitř oblasti. Podmínka, na základě které se toto rozhodování provádí, je barva bodu. Existuje několik možností:

- **hraniční vyplňování** – testovaný bod je vnitřní, pokud má jinou barvu než je barva hranice,

- **záplavové vyplňování** – testovaný bod je vnitřní, pokud má stejnou barvu jako zadané semínko,
- **měkké vyplňování** – testovaný bod je vnitřní, pokud má výrazně jinou barvu než hranice oblasti.



Obr. 15. 8-spojitý způsob vyplňování oblastí, jejíž hranice jsou i diagonální

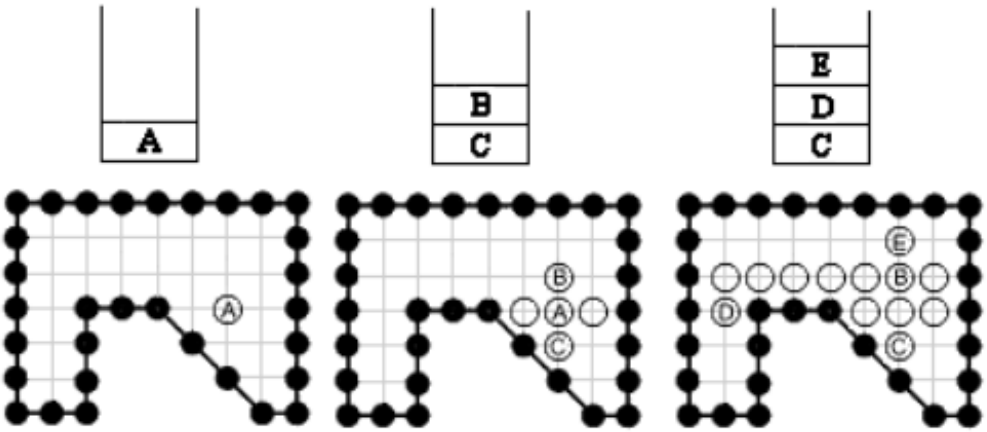
Jednoduché semínkové vyplňování je rekurzivní algoritmus, který by se dal v jazyce C zapsat funkcí

```
void Vypln (int x, int y, Color stara_barva, Color nova_barva)
{
 if (read_pixel (x,y) == stara_barva)              // test barvy pixelu
  {
   write_pixel (x, y, nova_farba);                 // zabarví pixel novou barvou
   Vypln (x, y-1, stara_barva, nova_barva);        // rekurzivně volá sám sebe
   Vypln (x, y+1, stara_barva, nova_barva);
   Vypln (x-1, y, stara_barva, nova_barva);
   Vypln (x+1, y, stara_barva, nova_barva);
  }
}
```

Tento algoritmus vypadá jednoduše a efektivně, ve skutečnosti se ale nepoužívá, protože je pomalý. Šíření semínek do všech směrů způsobuje, že každý vnitřní bod oblasti je testován několikrát i poté, co již byl obarven.

Efektivnější způsob představuje **řádkové semínkové vyplňování**. Hlavní rozdíl proti předchozí metodě spočívá ve vyhledávání sousedních bodů v jednom řádku, který se nazývá úsek. V dalším kroku se pak hledají nové body nad tímto řádkem nebo pod ním. Tento postup není rekurzivní a navíc se používá zásobník, do kterého se podle potřeby ukládá poloha některých vnitřních bodů vyplňované oblasti.

Postup řádkového semínkového vyplňování znázorňuje obrázek 16. Startovním semínkem je bod označený písmenem *A*. Ten se uloží do zásobníku a při prvním průchodu je vybrán. Vyhledá se vodorovná úsečka mezi nalezenými hranicemi. Pak jsou testovány body nad a pod touto úsečkou. Získají se tak body *B* a *C*, které se uloží do zásobníku. Při dalším průchodu se vede vodorovná úsečka bodem *B*. Získají se tak body *D* a *E*., které se uloží do zásobníku, a vše se znovu opakuje. O tom, kde se vede úsečka, rozhoduje bod, který leží na vrcholu zásobníku.



Obr. 16. Postup řádkového semínkového vyplňování

Podobně jako vyplňování oblastí jednou barvou, lze oblasti plnit barevným vzorem. Problém nastává, pokud je ve vzoru obsažena původní barva uvnitř oblasti, protože v těchto případech dochází k zacyklení, neboť výše uvedené algoritmy stále přepisují totéž místo. Tehdy se musí provést rozšíření zásobníku, který bude obsahovat nejen polohu semínka, ale i jeho velikost ve vertikálním a horizontálním směru a dále příznak směru, odkud se řádkové vyplňování k semínku dostalo. Při vkládání nového semínka do zásobníku se zapisované údaje porovnávají s údaji celého zásobníku. Pokud je nalezena položka obsahující semínko ve stejném vodorovném úseku, ale s opačným směrem šíření vyplňování, je třeba úsek jednorázově vyplnit a vypustit ze zásobníku.

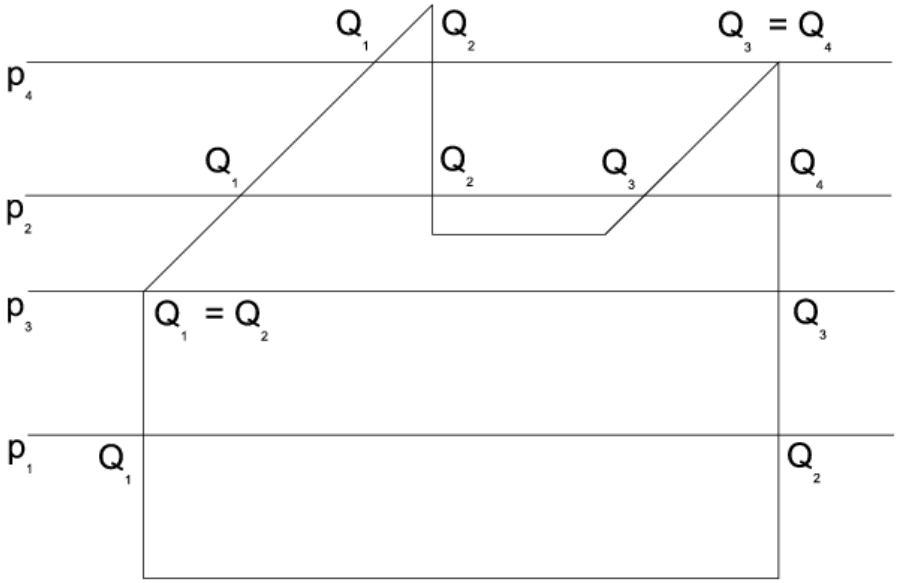
1.1.4.2 Vyplňování oblasti, jejíž hranice je určena geometricky

Základní metodou vyplňování oblasti, která má hranice určeny geometricky, je tzv. **paritní řádkové vyplňování**. Každým řádkem rastru je vedena vodorovná čára a jsou hledány průsečíky s hranicí oblasti. Nalezené průsečíky na jedné čáře se seřadí podle *x*-ových souřadnic a dvojice těchto průsečíků definují úsečky uvnitř oblasti.

Předpokládejme polygon podle obr. 17. Pro přímku p_1 platí, že daný polygon protíná v bodech Q_1 a Q_2 a úsečka Q_1Q_2 leží uvnitř polygonu. Podobně úsečka p_2 protíná polygon v bodech Q_1, Q_2, Q_3 a Q_4 a úsečky Q_1Q_2 a Q_3Q_4 leží uvnitř polygonu.

Tento algoritmus vychází z předpokladu, že přímka protíná polygon v sudém počtu průsečíků. Oproti tomu přímka p_3 protíná polygon v lichém počtu průsečíků – Q_1, Q_2 a Q_3. Došlo by tedy k nesprávnému vyplnění. Řešení tohoto problému spočívá v tom, že se do každé strany nezařadí jeden z krajních bodů (počáteční nebo koncový). V případě úsečky p_4 je třeba navíc pro každou úsečku ověřit, zda leží uvnitř nebo vně polygonu. Provádí se to například testováním středu dané úsečky, zda leží uvnitř nebo vně polygonu.

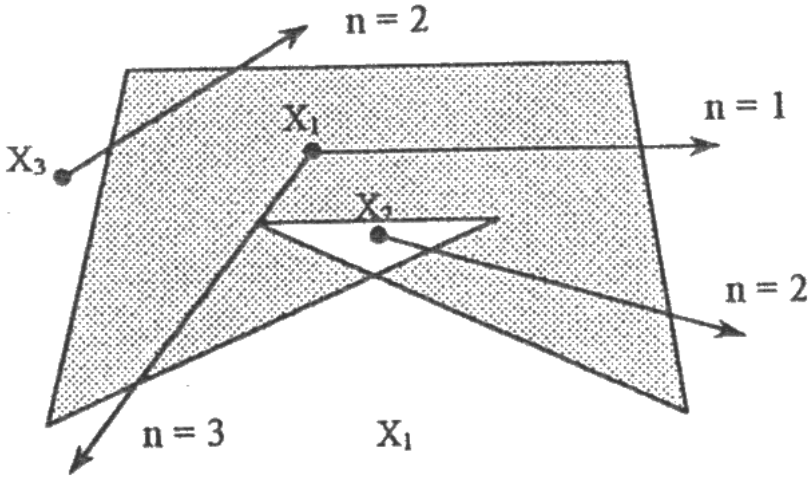
Tímto algoritmem se vyplnila oblast horizontálními čarami. V případě vyplnění oblasti čarami pod libovolným úhlem lze upravit algoritmus tak, že vyšrafovaný objekt se otočí, aby se dal šrafovat horizontálními čarami, a poté se otočí do původní polohy.



Obr. 17. Polygon vyplňovaný přímkovými úseky

1.1.4.3 Vyplňování složitějších objektů

Intuitivně lze pod pojmem polygonální oblast rozumět prostor uvnitř uzavřené lomené čáry. V případě oblasti na obr. 18 se na první pohled tento vnitřní prostor těžko určuje. V těchto případech platí následující pravidlo: bod *X* je vnitřním bodem polygonální oblasti tehdy, pokud lze sestavit polopřímku z bodu *X* tak, že protíná hranice oblasti v lichém počtu průsečíků. Dle obr. 18 je bod X_1 vnitřním bodem oblasti, naproti tomu body X_2 a X_3 nejsou vnitřními body.



Obr. 18. Zjišťování vnitřních bodů oblasti

1.2 Barvy a barevné modely

Pod pojmem barva si lze představit vlastnost nějakého objektu (úsečky, kružnice, textu apod.). Různé barvy, které se používají při vytváření obrazu, se skládají z několika základních barev, které tvoří tzv. **barevný model**. Například na obrazovce počítače se výsledná barva skládá ze tří složek – červené (**R** *red*), zelené (**G** *green*) a modré (**B** *blue*). Každá základní barva je na obrazovce reprezentovaná hodnotou intenzity 0 až 255 (případně 0 – 1). Hodnota 0 znamená, že barva není zastoupena, maximální hodnota určuje, že barva nabývá své největší intenzity.

Počet možných kombinací barev reprezentovaných trojicí bytů je 256^3, což dělá celkem 16 777 216 barev. V případě, že obrazovka není schopna tolik barev vykreslit, se provádí redukce tak, aby lidské oko zaznamenalo co nejmenší ztrátu kvality obrazu.

V barevném modelu RGB se bílé barvy dosáhne tehdy, pokud mají všechny barvy maximální intenzitu (R = 255; G = 255, B = 255). Naproti tomu se černé barvy dosáhne nastavením intenzity RGB hodnot na minimum (R=0; G=0, B=0). Pokud by se intenzity jednotlivých složek barev měnily stejně od 0 do 255, získala by se posloupnost 256 odstínů šedi. Při redukci barevného obrazu na odstíny šedi nelze vytvořit průměry RGB složky. Lidské oko je totiž na jednotlivé barvy různě citlivé. Nejvíc je citlivé na zelenou, pak červenou a nejméně na modrou barvu. Celkový jas se tedy počítá podle vzorce

$$I = 0{,}299R + 0{,}587G + 0{,}114B\,. \tag{34}$$

Z programátorského hlediska je zřejmé, že pojem barva může být různorodý. Barva může například představovat pouze jednobitovou informaci (černá a bílá), osmibitové číslo (256 barev) až po různé zápisy barevných složek. Pro reprezentaci tří barev ve 2 bytech (65 536 kombinací) se používá označení **high color**, pro zápis ve 3 bytech pak označení **true color**.

1.2.1 Barevné modely

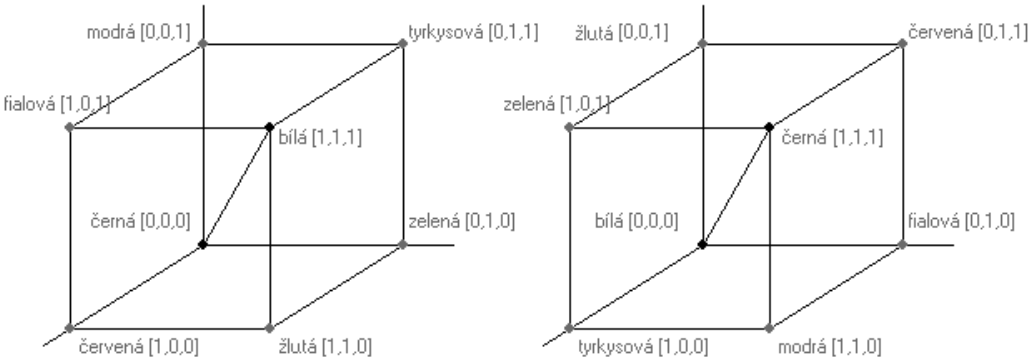
Popis barvy modelem RGB není jedinou možností. V roce 1905 vytvořil Albert H. Munsell systém třídění barev, zohledňující lidské vnímání. Munsellova notace vychází z toho, že barvu je možné popsat třemi základními vlastnostmi – odstínem, jasem a sytostí.

Při rozkladu bílého světla na barevné složky se rozlišuje 6 základních vlastností barev: fialová (*magenta* – M), modrá (*blue* – B), modrozelená (*cyan* – C), zelená (*green* - G), žlutá (*yellow* – Y) a červená (red – R).

Pomocí těchto barev lze vytvořit většinu ostatních barev, přičemž každý z barevných modelů má své charakteristické vlastnosti (pravidla pro jejich míchání apod.). Pro již zmíněný barevný model RGB je charakteristické **aditivní skládání barev** – čím více barev se sečte, tím je výsledek světlejší. Pokud roste svítivost obrazovky nelineárně, používá se tzv. gama korekce.

Barevný model RGB si lze prostorově představit jako jednotkovou krychli umístěnou v osách označených postupně *r*, *g* a *b* (obr. 19 vlevo). Počátku souřadnic odpovídá černá barvá a vrcholu o souřadnicích [1, 1, 1] zase barva bílá. Vrcholy krychle, které leží na osách představují základní barvy a zbývající vrcholy reprezentují doplňkové barvy. Každému barevnému vektoru v RGB reprezentaci odpovídá jeden bod krychle. Například žlutá se získá složením červené a zelené (odpovídá bodu se souřadnicemi [1, 1, 0]). Odstíny šedi odpovídají bodům na diagonále krychle, která spojuje černý a bílý vrchol.

V počítačové grafice se kromě modelu RGB lze setkat i s kombinací **RGBA**. Tento model má ještě čtvrtý parametr definující průhlednost. Každý barevný bod pak s sebou nese i informaci, která určuje, v jakém poměru se barva bodu smíchá s barvou bodu na pozadí. Hodnota 0 znamená neprůhledný barevný bod, hodnota 1 zcela průhledný. Hodnota parametru *A* (nazývá se též alfa-kanál) nemá význam při zobrazení obrazu, ale používá se pouze při kombinaci více obrazů do jednoho celku.



Obr. 19. Reprezentace barevných modelů RGB (vlevo) a CMY (vpravo)

1.2.1.1 Barevný model CMY (CMYK)

Zatímco model RGB se používá pro práci na počítači a je založen na aditivním skládání barev, lze se v praxi setkat s opačným postupem, tzv. **subtraktivním** skládání barev (např. v malířství nebo tiskařské technice). Podstata je taková, že každým přidáním pigmentu je výsledná barva tmavší. Složením všech barev vznikne černá.

Těmto účelům vyhovuje model **CMY**, který obsahuje tři základní barvy – modrozelenou (*cyan* – **C**), fialovou (*magenta* – **M**) a žlutou (*yellow* – **Y**). Tento model lze také popsat jednotkovou krychlí (obr. 19 vpravo), která má podobné vlastnosti jako krychle RGB – je jejím jakýmsi doplňkem. Převod mezi těmito dvěma barevnými modely je jednoduchý. Pokud je vektor barvy v RGB určený [*r*, *g*, *b*], je možné vektor [*c*, *m*, *y*] v modelu CMY vypočítat jako

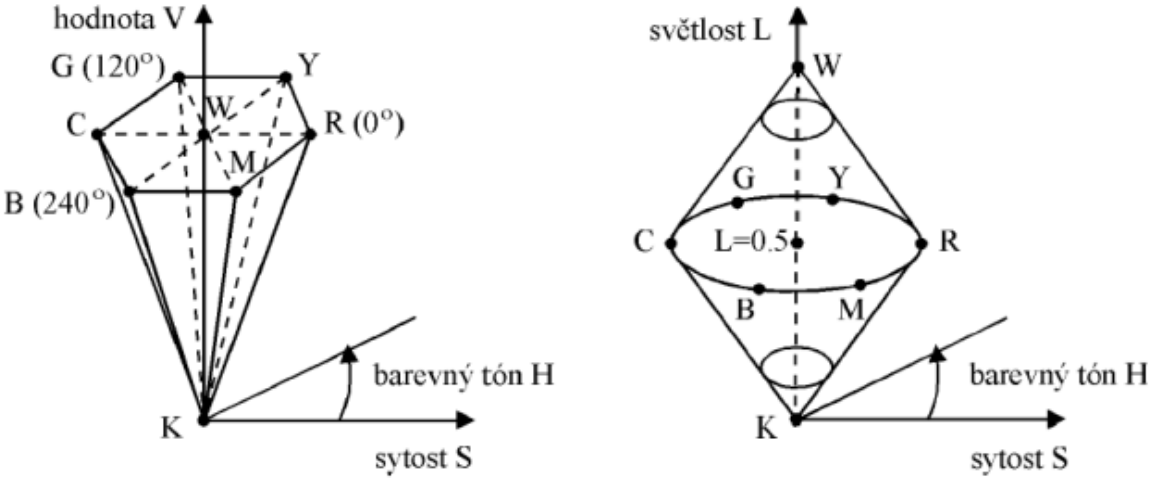
$$\begin{bmatrix} c \\ m \\ y \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} - \begin{bmatrix} r \\ g \\ b \end{bmatrix}. \tag{35}$$

Při tisku se často uplatňuje model **CMYK**, což je CMY, ke kterému je přidaná černá barva. K rozšíření došlo, neboť se při tisku výsledná barva nemíchá, ale postupně se nanášejí jednotlivé barvy. Tento postup ale způsobuje, že složením všech tří barev nevzniká žádoucí černá, ale pouze tmavě hnědá. Z tohoto důvodu se černá barva přidává zvlášť a zvlášť se i tiskne.

1.2.1.2 Barevné modely HSV a HLS

Modely RGB, případně CMY jsou vhodné pro zpracování počítačem. Pro člověka je však daleko obtížnější vložit konkrétní barvu v těchto modelech. Například se mohou u jednotlivců lišit představy o tom, jaká barva vznikne smícháním základních složek. Pro tyto účely vznikly další barevné modely, které jsou blízké lidskému (intuitivnímu) vnímání barev.

Tyto modely se popisují také trojicí složek, které však nepředstavují základní barvy. Model **HSV** má parametry: barevný tón (**H** – *hue*), sytost (**S** – *saturation*) a jasová hodnota (**V** – *value*). Tón barvy určuje převládající spektrální barvu, sytost určuje příměs dalších barev a jas množství bílého (bezbarvého) světla. Pro prostorové zobrazení modelu se nepoužívá krychle, ale šestiboký jehlan (obr. 20 vlevo), jehož vrchol leží v počátku souřadnic. Parametry *S* a *V* se mění od 0 do 1, *H* reprezentuje úhel v intervalu 0 - 360 °. Ve vrcholu jehlanu je černá barva, směrem k podstavě narůstá jas a ve středu podstavy je barva bílá. V jednotlivých vrcholech podstavy jsou umístěny základní barvy (R, G, B, C, M a Y).



Obr. 20. Reprezentace barevných modelů HSV (vlevo) a HLS (vpravo)

Nedostatkem modelu **HSV** je jeho jehlanovitý tvar, který způsobuje šestiúhelníkový tvar ve vodorovném řezu (konstantní hodnota jasu V). Tento nedostatek odstraňuje model **HLS**, jehož reprezentace má kuželový (resp. dvojitě kuželový) tvar, jak je patrné na obr. 20 vpravo. Název modelu je odvozen z barevného tónu (**H** – *hue*), světlosti (**L** – *lightness*) a sytosti (**S** – *saturation*). Barevný tón je opět vyjádřen úhlovou hodnotou, světlost se mění od 0 (černá v dolním vrcholu) do 1 (bílá v horním

vrcholu). Sytost se pohybuje také v rozmezí 0 – 1 (obvody podstav kuželů). Nejjasnější čisté barvy mají tedy souřadnice $S = 1$ a $L = 0{,}5$.

Modely HSV a HLS umožňují postupně měnit barevné charakteristiky při zachování ostatních vlastností barvy. Modely jsou tak vhodné i např. pro vzorníky barev, neboť jsou intuitivní pro uživatele k definici barev jako je sytost, světlost a dominantní barva.

1.2.1.3 Modely pro televizní techniku

S dříve uvedenými modely se lze setkat u většiny grafických programů. Méně známá je skupina modelů které se používají při spojení počítačové a televizní techniky (animace, multimédia, filmové triky apod.).

K modelům pro přenos televizních signálů v normě patří **YUV** (PAL) a $\mathbf{YC_BC_R}$ (SECAM). Jejich společným znakem je oddělení jasové složky od barevné. Toto uspořádání dovoluje využít stejný jasový signál **Y** pro barevné i černobílé televizory. Zbývající dva signály nesou informace o velikosti barevných složek obrazu. Model YC_BC_R se navíc používá při zápisu rastrových obrázků ve formátu JPEG. Tento model je definovaný podle standardu CCIR 601, což je norma pro digitalizaci obrazu.

Vztahy pro konverzi mezi uvedenými barevnými modely lze nalézt např. v [13].

1.3 Zpracování obrazu

Pod pojmem obraz je možné rozumět průmět reálného světa na sítnici lidského oka (fotografie, obraz). Matematickým modelem obrazu je tzv. **obrazová funkce**, která je definována jako

$$z = f(x, y), \tag{36}$$

kde x, y jsou souřadnice bodu ve dvojrozměrném prostoru, ve kterém nabývá funkce hodnoty ze svého definičního oboru. Definičním oborem této funkce je spojitý interval I z R^2. Z matematického hlediska může z nabývat libovolnou hodnotu, v praxi ale bývá tato proměnná určena konkrétní aplikací. Zpravidla se hodnota reprezentuje jediným číslem (např. jas nebo intenzita barvy). V počítačové grafice jsou to nejčastěji tři (nebo čtyři) hodnoty barev některého z modelů RGB (RGBA), CMY, HSV apod. – viz kap. 1.2.

Hodnota funkce z představuje důležitou informaci o obrazovém bodě. Pokud je definiční obor funkce dostatečně velký, lze si vybrat přesnější hodnotu. Proto například obraz s true color barvami poskytne více informací, než kdyby byl obraz jen 16 barevný. Nejčastěji se přitom používají barevné palety s 2^8, 2^{16} a 2^{24} barvami, případně 2^8 stupni šedi.

Při vytváření obrazu je obor hodnot funkce omezen nejen vlastní aplikací, ale i fyzikálními vlastnostmi technického zařízení, které se používá. Nevýhodou, která vyplývá z matematického

modelu a vlastnosti obrazu, je skutečnost, že téměř nikdy není k dispozici spojitý definiční obor funkce (36). Většinou se používá bodový rastr, který se skládá ze základních obrazových elementů – **pixelů**. V trojrozměrném prostoru je základním objemovým elementem **voxel**.

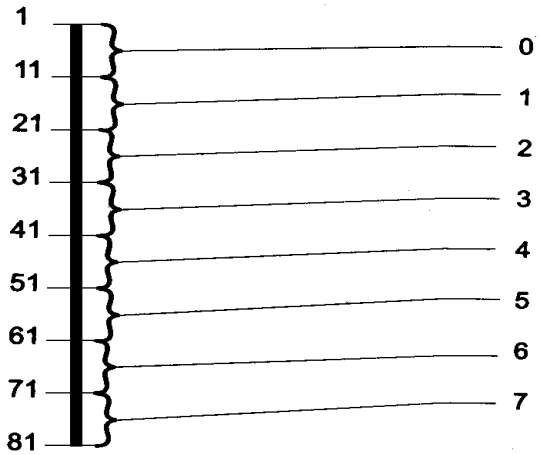
Obrazová funkce (36) je definovaná na podmnožině dvojrozměrného prostoru. Pro jednoduchost bývá někdy názornější ukázat princip v jednorozměrném definičním oboru. Proto se v dalším textu někdy uvádí dvojrozměrná funkce $f(x, y)$ a její odpovídající diskrétní funkce $I(x, y)$, jindy jednorozměrná funkce $f(x)$ a její diskrétní obraz $I(x)$.

1.3.1 Digitalizace

Při získávání digitálního obrazu dochází k přechodu od spojité funkce $f(x, y)$ k diskrétní funkci $I(x, y)$, a to jak v definičním oboru funkce $f(x, y)$, tak i v jejím oboru hodnot. Tento proces se nazývá **digitalizace** a odehrává se ve dvou nezávislých krocích – kvantování a vzorkování.

1.3.1.1 Kvantování

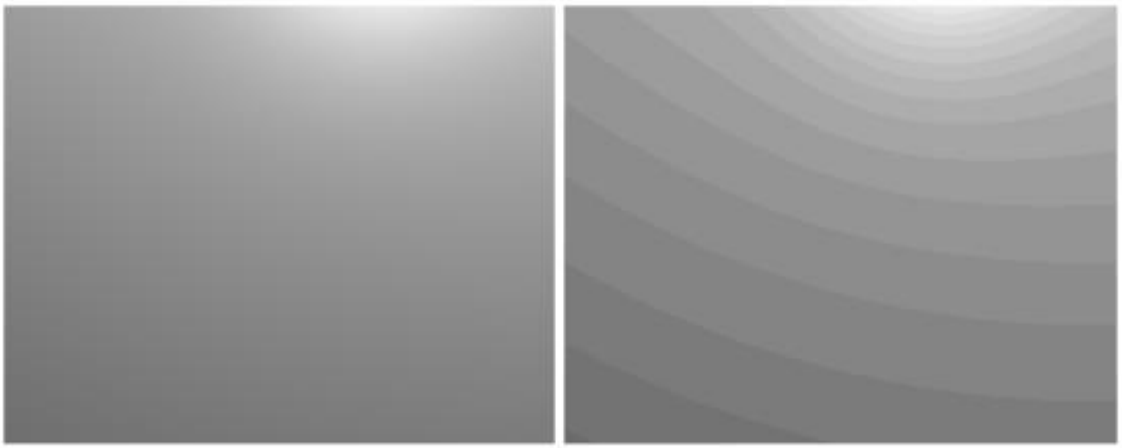
Podstatou kvantování je diskretizace hodnot obrazové funkce (36). Obor hodnot funkce se rozdělí na intervaly, kterým se pak přidělí jediná zástupná hodnota – viz obr. 21. Pokud je délka intervalu konstantní, hovoří se o kvantování **uniformním**. Při neuniformním kvantování se používá proměnná délka intervalu. To je vhodné pro převod obrázku s velkým počtem barev na barevné obrázky s menším počtem barev. Neuniformní kvantování se používá méně často a způsob výběru zástupců intervalu závisí na použití obrazu. Obvykle se používá průměr hodnot z celého intervalu, vážený průměr, medián apod.



Obr. 21. Kvantování

Při kvantování může dojít k tzv. **kvantizační chybě**. Její příčinou je náhlá změna barev, pokud v intervalu, který se diskretizuje, je hladká změna barev, která se nahrazuje jedinou vypočítanou

hodnotou. Takovým příkladem je obrázek 22. Při kvantování dochází ke skokovým přechodům, na které je lidské oko citlivé, a chyba tak působí velice rušivě. Částečně lze kvantizační chybu eliminovat neuniformním kvantováním, případně adaptivním výběrem zástupné hodnoty.

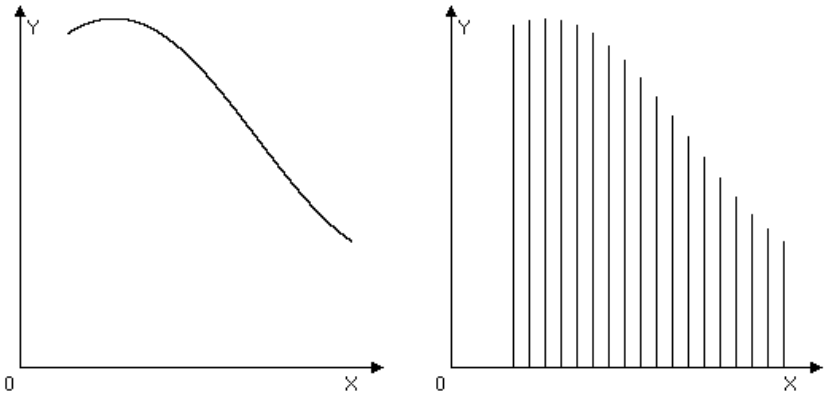


Obr. 22. Chyba kvantování. Vlevo je hladký přechod nahrazený skokovým přechodem (vpravo)

1.3.1.2 Vzorkování

Vzorkování (*sampling*) spojité funkce $f(x, y)$ představuje zjišťování hodnot (vzorků) v pravidelném intervalu. Lze tak chápat i přechod od spojitého na diskrétní případ – viz obr. 23, čímž se získá nová funkce $I(x, y)$. Přitom se využívá skutečnosti, že pixel není bod, ale plocha nenulové velikosti, kterou reprezentuje jedna hodnota. Tento proces se také nazývá bodové vzorkování (*point sampling*).

Méně často se lze setkat s plošným vzorkováním, kde se celé oblasti přiřazuje jediná hodnota na základě výpočtu, nejčastěji průměrná hodnota dané oblasti. Tato metoda je náročná na výpočet (při vzorkování spojité funkce se používá integrace po ploše), což je obtížněji realizovatelné. Z tohoto důvodu se v praxi plošné vzorkování aproximuje několika body, které jsou rozprostřeny uvnitř plochy funkce $f(x, y)$, a výsledná barva se určí jako vážený průměr jednotlivých vzorků. Tato metoda se nazývá vzorkování s vyšší frekvencí (*supersampling*).



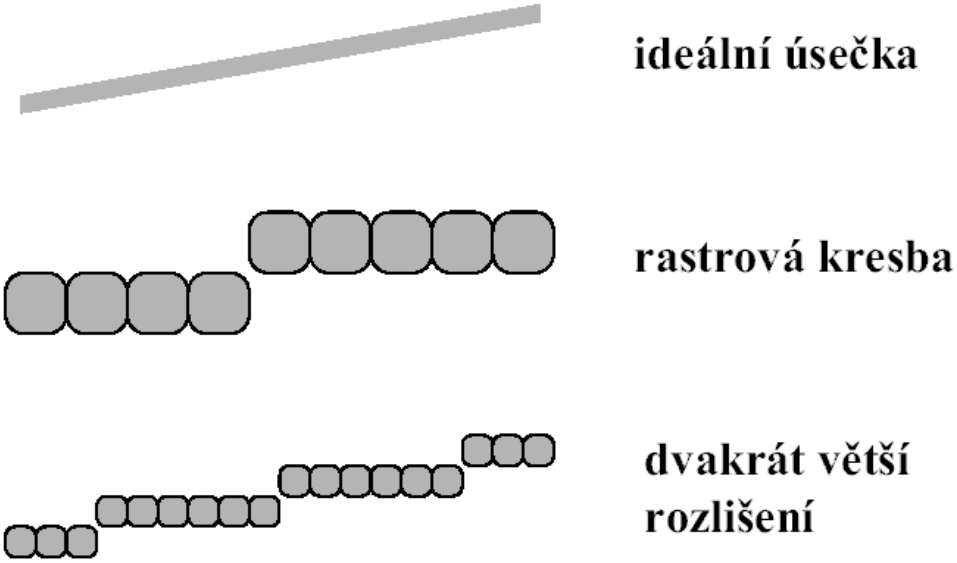
Obr. 23. Vzorkování spojité funkce (vlevo) na diskrétní (vpravo)

Příkladem vzorkování jednorozměrného signálu je digitální nahrávka na kompaktním disku. S určitou frekvencí je ze spojitého zvukového signálu zaznamenána jeho okamžitá hodnota, která se na disk ukládá jako číslo. Čím je vzorkovací frekvence vyšší, tím je původní signál reprezentován přesněji, ovšem za cenu vyšší paměťové náročnosti.

1.3.2 Alias a antialiasing

Výraz **alias** v grafice představuje nechtěný optický jev, kaz obrazu vznikající při podvzorkování spojité funkce. Setkat se s ním lze například při kreslení úseček do rastru. Diagonální úsečka ve čtvercovém rastru se skládá ze stejného počtu bodů jako při jejím horizontálním průmětu na osu *x*, což způsobuje, že diagonální úsečka je jasově řidší a nevypadá hladce (obr. 24). Je to způsobeno rasterizací obrazu.

Dalším příkladem, kde je možné se v počítačové grafice s aliasem setkat, je vzorkování pravidelných textur. Při vzorkování šachovnice se může stát, že všechny vzorky zasáhnou pouze černé políčko a výsledkem je pak jednobarevná (černá) plocha. Dalším příkladem mohou být vzdálené objekty, jejichž velikost se blíží jednomu pixelu – v takovém případě cyklicky nastává jejich objevování se a mizení.



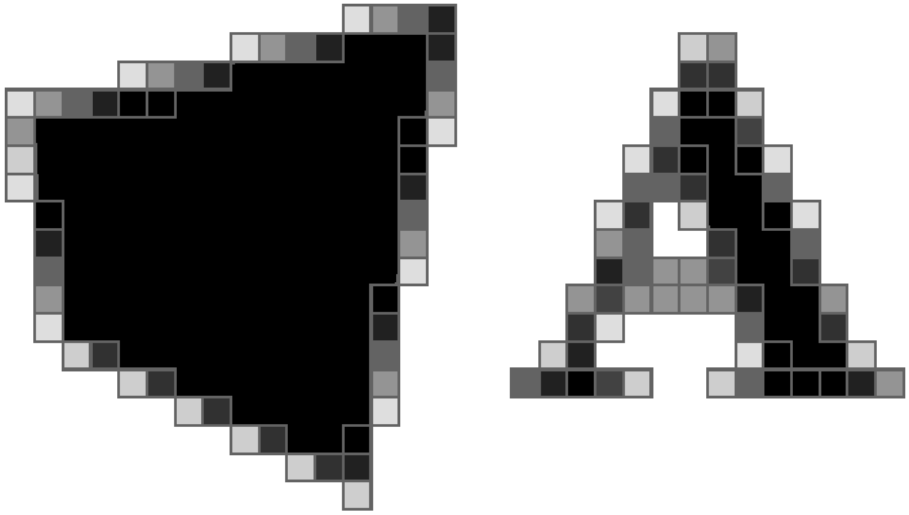
Obr. 24. Porovnání úsečky s její rasterizací, při které dochází k aliasu

Alias nelze odstranit, ale je možné ho částečně potlačit (nejčastěji vyšší frekvencí). Tímto se zabývá tzv. **antialiasing**, který v počítačové grafice představuje vyhlazování hran. Základní myšlenka vyhlazování spočívá v předpokladu, že obrazový bod má nenulovou plochu. Pixely i ostatní kreslené objekty jsou tedy plošné útvary. K vykreslení se pak použije více odstínů dané barvy, jejíž intenzita je

úměrná ploše pokryté části. Na obrázku 25 je příklad pokrytí pixelů úsečky. V tomto případě se vždy vykreslují pixely, mezi kterými úsečka prochází. Intenzita okolních bodů se určuje podle vzdálenosti středu pixelu od úsečky (DDA či Bresenhamovým algoritmem).

Obr. 25. Pokrytí plochy pixelů úsečky

V případě mnohoúhelníku nebo textu (obr. 26) se kreslí všechny pixely, do jejichž plochy objekt zasahuje. Intenzita okrajových pixelů se opět počítá podle vzdálenosti. Časová náročnost vyhlazování je však poměrně náročná, proto se při vykreslování v aplikacích většinou povoluje nebo zakazuje, zda se má antialiasing při kreslení používat nebo ne.



Obr. 26. Pokrytí plochy mnohoúhelníku a textu

1.3.3 Reprezentace rastrového obrazu

1.3.3.1 Matice

Nejjednodušší reprezentací diskrétního obrazu je dvojrozměrná matice bodů (pixelů), z nichž každý nabývá hodnot podle typu obrazu. Pokud je obraz monochromatický, stačí, když je každý pixel na obrazovce popsán jedním bitem, přičemž obraz nemusí být dán kombinací barev černou a bílou, ale libovolnými dvěma barvami.

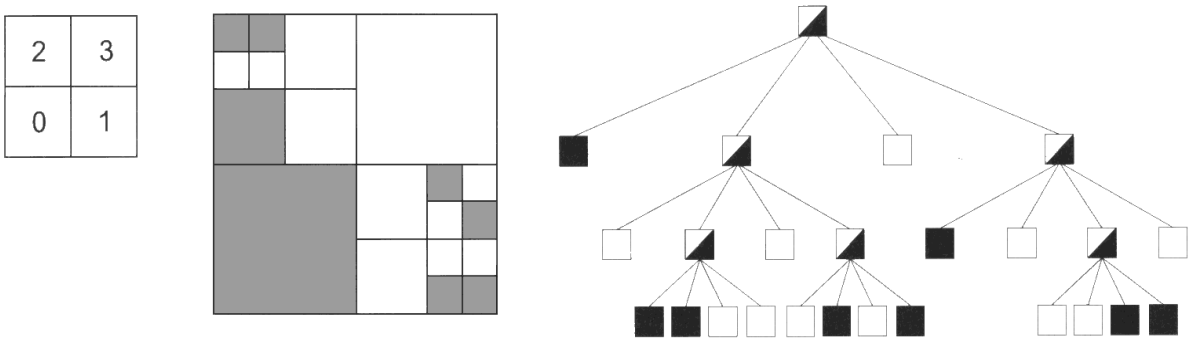
Pokud má obraz 256 barev, používá se pro jejich reprezentaci tzv. **indexový mód**. U takového obrazu nereprezentuje hodnota pixelu přímo barvu, ale je ukazatelem do tabulky – barevné palety. Tou je obvykle převodní tabulka o velikosti 768 bytů (3 x 256 - každý pixel má 3 byty, které definují přímo barvu z 2^{24} možností). Barevná paleta je buď součástí obrazu nebo se vytváří až při jeho zobrazování. V případě, že je obraz reprezentován odstíny šedi, pak může být každý z bodů reprezentován přímo příslušným odstínem (*static gray*).

Dalším případem je obraz, ve kterém je každý pixel reprezentován třemi barvami (*true color* mód) nejčastěji v modelu RGB. Takový obraz obsahuje přímo barevné hodnoty jednotlivých pixelů. Pro uchování barev lze použít i tzv. *direct color* obraz, který pracuje také s RGB hodnotami. Tyto hodnoty však neslouží pro přímé zobrazení, ale jsou odkazem do barevných palet pro každou jednotlivou barevnou složku. S obrazem jsou tedy spojeny tři palety, pro každý barevný kanál jedna. Tento způsob se používá zejména pro jednoduchou změnu všech barev, aniž by se měnily hodnoty pixelů v obrazu.

1.3.3.2 Kvadrantový strom

Nevýhodou maticové reprezentace je paměťová náročnost. To se projeví zvlášť u velkých obrazů, z tohoto důvodu byly hledány cesty úspory s přihlédnutím ke skutečnosti, že se v obraze mohou nacházet stejně velké barevné plochy. Pro takové (černobílé i barevné) obrazy je vhodná reprezentace **kvadrantového stromu**. Algoritmus vytvářející kvadrantový strom rozděluje obraz na pravidelné čtverce a testuje jejich pokrytí pixely stejné barvy. Pokud je čtverec těmito pixely pokryt, dělení končí, v opačném případě se dělí dál. Výsledkem pro černobílý obraz je rekurzivní datová struktura, která se skládá ze tří typů uzlů:

- prázdný uzel (*void, V*) – v příslušném čtverci se nachází pouze pixel bílé barvy,
- plný uzel (*full, F*) – v příslušném čtverci jsou pixely černé barvy,
- smíšený uzel (*mixed, M*) – čtverec se skládá z bílých i černých pixelů.



Obr. 27. Příklad kvadrantového stromu

Výsledný strom může být tvořen rekurzivní procedurou, která vrací hodnotu *V* nebo *F* podle toho, zda je daný čtverec černý nebo bílý. Kromě toho může ještě vracet *M*, pokud se má čtverec rozložit dále na čtvrtiny. Na obr. 27 je znázorněno číslování kvadrantů a příklad rozdělení obrazu. Kvadrantový strom lze v tomto případě zapsat jako

M F M V M F F V V V M V F V F V M F V M V V F F V,

kde za každým smíšeným uzlem (*M*) následuje čtveřice symbolů *V* a *F*, případně další symbol *M*. Na první pohled jde o zápis nepřehledný, ovšem rekurzivně je snadno zapisovatelný i čitelný, protože se jedná o lineární seznam reprezentující průchod kvadrantovým stromem od kořene stromu a pokračuje se jeho uzly zleva doprava. Používá se i jiná forma zápisu, kdy se symbol *M* nahrazuje dvojicí závorek

(F (V (F F V V) V (V F V F)) V (F V (V V F F) V)).

Tato metoda se dá snadno zobecnit a používá se často ve 3D grafice pro vyjádření obsazenosti prostoru (oktálový strom – viz kap. 2.2.3) pro urychlení metody sledování paprsku.

1.3.4 Komprese rastrových obrazů

Již zmíněnou nevýhodou rastrových obrazů je jejich veliká paměťová náročnost, která kvadraticky narůstá s jejich rozlišením. Z tohoto důvodu byla věnována značná pozornost kompresi obrazů. Na rozdíl od komprese obecných souborů se u obrazu často dosahuje vyšších kompresních poměrů, což je dáno charakteristickými rysy konkrétních rastrových souborů.

Existuje velké množství kompresních metod. Většina z nich je spojena s konkrétním grafickým formátem (viz kap. 1.4). Příčin rozsáhlosti kompresních metod a grafických formátů je několik – formáty odrážejí technický vývoj (jde zejména o zvyšující se barevné možnosti grafických adaptérů), spousta grafických programů používá „vlastní“ grafické formáty, které odpovídají aplikačním požadavkům apod.

Tab. 1. Kompresní metody a jejich vlastnosti

Kompresní metoda	**Zkratka**	**Ztrátová**	**Příklad formátu**
Run lenght encoding	RLE	ne	PCX
Huffmanovo kódování	CCITT	ne	TIFF
Lempel – Ziv – Welch	LZW	ne	GIF, PNG, ZIP…
Diskrétní kosinová transformace	DCT	ano	JPEG

Obecně lze rozdělit komprese do dvou skupin. První z nich je **bezztrátová**, při které nesmí dojít k žádným ztrátám nebo změnám obrazových dat. Naopak u **ztrátové** komprese je tolerováno pozměnění barevných hodnot pixelů, čímž dochází ke zhoršení kvality výsledného obrazu. Důvod

použití ztrátové komprese spočívá v dosažení vyššího kompresního poměru než u komprese bezztrátové.

V tabulce 1 jsou uvedeny nejčastěji používané kompresní metody, které zde budou podrobněji popsány. U všech obrazových formátů se komprese týká pouze obrazových dat. Ostatní části grafických souborů (hlavička, definice palety apod.) se zapisují v nekomprimovaném tvaru.

1.3.4.1 Run lenght encoding

RLE kódování je jednoduchá a pro velkou skupinu obrázků i efektivní metoda, která se používá při kódování např. BMP a PCX souborů. Vychází se z předpokladu, že se opakují hodnoty sousedních pixelů. Do souborů se pak zapisuje počet opakujících se totožných hodnot a poté samotná hodnota. Například posloupnost bytů 8, 8, 8, 8, 8, 8, 8, 8, 8 se zapíše jako 9, 8.

V praxi se tento způsob kódování pixelů definovaných jedním bytem zpravidla realizuje tak, že opakování se rozezná hodnotou nejvyššího bitu. Pokud je nastaven na jedničku, reprezentuje byte počet opakování (hodnota bytu je dána jedničkou nejvyššího bitu + počet opakování), za nímž následuje byte se samotnou hodnotou. Pokud má byte nejvyšší bit nastavený na nulu, představuje samotný pixel bez komprese.

Z uvedeného vyplývá, že efektivita kódování klesá při zpracování hodnot s nenulovým nejvyšším bitem. Proto je vhodné uspořádávat paletu barev tak, aby byly nejméně často používané odstíny umístěny v nejvyšších řádcích palety. Také platí, že RLE komprese je vhodná pro rozlišení 1 či 8 bitů na pixel. Při vyšším počtu barev totiž není zajištěna opakující se posloupnost sousedních pixelů.

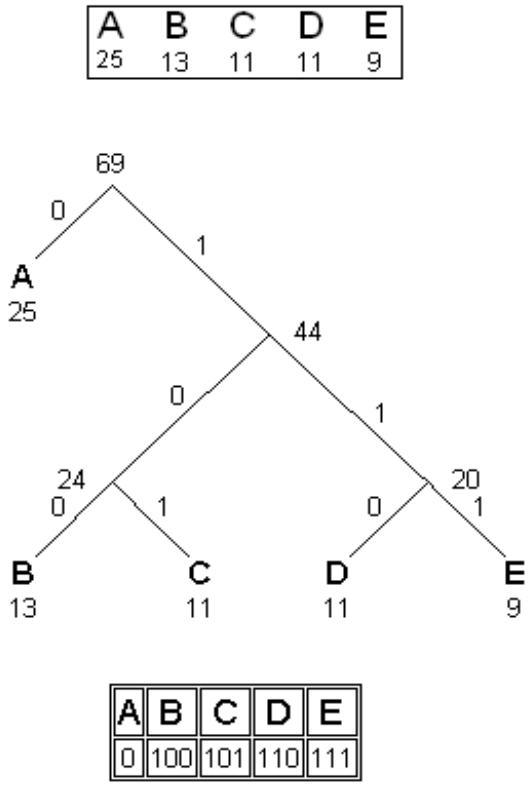
Pokud se v kódovaném obrázku nevyskytují opakující se hodnoty sousedních pixelů, dochází k tzv. záporné kompresi, tj. ke zvětšení výsledného souboru. Z tohoto důvodu se RLE komprese užívá zejména na obrázky kreslené „ručně“ nebo ilustrace s většími stejnobarevnými plochami.

1.3.4.2 Huffmanovo kódování

Jedna z prvních metod kódování byla Shannon-Fanova metoda vyvinutá C. Shannonem z Bell Laboraroties a R. M. Fanem z M.I.T. K vytvoření lepšího kódování jim stačilo pouze znát výskyty znaku v datech. Problém dekódování znaku různých délek vyřešili pomocí dekódovacího stromu (obdoba binárního stromu). Trochu nespravedlivě však byli zastíněni D. A. Huffmanem, který v roce 1952 publikoval zcela stejnou metodu lišící se pouze ve zdokonaleném způsobu výstavby stromu. Huffmanovo kódování je dodnes hojně používané, používají ho některé archivační programy a často je implementován i hardwarově např. v modemech.

U Huffmanova kódování se strom vytváří odspodu a kódy odzadu (na rozdíl od Shannon-Fanovy metody). Na začátku je třeba znát četnost jednotlivých symbolů, ze kterých se vytváří listy

stromu. Vezmou se dva znaky s nejnižší četností výskytu a spojí se do nového uzlu. Symbol vlevo dostane na konci 0, vpravo 1. Posléze se nahradí společným otcem s četností rovnající se součtu četností jeho synů. To celé se opakuje tak dlouho, dokud je co spojovat.



Obr. 28. Princip sestavení kódovací tabulky na základě četnosti znaků

Na obr. 28 je příklad sestavení kódovací tabulky. Na počátku je tabulka četnosti jednotlivých znaků, ze kterých se sestavuje dekódovací strom. Z dekódovacího stromu se konečná tabulka odvozuje na základě binárních číslic, které tvoří jeho větve od kořene až po samotný znak. V kódovací tabulce je každý znak určen svojí binární hodnotou, která se zarovnává zpravidla na 8 bitů. Nyní je zakomprimován blok dat a k jeho opětovnému obnovení je potřeba mít celý strom, který byl použit při komprimaci. Z tohoto důvodu se do výsledného souboru ukládá společně s kódovací tabulkou.

Dekomprimace probíhá tak, že se převede znak na binární reprezentaci a postupuje se stromem (1 - doleva, 0 - doprava). Pokud se dojde až do listu, zapíše se znak, který obsahuje, a začíná se opět od kořene stromu hledat další znak.

1.3.4.3 LZW komprese

Tato tři písmena označují jména tvůrců tohoto algoritmu (Lempel-Ziv-Welch). Zpočátku tento způsob komprese sestavili pouze první dva, kteří vytvořili algoritmus LZ77 a pak i LZ78. Welch o několik let později jejich algoritmus upravoval pro potřeby jiných řadičů.

Princip je založen podobně jako metoda RLE na opakující se posloupnosti. Rozdíl je ten, že RLE zpracovával několikrát se opakující znak a LZW několikrát se opakující posloupnost znaků. Tedy například řetězec „leze po železe“ by se algoritmem LZ77 zakomprimoval jako „leze po že[10,4]“, což znamená, že na místo čísel uvedených v hranaté závorce se při dekomprimaci užijí 4 znaky z předcházejících deseti.

Z výše uvedeného je již patrné, že je tento algoritmus vhodnější pro nejrůznější soubory - nejen pro obrazy, ale třeba i texty, programy a zvuky. Komprimační algoritmus LZ77 používá tzv. posuvné okno, jež má dvě části. Lze si představit, že část textu určená ke komprimaci je v jakési tabulce – onom okně a jeho rozdělení na dvě části slouží k tomu, aby se zjistilo, zda text ve druhé části je shodný s tím v první části. Na uvedeném příkladu by ve druhé části byl úsek slova – „leze“ a na začátku prvního také slovo „leze“. Hranice mezi první a druhou částí okna se posouvá, stejně jako se posouvá ono celé posuvné oko, a po každém posunutí se zjišťuje, zda se text na začátku obou oken shoduje. Celé posuvné okno má však jen jistou délku a to znamená, že pokud by ony totožné posloupnosti znaků byly od sebe příliš vzdálené tak k jejich zakomprimování vůbec nedojde. Pokud by totiž celé posuvné okno bylo příliš dlouhé, mohla by komprimace trvat až několik hodin, i když by dosáhla velmi dobrých poměrů. Proto je obvykle možné komprimační program nastavit podle toho, zda je pro uživatele podstatnější rychlost komprese nebo výsledné zmenšení. Čím je komprese větší, tím je poměr lepší, avšak doba komprimace delší.

Algoritmus LZ78 a LZW (prakticky se od sebe neliší) využívají tzv. **adaptivní slovníkovou metodu**. Program si vytvoří pomocný soubor, tzv. slovník, který je využíván při komprimaci a dekomprimaci. Tento slovník je však po zabalení možno odstranit, jelikož k rozbalení není potřeba. Dekomprimační algoritmus je schopen si slovník opět vytvořit na základě zabalených dat. Komprimační algoritmus tedy prochází posloupnosti znaků a zjišťuje, jestli jsou obsaženy ve slovníku. Pokud ne, tak je tam přidá a přiřadí jim jistou hodnotu (podstatné je, že tato hodnota je menší než těchto několik znaků). Na výstup pak zapíše původní znaky (výstup je stejný jako vstup). Pokud však komprimační algoritmus tyto znaky ve slovníku našel, tak k nim přičte ze vstupního souboru další znak a znovu testuje jestli jsou ve slovníku, pokud uspěje i teď, přičítá další a testuje znovu až do doby, kdy tato testovaná sekvence znaků ve slovníku není. Pak tedy tuto posledně testovanou sekvenci do slovníku zapíše, stejně jako by ji tam nikdy nenašel a na výstup zapíše hodnotu, která byla ve slovníku této sekvenci znaků přiřazena (a která, jak bylo řečeno, je menší, než by bylo těchto několik znaků a tím dochází ke komprimaci). Aby nebylo dosaženo gigantických rozměrů, je algoritmus obvykle naprogramován tak, že až slovník dosáhne určité velikosti, tak se prostě v průběhu komprimace smaže a začne se budovat od začátku. Výmaz slovníku není na závadu. Pouze při následné komprimaci již není možné využít data ze smazaného slovníku.

Tato metoda dosahuje velice příznivých poměrů a používá ho spousta komprimačních programů (ZIP, ARJ apod.). Navíc je také dobře použitelná pro přenos po síti, protože lze data začít odesílat již před dokončením komprimace a na druhé straně již mohou být postupně dekomprimována.

1.3.4.4 Diskrétní kosinová transformace a JPEG

Metody RLE a LZW nejsou efektivní při kompresi plně barevných obrázků s mnoha barevnými přechody. Byla tedy vyvinuta metoda, při níž je kompresní poměr řízen požadavkem na výši kvality dekomprimovaného souboru. V praxi se ukazuje, že snížení kvality na 75% je pro většinu uživatelů nepozorovatelné, přitom kompresní poměr může v takovém případě být 20:1 až 25:1.

Metoda diskrétní kosinové transformace je vhodná především pro kódování fotografií, u nichž sousední pixely (na řádku či sloupci) mají sice odlišné, ale přesto blízké barvy. Snižování kvality obrazu se projeví potlačováním rozdílu v blízkých barvách. Lidské oko je mnohem citlivější na rozdíly v úrovních jasu, než na změny barevných odstínů. je tedy možné snížit objem dat (pro vstup do DCT) zprůměrováním barevných složek několika sousedních pixelů a nahrazením jedinou hodnotou.

Metoda řízené ztrátové komprese využívající diskrétní kosinové transformace byla vyvinuta skupinou JPEG. Tento způsob komprese není vhodný pro obrazy s nižším barevným rozlišením. Na velkých jednobarevných plochách vytváří artefakty v podobě čtverců či pruhů. Optimálním vstupem je obraz ve 24 bitech na pixel.

Diskrétní kosinová transformace je formou diskrétní Fourierovy transformace (např. [11]). Obrazová data jsou považována za barevné vzorky spojitých barevných signálů naměřených v diskrétní síti pixelů. Výsledkem je pak rekonstrukce původního obrazu. Rozklad na jednotlivé signály je výsledkem dopředné transformace, která představuje harmonický analyzátor.

Jednotlivé fáze při kompresi JPEG jsou: transformace barev (zpracování na model YC_BC_R – viz kap. 1.2.1.3), redukce barev (snížení objemu dat zprůměrováním barevných složek několika sousedních pixelů), dopředná diskrétní kosinová transformace, kvantování koeficientů (zde se určuje kvalita komprese) a kódování (užívá se zpravidla Huffmanovo).

1.3.5 Transformace barev

V této části budou probrány rozličné algoritmy, které zpracovávají dvojrozměrné pole pixelů s cílem je barevně transformovat. Půjde zejména o změnu počtu barev, což se užívá při přípravě obrazu pro tisk nebo převodu mezi různými formáty.

Transformace barev je často prováděnou operací. Pomocí ní lze tmavému obrazu dodat jas, nevýraznému kontrast nebo docílit celkové či lokální změny vzhledu obrazu. V těchto případech se jedná zejména o redukci počtu použitých barev.

Snížením počtu barevných odstínů dochází ke ztrátě informace. Existuje několik algoritmů, jejichž cílem je tuto ztrátu minimalizovat. Techniky, které umožňují z několika barev docílit bohaté barevné škály, se nazývají **polotónování** (*halftoning*). S tímto případem se lze setkat zejména u tiskáren, kde je jeden barevný pixel původního obrazu převeden na matici bodů s omezenou velikostí palety. Dochází tak ke zvětšení rozlišení obrazu. Druhou technikou je **rozptylování**

(*dithering*), kdy nedochází ke změně rozlišení obrazu a z pohledu použitých algoritmů jde o speciální případ polotónování. Obě tyto metody využívají schopnosti lidského oka vytvářet z několika blízkých barevných bodů vjem jediného bodu, jehož barva je dána aditivním složením barev původních bodů. Tato vlastnost je základem veškerých metod počítačové grafiky pro generování obrazů s mnoha odstíny a omezenou velikostí barevné palety.

V následujícím textu budou vysvětleny principy převodu šedotónového obrazu (16 odstínů šedi C_{in} = <0, 15>) na černobílý, kdy původní kvalita barev 4 bity na pixel je snížena na 1 bit na pixel (C_{out} = <0, 1>) tak, aby si lidské oko dokázalo z různých kombinací sousedních bodů vytvořit představu několika odstínů šedi. Redukce barevného obrazu by se prováděla obdobně po jednotlivých barevných složkách.

Používá se několik základních metod:

- náhodné rozptýlení
- pravidelné (maticové) rozptýlení
- distribuce zaokrouhlovací chyby.

1.3.5.1 Náhodné rozptýlení

Podstata metody náhodného rozptýlení je velice jednoduchá. Využívá se při ní generátoru náhodných čísel, který produkuje čísla v rozsahu C_{in}. Pokud je hodnota vstupní intenzity nulová, je výstupní také nulová, pokud je hodnota 15, je výstup nastaven na 1. V ostatních případech se generované náhodné číslo porovnává se vstupní hodnotou. Pokud je intenzita jasu větší než náhodné číslo, nastaví se výstup na 1, jinak na 0.

Obr. 29. Porovnání originálního obrazu (vlevo) a černobílého vytvořeného náhodným rozptýlením

V případě, že je výstupní škála barev větší než 2, výše uvedená podmínka se nevyhodnocuje náhradou na 1, ale zvyšuje se o jedničku. Z uvedeného algoritmu vyplývá, že je náhodné rozptýlení jednoduchou a rychlou metodou. Větším plochám dodává vzhled drsného povrchu a výsledné obrazy působí dojmem fotografie s velkým zrněním (obr. 29).

1.3.5.2 Pravidelné (maticové) rozptýlení

Tato metoda se používá pro generování různých odstínů šedi v pravidelných, předem daných vzorcích složených z černých a bílých bodů. Pokud jsou tyto vzorky dobře navrženy, působí výsledný obraz jako kresba s jemným šrafováním.

Původně byla tato metoda vyvinuta pro převod obrazů, kdy se vstupní pixel nahrazoval maticí pixelů výsledných (docházelo ke zvětšení rozlišení). Například pro vstupní rozsah odstínů šedi C_{in}=<0, 4> byly navrženy matice

$$\begin{matrix} \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} & \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} & \begin{bmatrix} 0 & 0 \\ 1 & 1 \end{bmatrix} & \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} & \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}. \\ C_{in} = 0 & C_{in} = 1 & C_{in} = 2 & C_{in} = 3 & C_{in} = 4 \end{matrix} \tag{37}$$

Pro větší rozsah vstupních intenzit lze užít matice vyšších řádů. Zpravidla se dodržuje zásada, že každá následující matice vznikne z předchozí přidáním jedničky na jednu novou pozici. Jiné kombinace uvnitř matice zpravidla způsobují lokální zesvětlení či ztmavení obrazu v závislosti na vstupních odstínech sousedních pixelů.

Aby se nemusely všechny matice trvale udržovat v paměti, lze provést jejich náhradu maticí jedinou. Například pro čtyřnásobné zvětšení obrazu a C_{in}=<0, 4> se používají matice

$$M_d = \begin{bmatrix} 0 & 12 & 3 & 15 \\ 8 & 4 & 11 & 7 \\ 2 & 14 & 1 & 13 \\ 10 & 6 & 9 & 5 \end{bmatrix}; \quad M_p = \begin{bmatrix} 1 & 5 & 9 & 2 \\ 8 & 12 & 13 & 6 \\ 4 & 15 & 14 & 10 \\ 0 & 11 & 7 & 3 \end{bmatrix}. \tag{38}$$

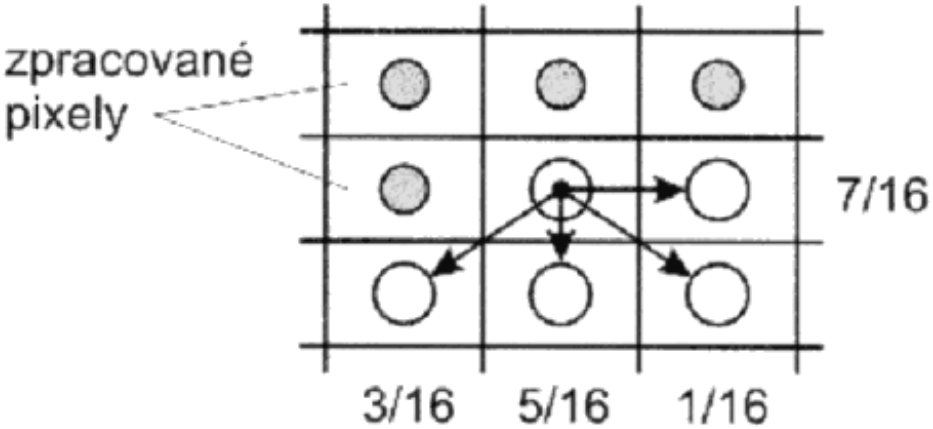
Hodnoty v těchto maticích určují, při jaké velikosti vstupního odstínu šedi se objeví na výstupu jednička a kdy nula. Je zřejmé, že jedničky a nuly lze v rozptylovacích maticích uspořádat mnoha různými způsoby. Pro různé typy zobrazovacích zařízení se skutečně používají různá uspořádání matic. Ve vyjádřeních (38) se matice M_d používá pro displeje a vytváří tzv. křížkový stínovací vzor (obr. 30). Naopak pro tiskárny se generují obrazové vzory s použitím matice M_p. Takto se generují obrazy známé z novin, které jsou složeny z menších či větších teček. Důvodem je skutečnost, že tiskové body se vzájemně částečně překrývají a při tisku pravidelně se střídajících černých a bílých bodů vzniká velmi černý vzorek. Sdružování těchto bodů dokáže tento problém překlenout.

Obr. 30. Porovnání originálního obrazu (vlevo) a černobílého vytvořeného pravidelným rozptýlením se zvětšeninou (vpravo)

1.3.5.3 Distribuce zaokrouhlovací chyby

Distribuce zaokrouhlovací chyby (*error diffusion*) představuje efektivní způsob, jak v co nejvyšší míře provést náhradu intenzity vstupního pixelu na binární hodnotu na výstupu. Podstata této metody je taková, že vstupní hodnota barvy se používá nejen k nalezení nejbližšího stupně intenzity, ale zanedbaná hodnota (chyba), která vznikla, se používá i k modifikaci sousedních pixelů. Například má-li vstupní pixel intenzitu 6, výstupnímu pixelu se přiřadí 0 a zanedbaná hodnota 6 se přičítá k nejbližším sousedním pixelům. Při vstupní intenzitě 12 se naopak přiřadí výslednému pixelu 1 a hodnota 3 se odečítá od okolních pixelů.

Jedna z možností distribuce chyby je uvedena na obr. 31. Při tomto způsobu je třeba, aby se vstupní pixely zpracovávaly po řádcích nebo sloupcích. Pro jeden řádek se vymezuje jednorozměrné pole, které představuje paměť chyby. Před vykreslením každého pixelu se dosud získaná chyba z okolních, dříve zpracovaných pixelů, přičítá k jeho intenzitě a teprve výsledná hodnota se převádí do výstupního rozsahu. Zanedbaná hodnota se opět přičte do paměti chyby.



Obr. 31. Postup výpočtu výsledné barvy u distribuce zaokrouhlovací chyby

Na obr. 31 jsou dále uvedeny koeficienty udávající podíl chyby určený k připočtení k sousedním pixelům. Je patrné, že tento podíl je větší ve vodorovném či svislém směru a naopak menší při směru úhlopříčném. Protože se při výpočtech uplatňuje celočíselný charakter intenzity, je třeba poslední díl distribuované chyby určit ne uvedeným podílem, ale jako rozdíl chyby a součtu všech dříve vypočítaných dílů této chyby.

Obr. 32. Porovnání originálního obrazu (vlevo) a černobílého vytvořeného distribucí zaokrouhlovací chyby se zvětšeninou (vpravo)

1.3.5.4 Barevná paleta

Výše uvedené principy byly uvedeny pro redukci šedotónových obrazů na černobílé. V případě barevné palety je situace obdobná s rozšířením, že vstupní pixel s sebou nese informaci o barevném modelu (nejčastěji RGB). Rozptylovací algoritmus se pak provádí pro každou barevnou složku zvlášť. Dalším rozdílem bývá výstupní barevná paleta, protože na výstupu bývá zpravidla větší počet barev.

Velice častým případem je redukce 24bitové barevné palety na 8bitovou, proto budou následující úvahy vázány na tento případ. Výsledné barvy bývají indexem do palety, ve které lze nastavit libovolných 256 odstínů z 16 milionů možných barevných kombinací. Kromě volby rozptylovací metody vzniká otázka nastavení barevné palety. Požadavkem bývá, aby barevná paleta obsahovala ty barevné odstíny, které se v daném obrazu vyskytují nejčastěji. V úvahu se bere hledisko, zda je celkový obraz při převodu již sestaven nebo se transformace provádí průběžně při jeho tvorbě. K řešení se dá použít standardní paleta 3-3-2 nebo adaptivně vytvářená paleta.

Barevná paleta 3-3-2

Jde o standardní paletu, pro kterou platí, že se do ní umísťují pravidelně vybraní zástupci z barevné krychle RGB (viz kap. 1.2.1). Krychle se tak rozděluje na 8 (2^3) řezů v ose *r*, 8 (2^3) řezů

v ose g a 4 (2^2) řezy v ose b. Počet bitů kódování modré barvy je nižší než ve zbylých dvou složkách, neboť je lidské oko na odstíny modré nejméně citlivé.

Vrcholy vytvořené sítě řezů v krychli RGB představují 256 zástupců do palety, která se nazývá **paleta 3-3-2**. Všechny hodnoty barev pro index $i \in <0;255>$ lze vyjádřit vztahy

$$\begin{aligned} r &= (i >> 5) * 255/7 \\ g &= ((i >> 2) \,\&\, 7) * 255/7 \quad . \\ b &= (i \,\&\, 3) * 255/3 \end{aligned} \tag{39}$$

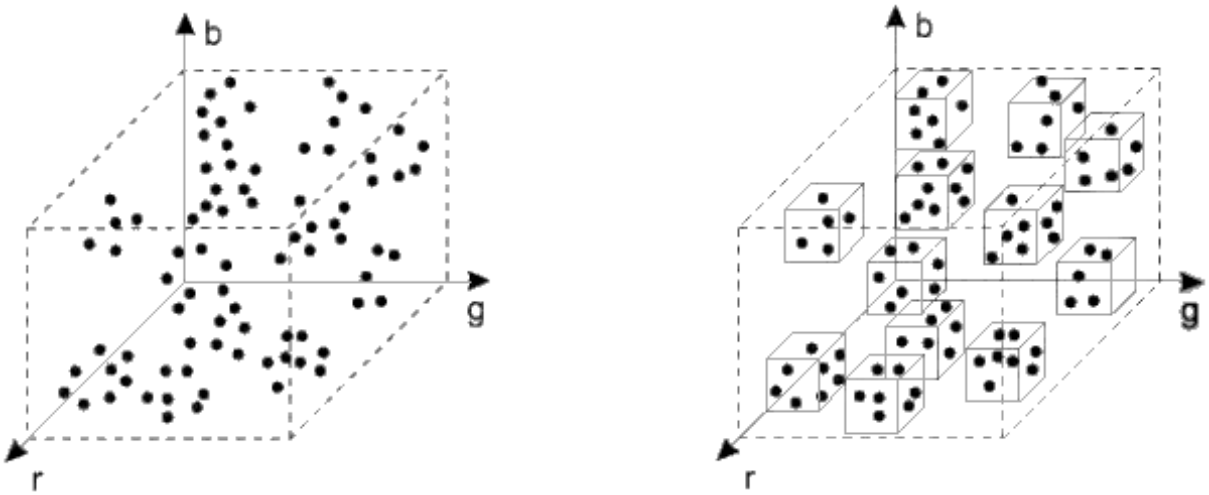
První (nejvyšší) 3 bity každého pixelů obsahují kódy 0 - 7 pro 8 stupňů červené, druhé 3 bity pro 8 stupňů zelené a zbylé 2 bity pro modrou složku. Pro zpětné celočíselné určení indexu i v paletě platí

$$i = (r << 5) + (g << 2) + b\,. \tag{40}$$

Adaptivně vytvářená paleta

Barevná paleta 3-3-2 se používá v případech, kdy není předem známá barevná struktura celého obrazu. Vzhledem k této skutečnosti se tato paleta v řadě případů nepoužívá, protože obsahuje i takové odstíny, které se v obraze vůbec nemusí vyskytovat nebo jsou zastoupeny pouze v minimálním počtu. V ostatních případech je vhodnější použít **adaptivní paletu**, pro kterou je charakteristické, že se pro každý obraz vytváří zvlášť. Její nevýhodou je časová a paměťová náročnost.

Při vytváření se vytváří histogram četností jednotlivých odstínů barev (obr. 33) používaných v celém obrazu. V dalších výpočtech se uplatňuje nejen skutečnost, jaký odstín se v daném obraze vyskytuje, ale i jak často je v něm obsažen. Na rozdíl od jiných technik se neuchovávají složky zvlášť, ale vytváří se trojrozměrná tabulka zpravidla o velikosti 32 (jeden rozměr). Postačí tak uchovávat pět nejvýznamnějších bitů každé základní barevné složky.



Obr. 33. Histogram vstupních odstínů (vlevo) a sjednocení oblastí blízkých barev (vpravo)

V další části algoritmu se na základě histogramu nachází 256 nejvhodnějších zástupců (na obr. 33 jsou vyznačeny tečkami), které vytvoří oblasti obklopující skupiny blízkých barev. Cílem je nalézt oblasti blížící se tvarem krychli a mající podobnou velikost nebo oblasti obsahující stejný počet odstínů.

1.4 Základní grafické a animační formáty

Podobně jako jiná data počítačových programů je třeba i grafické informace ukládat do souborů. V této kapitole je uveden popis nejpoužívanějších grafických formátů. Podrobnější informace jsou uvedeny např. v [8].

K obrovské explozi rastrových grafických formátů došlo přibližně před dvaceti lety, kdy vznikaly uživatelsky přístupné grafické editory. Podobně jako to platí dnes u kancelářských aplikací, každý výrobce si zavedl svůj vlastní formát, nepřevoditelný na cokoliv jiného. Některé formáty přežily (PCX, BMP, ICO, IMG, TGA) a staly se v podstatě standardy, některé se alespoň dočkaly převodníku do standardních formátů, jiné zcela vymizely v propadlišti dějin. Další z nich byly navrženy jako formáty pro přenos dat, aniž by měly za cíl stát se standardy (PNM, MIFF). Poslední kategorii tvoří ty, jež byly již jako standardy navrženy (GIF, TIFF, JPEG, PNG, MNG, IFF, PostScript).

Samostatnou skupinu tvoří také spousta vektorových grafických formátů. Z těch nejrozšířenějších lze jmenovat např. DWG, který jako výchozí používá Autocad, dále pak DXB, DXF, EPS či operačním systémem Windows používaný WMF.

1.4.1 PCX

Tento grafický formát patří mezi nejstarší, přesto se i dnes často používá. Umožňuje kódovat 2,4,8 a 24 bitové obrázky. Na kompresi používá RLE kódování (viz kap. 1.3.4.1) Princip kódovaní je založený na opakování stejných znaků a příznak opakování je charakteristický nastavením prvních dvou bitů na 1 (C0 H). Je-li např. šestkrát za sebou 10 H, uloží se toto do dvou bytů jako C6 H a 10 H (první číslo obsahuje součet počtu opakování s příznakem a druhé příslušnou hodnotu). Tento způsob kódování je tedy výhodný, pokud se jednotlivé byty (pixely) v obrázku opakují.

Formát byl původně optimalizován na 16 barev. Tomu je přizpůsobená i hlavička, která má prostor pro definování maximálně 16 barev. V případě, že je třeba uložit barev více, paleta se ukládá na konec souboru. V případě nejvíce používané 256 barevné palety zabírá na konci 3*256 = 768 bytů, kde každá barva je z RGB modelu. V případě, že má obrázek 24 bitovou barevnou paletu, každá složka z R, G a B se ukládá do samostatného bloku.

Jak vyplývá z předchozího odstavce, uspořádání souboru formátu PCX je takové, že na začátku je hlavička mající velikosti 128 bytů, pak následuje blok grafických dat a na konci může být

uložena paleta barev. V hlavičce jsou uloženy informace o souboru (na počátku je vždy 0AH, který charakterizuje formát PCX a pak následují další informace jako verze formátu, počet bitů na pixel a rozlišení). Ukázka kódu je na obr. 34.

```
00000000: 0A 05 01 01 00 00 00 00|5D 01 4D 02 80 02 E0 01 | ********]*M****ŕ*
00000010: 00 00 00 FF FF FF FF FF|FF FF FF FF FF FF FF FF | ***·************
00000020: FF FF FF FF FF FF FF FF|FF FF FF FF FF DF FF F7 | ************ß·÷
00000030: FF FF FF FF FF FF EF FF|FF FF FF FC 1F FC 03 7F | ******ď····ü*ü**
00000040: 00 01 2C 00 01 00 80 02|E0 01 FF BF FF FF FF FF | **,*****ŕ*·ż····
00000050: FF FF FF FF FF FF 00 FF|FF FF FF FF FF FF FF FF | ******·*********
00000060: FF FF FF FF FF FF FF FF|FF FF FF FF FF FF FF FF | ****************
00000070: FF FF FF FF FF FF FF FF|FF FF FF FF FF FF FF FF | ****************
00000080: EC FF EC FF EC FF EC FF|EC FF EC FF EC FF DA FF | ě·ě·ě·ě·ě·ě·ě·Ú·
00000090: 7F D1 FF DA FF 7F D1 FF|DA FF 7F D1 FF DA FF 7F | *Ń·Ú·*Ń·Ú·*Ń·Ú·*
000000A0: D1 FF DA FF 7F D1 FF DA|FF 7F C1 FF C1 F1 CF FF | Ń·Ú·*Ń·Ú·*Á·ÁńĎ·
000000B0: DA FF 7F C1 FF C1 EE CF|FF DA FF 7F C1 FF C1 EF | Ú·*Á·ÁîĎ·Ú·*Á·Áď
000000C0: 7F CE FF DA FF 7F C2 FF|7F CE FF DA FF 7E 22 3F | *Î·Ú·*Â·*Î·Ú·~"?
000000D0: 7F CE FF DA FF 7F 77 7E|CF FF DA FF 7F 77 7D CF | *Î·Ú·*w~Ď·Ú·*w}Ď
000000E0: FF DA FF 7F AA C1 FB CF|FF DA FF 7F AA C1 F7 CF | ·Ú·*ŞÁűĎ·Ú·*ŞÁ÷Ď
000000F0: FF DA FF 7F 99 C1 EF 7F|CE FF DA FF 7F C1 DD C1 | ·Ú·*™Áď*Î·Ú·*ÁÝÁ
```

Obr. 34. Začátek grafického souboru s příponou PCX

1.4.2 BMP

Tento grafický formát používá operační systém Microsoft Windows. Původně byl určený na 16ti barevné obrázky. Pro časté používání a z důvodu rychlosti celého systému podporuje jen ty nejjednodušší komprese. Jeho charakterickými vlastnostmi je tedy jednoduchost, rychlost a nezávislost na zobrazovacích metodách. Nevýhodou je velikost obrázku.

```
00000000: 42 4D 8A 6E 05 00 00 00|00 00 36 04 00 00 28 00 | BMSn******6***(*
00000010: 00 00 0C 03 00 00 C7 01|00 00 01 00 08 00 00 00 | ******Ç*********
00000020: 00 00 54 6A 05 00 13 0B|00 00 13 0B 00 00 00 01 | **Tj************
00000030: 00 00 00 01 00 00 00 00|00 00 08 08 00 00 10 10 | ****************
00000040: 00 00 00 42 00 00 00 4A|00 00 00 6B 00 00 52 84 | ***B***J***k**R„
00000050: 00 00 5A 84 00 00 00 8C|00 00 5A 8C 00 00 63 94 | **Z„***Ś**ZŚ**c”
00000060: 00 00 00 9C 00 00 21 C6|00 00 00 E7 00 00 00 FF | ***ś**!Ć***ç***˙
00000070: 00 00 08 08 08 00 08 FF|08 00 00 00 10 00 10 08 | ********˙********
00000080: 10 00 10 10 10 00 18 18|10 00 10 FF 10 00 18 18 | ************˙****
00000090: 18 00 29 29 18 00 18 FF|18 00 00 00 21 00 21 18 | **))***˙****!*!*
000000A0: 21 00 21 21 21 00 6B B5|21 00 63 C6 21 00 4A D6 | !*!!!*kµ!*cĆ!*JÖ
000000B0: 21 00 21 F7 21 00 21 FF|21 00 29 18 29 00 29 29 | !*!÷!*!˙!*)*)*))
000000C0: 29 00 29 39 29 00 42 42|29 00 29 9C 29 00 29 FF | )*)9)*BB)*)ś)*)˙
000000D0: 29 00 31 FF 29 00 08 08|31 00 31 31 31 00 B5 31 | )*1˙)***1*111*µ1
000000E0: 31 00 31 B5 31 00 6B C6|31 00 31 FF 31 00 08 08 | 1*1µ1*kĆ1*1˙1***
000000F0: 39 00 39 39 39 00 4A 4A|39 00 39 FF 39 00 42 42 | 9*999*JJ9*9˙9*BB
```

Obr. 35. Začátek grafického souboru s příponou BMP

Každý bitmapový soubor se skládá se čtyř částí. První z nich je bitmapová hlavička o velikosti 14 B (první dva byty jsou vždy znaky BM (42 H, 4D H), následuje celková velikost souboru v bytech a ofset začátku obrazových dat). Druhou částí je informační blok o bitmapě. Ten obsahuje informace o rozlišení a rozměrech obrázku, počtu bitů na pixel, druhu komprese (žádná, či RLE) a počtu barev. Třetím blokem je paleta barev. Ta je definována polem v RGB struktuře, kde jsou barvy uspořádané s rostoucím významem. Paleta má velikost počet barev * 4 byty (3 byty RGB a poslední je nevyužitý). Paleta má význam při maximálním počtu 256 barev, při počtu větším se barvy ukládají přímo do pole dat, které tvoří čtvrtou část souboru.

Ukázka kódu formátu BMP je na obr. 35.

1.4.3 TIFF

Tento formát je příkladem velmi univerzálního grafického formátu. Oblast jeho hlavního použití je typografie a tyto soubory jsou často generované různými kreslicími programy, skenery či kamerami pro počítačové vidění. TIFF je charakterizován velkou pružností, proto jen velmi málo programů dokáže zobrazovat všechny možné varianty tohoto formátu (podporuje velké množství kompresí – např. RLE, LZW, JPEG a barevných modelů).

Struktura souboru je složena ze tří částí. První je hlavička (8 B - v ní je uloženo, pro jaký typ počítače je soubor určen a verze formátu), následují bloky informací určujících interpretaci obrazových údajů a samotné obrazové údaje. Hlavička je vždy jen jedna, druhý a třetí blok se mohou vícekrát opakovat. Tato vlastnost umožňuje ukládat do jednoho souboru více obrázků. Tyto tzv. „dlaždice“ se mohou zpracovávat nezávisle na okolních částech, což má význam např. u velkých obrázků, kdy je třeba editovat jen část. Další vlastností těchto bloků je, že mohou být komprimované dovolenými komprimovanými metodami bez ohledu jako jsou komprimované sousední dlaždice.

1.4.4 GIF

GIF (Graphics Interchange Format) je grafický formát s využitím komprese LZW (viz kap. 1.3.4.3). Podporuje maximálně 8-bitové obrázky (256 barev nebo 256 odstínů šedi). Má dvě verze 0.87 a 0.89a. Kromě samotných obrázků formát GIF podporuje tzv. **animované gify**, což jsou jednotlivé obrázky zobrazované v určitých intervalech. Tento formát se velmi rozšířil i díky internetu, protože umožňoval postupné zobrazování obrázku již po načtení 1/8 svých dat, zatímco jiné formáty potřebovaly na zobrazení načítat celý soubor. Formát však postupem času zastaral. Podnětem byla mimo jiné i skutečnost, že firma, která tento formát vymyslela (*CompuServe Incorporated*), si po čase začala na něj uplatňovat autorská práva.

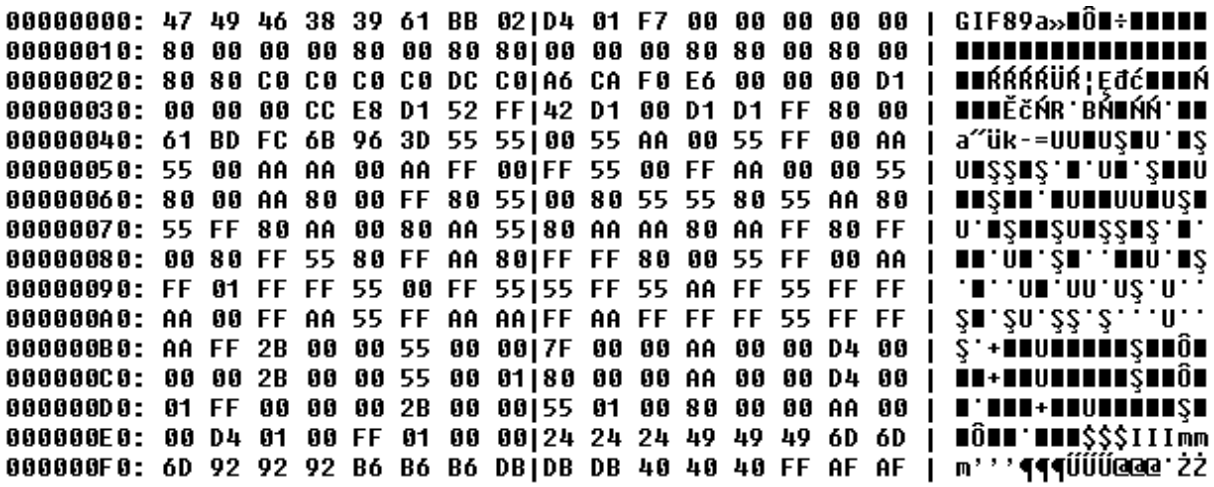
Podobně jako ve formátu TIFF je v GIFu grafický obraz ukládán jako určitý počet různě velkých pravoúhlých obrázků se společným grafickým pozadím. Každý z obrázků je v souboru

reprezentován blokem, ve kterém je definována relativní poloha vzhledem k pozadí, počet barevných odstínů a vlastní obrazová informace.

Struktura souboru GIF je uvedena v tab. 2 a ukázka začátku souboru je na obr. 36.

Tab. 2. Struktura souboru GIF

Délka	Význam	Povinnost
6 bytů	GIF - signatura	povinné
7 bytů	globální deskriptor obrazovky (rozměry obrazovky, počet barev, barevný odstín pozadí…)	povinné
různá	globální tabulka barev (vztah mezi barevným indexem a barvou)	nepovinná
různá	blok popisu obrázku 1	povinný
různá	blok popisu obrázku 2	nepovinný
	……..	
různá	blok popisu obrázku N	nepovinný
1 byte	koncový znak GIFu – 3B H	povinný



Obr. 36. Začátek grafického souboru s příponou GIF

1.4.5 PNG

Formát PNG je poměrně mladý a bývá označován za nástupce GIFu. Přebírá jeho vlastnosti a rozšiřuje je o další. Jde např. o true color obrázky (až 48 bitů na pixel) s bezztrátovou kompresí, obrázky až se 16-bitovými stupni šedi nebo zjišťování poškození souboru.

PNG je určený zejména pro přenos obrazu po síti. Jeho kód je poměrně jednoduše implementovatelný a snadno přenositelný a nevztahují se na něj žádná autorská práva. Další jeho vlastnosti jsou flexibilita, která umožňuje strukturu souboru v budoucnu dále rozšiřovat, a již zmíněná robustnost, která povoluje úplnou kontrolu integrity souboru.

Novinkou tohoto formátu je i zavedení předzpracování každého pixelu. Ten se může ukládat jednou z pěti metod:

- pixel se přímo zakóduje a ukládá
- ukládá se rozdíl hodnot s pixelem nalevo
- ukládá se rozdíl hodnot s pixelem nahoře
- ukládá se průměrná hodnota pixelu a sousedů vlevo a nahoře
- ukládá se průměrná hodnota pixelu a sousedů vlevo, nahoře a vlevo nahoře.

Struktura souboru je následující: identifikační hlavička 8 bytů (všechny byty mají hodnotu uvedenou na obrázku 37), obrazová hlavička (17 bytů, kde je uvedeno rozlišení obrázku, bitová hloubka a typ komprese) a třetí částí je blok vlastních obrazových dat. Na rozdíl od prvních dvou částí se tento blok dat může v souboru opakovat vícekrát. Podrobnější specifikaci PNG souboru lze nalézt např. na http://www.w3.org/TR/REC-png.html.

```
00000000: 89 50 4E 47 0D 0A 1A 0A|00 00 00 0D 49 48 44 52 | %PNG■■■■■■■■IHDR
00000010: 00 00 02 BB 00 00 01 D4|08 02 00 00 00 B5 4B 77 | ■■■»■■■Ô■■■■■µKw
00000020: C7 00 00 00 07 74 49 4D|45 07 D3 07 1C 0A 1C 19 | Ç■■■■tIME■Ó■■■■■
00000030: 7F 1A 80 5D 00 00 00 09|70 48 59 73 00 00 0B 31 | ■■■]■■■■pHYs■■■1
00000040: 00 00 0B 31 01 ED 4C 7E|05 00 00 00 04 67 41 4D | ■■■1■íL~■■■■■gAM
00000050: 41 00 00 B1 8F 0B FC 61|05 00 00 25 82 49 44 41 | A■■±Ź■üa■■■%,IDA
00000060: 54 78 DA ED DD 0B 7A 9B|BA A2 06 50 E7 7C 9D 57 | TxÚíÝ■z›ş˘■Pç|ťW
00000070: 33 B3 A6 33 4B 47 96 E3|84 96 4D 24 90 04 08 10 | 3ł¦3KG-ă„-M$■■■■
00000080: B0 D6 D9 B7 D7 B1 31 E6|61 A3 1F 49 88 97 F7 F7 | °ÖŮ·×±1ćaŁ■I■-÷÷
00000090: F7 07 00 C0 B4 D7 D7 D7|1F CF FF F7 F3 E7 CF A3 | ÷■■Ŕ´×××■Ď˙÷óçĎŁ
000000A0: 97 04 00 68 D4 EF DF BF|9F FF FE E8 FE F8 F3 E7 | -■■hÔďßżź˙ţčţřóç
000000B0: CF D1 CB 03 00 B4 EB 7F|47 2F 00 00 70 02 3F 86 | ĎŃË■■´ë■G/■■p■?†
000000C0: 7F BC BE BE 1F BD 3C 00|40 2B DE DF 5F FB C7 3F | ■ľľľ■˝<■@+ŢB_ŰÇ?
000000D0: E2 97 3F 3E 7E 95 CF EB|E5 F3 FF 7E 1F BD 46 00 | â-?>~•ĎëÍó˙~■˝F■
000000E0: C0 B6 7E 8C 3E FB E7 CF|9F D7 F7 CF FF 65 DF FF | Ŕ¶~Ś>ŰçĎź×÷Ď˙eß˙
000000F0: F6 FE FE 76 F4 3A 00 C0|79 CD 3B 51 7F 79 F9 F8 | öţţvô:■ŔyÍ;Q■yůř
```

Obr. 37. Začátek grafického souboru s příponou PNG

1.4.6 JPG (JPEG)

Grafický formát JPEG nedefinuje strukturu souboru, ale rychlý a spolehlivý komprimační způsob ve 24 bitovém rozlišení. Komprimace se provádí buď programovou podporou (pomalejší) nebo s programově-technickou podporou (speciální čipy). Tento standard byl vytvořen za účelem práce se statickým obrazem a videem, proto se hojně využívá v oblasti multimédií.

Základem metody JPEG je ztrátová diskrétní kosínova transformace (viz kap. 1.3.4.4). Stupeň kvality komprimace se definuje pomocí tzv. Q-faktoru (faktor kvality). V praxi to znamená, že čím

nižší je Q-faktor, tím se dosáhne vyššího komprimačního poměru, ovšem za cenu vyšších ztrát. Tyto ztráty se projevují zejména u rozsáhlých jednobarevných ploch.

Při vytváření JPEG souboru se původní obraz (např. RGB) nejprve transformuje do optimálního barevného modelu, který lépe vyjadřuje barevné závislosti (YUV resp. YC_BC_R - viz kap. 1.2.1.3). Poté následuje redukce počtu pixelů (tzv. převzorkování, kdy např. 2 x 2 pixely se nahradí jedním pixelem). Dalším krokem je již zmíněná diskrétní kosínova transformace. Za ní následuje kvantizace. V tomto kroku vzniká ztráta daná velikostí Q-faktoru. Výsledná data se zakódovaná (používá se Huffmanovo kódování) ukládají do souboru. Při zobrazení dat ze souboru se postupuje opačným způsobem.

1.4.7 DXF

Tento formát se řadí mezi vektorové. Byl zvolen jako světový standard, proto všechny CAD aplikace podporují jeho kompatibilitu. Je tvořen posloupností ASCII hodnot, které definují jednotlivé entity ve výkresu.

Soubor je rozdělen do čtyř částí. První z nich tvoří hlavička, která obsahuje nastavení proměnných celého výkresu. Každá proměnná obsahuje hodnotu podobně, jako jsou např. systémové proměnné Autocadu. Druhá část je složena z tabulek, které obsahují definici prvků (LINETYPE – tabulka typů čar, LAYER – tabulka stylů apod.). Třetí část se nazývá část bloků a popisuje entity, které obsahuje každý blok výkresu. Poslední část je tvořena samotnými entitami. Každá z nich má svoje pole hodnot, která je jednoznačně definuje. Například nultá hodnota identifikuje její typ (může být LINE, POINT, CIRCLE, ARC, TRACE SOLID, TEXT, SHAPE, BLOCK).

Přesnou specifikaci formátu DXF je možné získat např. na adrese http://www.autodesk.com/techpubs/autocad/dxf/.

1.4.8 WMF

Formát WMF je často používán v MS Windows. Písmeno M ve zkratce znamená Metafile, což znamená, že tyto typy souboru mohou obsahovat vektorové i bitmapové části. Pro ukládání se používají standardní funkce Windows GDI (celkově je jich kolem 80 – např. Arc – kruhová výseč, LineTo – kreslení úsečky apod.).

Struktura souboru je následující. Na počátku je hlavičkový blok, udávající základní informace o souboru (velikost, počet objektů, rozměry obrázku, verze Windows, pro které byl vytvořen apod.). Za tímto hlavičkovým blokem následuje jeden či víc údajových bloků. Každý z objektů ve výkresu má svůj blok. V něm jsou uloženy tři hodnoty – velikost bloku, číslo funkce identifikující objekt (viz předchozí odstavec) a parametry příslušné funkce.

Soubor se zpracovává načítáním jednotlivých záznamů a provádí se příslušné funkce s předanými parametry. Ukončovací funkce souboru má číslo 0 a nemá žádné parametry.

1.4.9 FLI/FLC

Formát souboru FLI (někdy také nazývaný Flic) je jedním z nejpopulárnějších animačních formátů prostředí MS-DOS. Používá se velmi často v animačních programech, počítačových hrách a CAD aplikacích, které vyžadují trojrozměrnou manipulaci s vektorovými kresbami. FLI nepracuje s audio nebo video daty, ale místo toho ukládá pouze posloupnosti stálých datových předloh.

FLI je populární díky své jednoduchosti. Dovoluje rychlé animační přehrávání a nepožaduje speciální hardwarové vybavení pro kódování nebo dekódování dat. FLI se hodí nejlépe pro počítačově generované nebo ručně psané animační posloupnosti, zvláště ty, které byly vytvořeny animačními a CAD programy. Tyto předlohy dosahují nejlepších kompresních poměrů právě při použití FLI formátu. Přirozené předlohy z reálného světa mohou být také animovány ve formátu FLI. U těchto předloh se však vyskytuje velké množství šumu, které snižuje schopnost FLI kódovacího algoritmu komprimovat data a ovlivňuje tak rychlost animačního přehrávání. Rovněž platí, že čím je v animaci méně barev, tím lepší je výsledný kompresní poměr.

Existují dva typy FLI animačních souborů. Původní formát FLI (má příponu .FLD, má nejvyšší rozlišovací schopnost 320 x 200 pixelů a je schopen zobrazovat pouze 64 barev. Tento formát byl vytvořen pro program Autodesk Animátor. Nový FLI formát má příponu .FLC, má nejvyšší rozlišovací schopnost 64k x 64k pixelů a je schopen zobrazovat 256 barev. Datová kompresní schémata používaná v souborech FLC jsou mnohem účinnější než schémata používaná v souboru FLI.

FLI animace jsou posloupnosti stálých předloh, nazývaných snímky. Každý snímek obsahuje část animačních dat. Rychlost animačního přehrávání určuje hodnota prodlevy mezi jednotlivými snímky. Data jsou v každém snímku vždy barevně mapována. Každý pixel ve snímku obsahuje hodnotu indexu do barevné mapy definované pro daný snímek. Barvy v mapě se mohou měnit podle potřeby snímek od snímku. Ačkoli je soubor FLC omezen na zobrazení pouhých 256 barev, má každý pixel hloubku 24 bitů, což znamená, že se může vybírat z palety obsahující více jak 16 miliónů barev.

FLI formát pracuje s několika typy datové komprese. Každý snímek FLI animace je komprimován mezisnímkovým delta kódovacím schématem. Tímto schématem se kódují pouze rozdíly mezi sousedními snímky, ne snímky vlastní. Tak vzniknou mnohem menší soubory, než při kódování jednotlivých snímků. Komprimovaná data se rovněž rychleji dekomprimují a zobrazují.

První snímek každé FLI animace je komprimován celý použitím RLE kódovacího algoritmu. Je totiž třeba někde začít, protože v následujících krocích se již kódují pouze rozdíly mezi sousedními snímky. Pokud je snímek delta kódován a komprimovaná data jsou větší než data nekomprimovaná (což je možné zejména u dat se spoustou šumu), potom se snímek ukládá v nekomprimované podobě.

1.4.10 MPEG

Specifikace MPEG (*Motion Picture Experts Group*) je určena pro zakódované datové proudy, které obsahují komprimovanou audio a video informaci. MPEG byl navržen speciálně se záměrem ukládání zvukových a pohybových dat na standardních kompaktních discích (CD) a digitálních zvukových páskách (DAT). V současné době se MPEG používá zejména pro archivaci audio a video dat na CD-ROM nosiče pro multimediální systémy, které vyžadují schopnost ukládat a přehrávat zpět vysoce kvalitní audio a videomateriál pro komerční, výukové i zábavné účely. Nový standard MPEG-2 dovoluje přenos MPEG dat i přes televizní kanál a kabelovou síť.

Na většině systémů se používá speciální hardware pro sejmutí MPEG dat z "živého" video zdroje v reálném čase a ve vzorkovací frekvenci dosahující 30 snímků za sekundu. Každý snímek video dat je pak komprimován a uložen jako MPEG datový proud. Je-li rovněž použit zvukový zdroj, je také zakódován a smíšen (multiplexingem) s video proudem s několika přidanými informacemi, sloužícími k synchronizaci těchto dvou proudů pro přehrávání.

MPEG je postaven na digitálních televizních standardech, specifikovaných v CCIR-601, používaných v USA. Typické rozlišení je zde 720 x 576 a vyžaduje větší šířku datového pásma, než může MPEG se svými max. 1.86Mbit/sec povolit. Standardní televizní obrázky proto musí být zmenšeny v poměru 2:1 do formátu 352 x 240. Evropské (PAL, SECAM) a japonské normy se liší v mnoha ohledech, včetně rychlosti zobrazování (30 snímků/s v USA, 25 snímků/s v Evropě). Přehrávač MPEG proto musí být schopen rozlišit a zpracovat širokou škálu variant videosignálu.

MPEG formát nemá definovanou pevnou strukturu. Vše, co se požaduje pro přehrávání MPEG dat, je zakódováno přímo do datového proudu. Tudíž není nutná žádná hlavička či jiný obal. MPEG používá asymetrický kompresní model. MPEG používá dva typy kompresních metod pro zakódování videodat: kódování **interframe** a **intraframe**. Kódování interframe je založeno na prediktivním kódování a interpolační kódovací technice. Když se sejme snímek v rychlé frekvenci (obvykle 30 snímků/sec), bude mezi dvěma sousedními snímky velké množství identických dat. Pokud si je kompresní technika vědoma tohoto faktu, nemusí kódovat celý snímek, jak činí metoda kódování intraframe. Místo toho jsou zakódovány a komprimovány pouze rozdíly v informacích mezi jednotlivými snímky. Výsledkem je vyšší kompresní poměr s daleko menší potřebou dat, která mají být kódována. Tento typ interframe kódování je také nazýván prediktivní kódování.

2. ZÁKLADY PROSTOROVÉ GRAFIKY

2.1 Promítání

V počítačové grafice se zobrazují trojrozměrné objekty na dvojrozměrném zobrazovacím zařízení. Transformace charakterizující převod trojrozměrného objektu do dvojrozměrné reprezentace se nazývá promítání. Při promítání dochází ke ztrátě prostorové informace a tím i k možnému zkreslení názoru pozorovatele na skutečný tvar objektu. Proto jsou pro určité obory vybírány různé způsoby promítání a jsou doplňovány dalšími pravidly a postupy pro zvýšení reálného vjemu promítnutého objektu (řešení viditelnosti těles při zobrazování, ortogonální průměty, axonometrie apod.). Rovinné projekce se dělí na dvě základní skupiny - rovnoběžné (paralelní) a perspektivní promítání. Rovnoběžné promítání je charakterizováno směrem promítání (všechny promítací paprsky mají stejný směr), perspektivní promítání je určeno středem promítání (promítací paprsky vycházejí z jediného bodu). Pro oba druhy promítání lze obvykle volit umístění rovinné průmětny v prostoru. Úlohu promítání lze rozložit na tyto dílčí kroky:

1. Volba souřadných systémů (světový souřadný systém WCS 1 (*World Coordinate System*) a souřadný systém průmětny VCS 2 (*Viewing Coordinate System*)).
2. Formulace promítací úlohy, tj. výběr geometrických parametrů (bodů, vektorů, vzdáleností, souřadných systémů), s jejichž pomocí lze jednoznačně určit pozici pozorovatele (kamery), průmětny, směru promítání aj.
3. Stanovení transformace, která popíše promítání prostorových bodů do průmětny.
4. Nalezení lineární (nebo obecné) transformace mezi souřadnými systémy WCS a VCS a její vyjádření v maticovém tvaru.

Cílem je nalezení jedné nebo více transformačních matic (např. 4 x 4), které definují transformace (homogenních) souřadnic promítaných bodů podle vztahu

$$[x_p, y_p, z_p, w_p] = [x, y, z, 1]T_{proj}\,, \tag{41}$$

kde $P_p = [x_p, y_p, z_p, w_p]$ jsou VCS souřadnice promítaného bodu [x, y, z, 1] a T_{proj} je matice 4 x 4 použitá při transformaci promítání. Postupné transformace (více transformačních matic) se používají tehdy, když jsou během promítání prováděny ještě další výpočty (např. ořezávání) a tyto nelze nebo není vhodné řešit v cílovém souřadném systému průmětny.

2.1.1 Rovnoběžné promítání

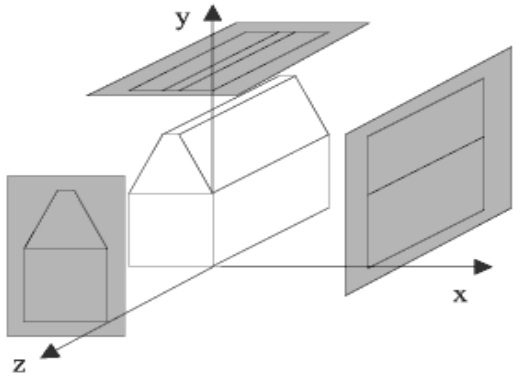
Nejjednodušší typ rovnoběžného promítání je promítání do některé z rovin $x = x_0$, $y = y_0$, $z = z_0$ ve směru příslušné osy kolmé na zvolenou promítací rovinu - průmětnu. Velmi často budou x_0,

y_0 nebo z_0 = 0, průmětnou bude některá z hlavních rovin *xy*, *xz* nebo *yz*. Projekce lze popsat jednoduchou transformací v homogenních souřadnicích:

$$T_{xy} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & z_0 & 1 \end{bmatrix}, \; T_{xz} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & y_0 & 0 & 1 \end{bmatrix}, \; T_{yz} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ x_0 & 0 & 0 & 1 \end{bmatrix}. \tag{42}$$

Skupina projekcí ve směru hlavních os do průměten v hlavních rovinách, která zahrnuje nárys a podle potřeby bokorysy, půdorys (pohled shora), spodní pohled a pohled zezadu, se nazývá Mongeova projekce. Příklad je uveden na obr. 38.

Mongeovo promítání je speciálním případem kolmého promítání. Promítací paprsky jsou kolmé na průmětnu, mají směr shodný s normálovým vektorem průmětny.



Obr. 38. Mongeova projekce

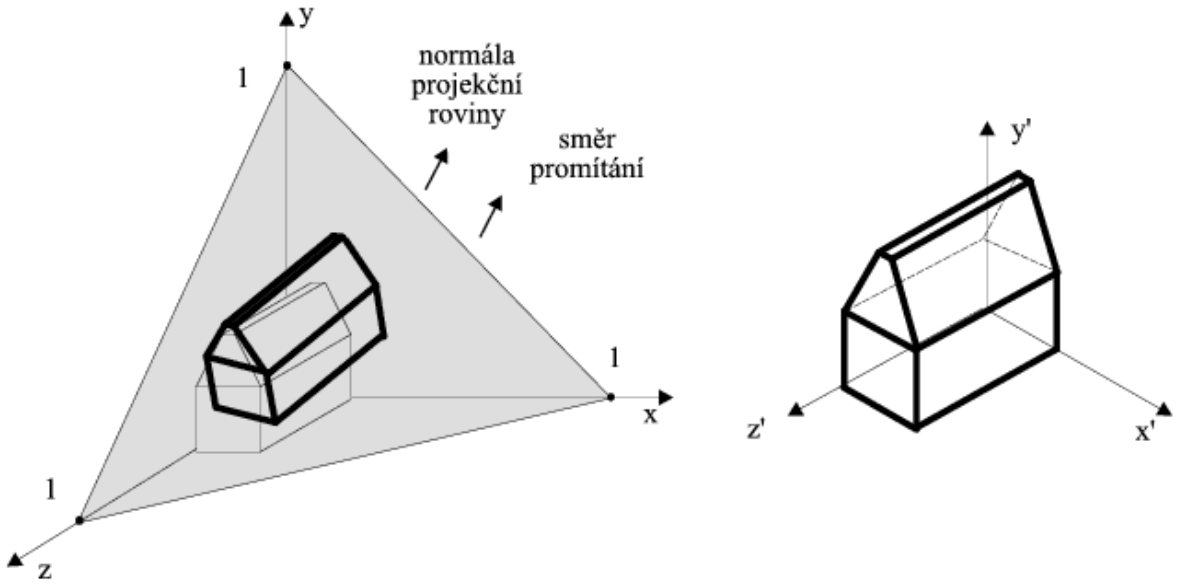
2.1.1.1 Axonometrie

Axonometrické promítání používá projekční roviny, které nejsou rovnoběžné s hlavními osami. Průmětna protíná 2 nebo 3 hlavní osy WCS. Příklad je na obr. 39. Tři úsečky v rovině, které mají společný jeden krajní bod a které neleží v přímce, lze pokládat za rovnoběžný průmět tří vzájemně kolmých a stejně velkých úseček, které mají jeden krajní bod společný. V souladu s tím je axonometrie často definována 5 hodnotami j_x, j_y, j_z, α, β, kde j_x, j_y, j_z jsou průmětny jednotek na osách *x*, *y*, *z* a α, β jsou úhly, které svírají promítnuté osy j_x, j_z s kolmicí na průmět osy j_y.

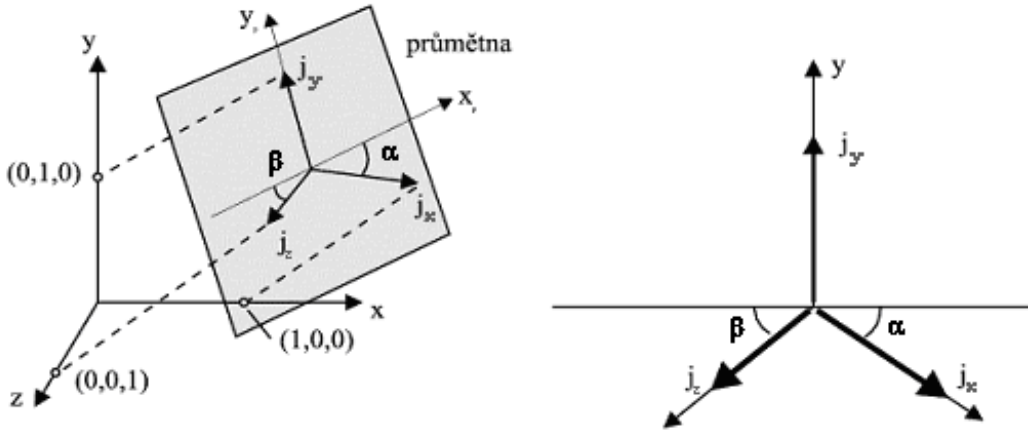
Pokud se souřadný systém průmětny (osy x_p, y_p) zvolí tak, aby průmět j_y ležel na ose y_p (viz obr. 40), pak axonometrii popíše matice

$$T_{axon} = \begin{bmatrix} j_x \cos\alpha & -j_x \sin\alpha & 0 & 0 \\ 0 & 1 & 0 & 0 \\ -j_z \cos\beta & -j_z \sin\beta & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \tag{43}$$

Pokud průmětna protne hlavní osy ve stejné vzdálenosti od počátku WCS (svírá stejný úhel se všemi osami), lze v průmětu měřit a porovnávat vzdálenosti. Zkreslení vzdáleností je totiž ve všech směrech promítnutých os stejné. Tato projekce se nazývá **izometrie** (obr. 39). Pokud jsou od počátku ve stejné vzdálenosti pouze 2 průsečíky, lze v průmětně měřit vzdálenosti ve dvou směrech promítnutých os, ve třetím směru jsou zkrácené nebo prodloužené vzdálenosti. Tento případ se nazývá **dimetrie**. Při obecném sklonu průmětny vůči osám se hovoří o **trimetrii**. Zde platí, že v každém směru promítnuté osy je třeba při porovnávání uplatnit odlišné měřítko.



Obr. 39. Axonometrie – izometrická projekce



Obr. 40. Zadání pravoúhlé axonometrie

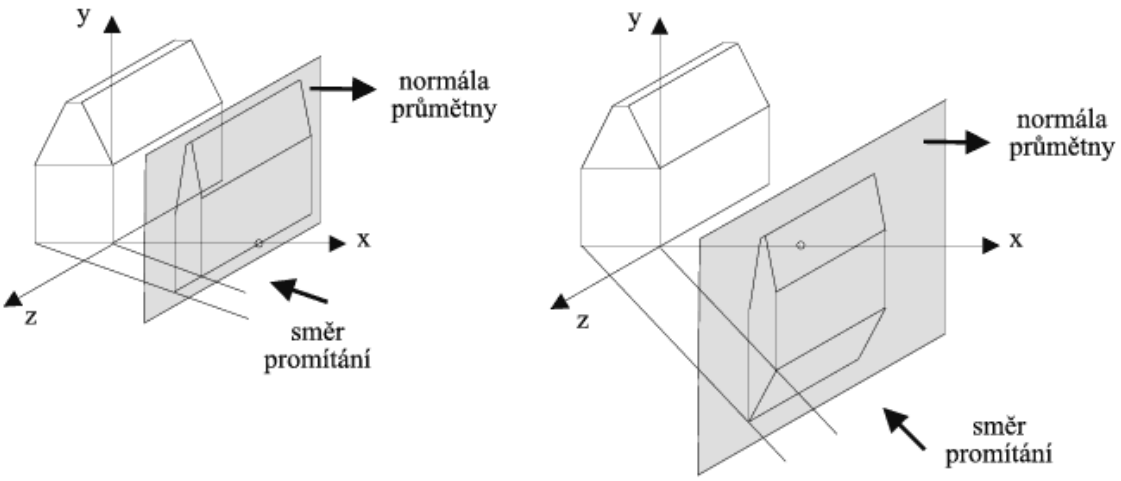
Axonometrické promítání zachovává rovnoběžnost promítnutých hran a mění úhly mezi hranami. Pokud je směr promítání kolmý na průmětnu, jedná se o **pravoúhlou axonometrii**. Obecné

rovnoběžné promítání se označuje jako **kosoúhlé promítání**. Speciální případy axonometrií jsou definovány následovně:

- izometrie: $j_x = j_y = j_z$, $\alpha = \beta$,
- dimetrie: $j_x = j_y$, $\alpha = \beta$,
- trimetrie: $j_x \neq j_y \neq j_z$,
- technické kosoúhlé promítání $j_y = j_z$, $\beta=0$.

2.1.1.2 Technické kosoúhlé promítání

Kosoúhlé promítání kombinuje vlastnosti Mongeova promítání s axonometrickým promítáním. Průmětna je rovnoběžná s některou s hlavních rovin a směr rovnoběžného promítání není kolmý na průmětnu. Příklady těchto projekcí jsou na obr. 41.

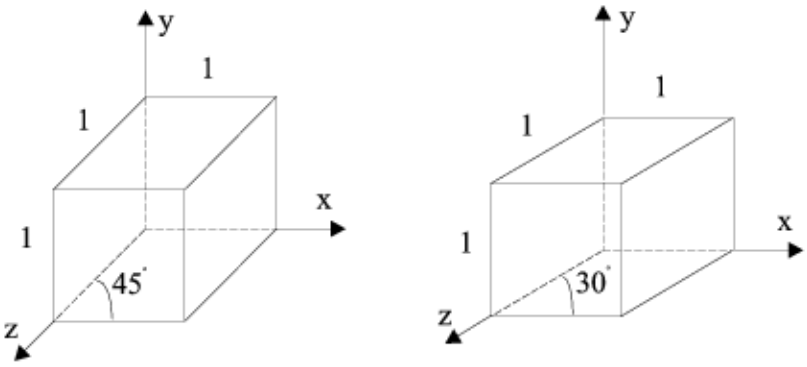


Obr. 41. Kosoúhlé promítání

Dva nejčastěji používané druhy kosoúhlé projekce se nazývají kabinet a kavalír. U kavalírské projekce svírá směr projekce s průmětnou úhel 45°. Výsledkem je, že úsečky rovnoběžné s průmětnou i kolmé na průmětnu se promítají se stejnou délkou. Na obr. 41 vlevo jsou dva příklady kavalírské projekce jednotkové krychle, nakreslené tak, že třetí osa svírá s horizontem úhel 45° nebo 30°.

U kabinetní projekce svírá směr promítání s průmětnou úhel arctan(2) = 63,4° a úsečky kolmé na průmětnu se zkracují na polovinu. Tato projekce je realističtější než kavalírská, neboť poskytuje představu o zkracování vzdáleností při pohledu na vzdálenější předměty. Příklady kabinetní projekce jsou na obr. 41 vpravo. Odpovídající transformace pro oba způsoby promítání jsou

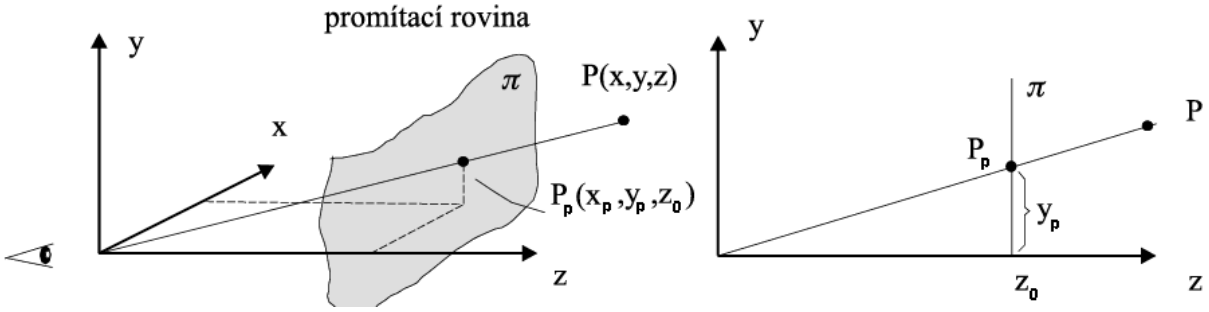
$$T_{kavalir} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ -\cos\beta & -\sin\beta & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad T_{kabinet} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ -\frac{\cos\beta}{2} & -\frac{\sin\beta}{2} & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \qquad (44)$$



Obr. 42. Kavalírské a kabinetní promítání

2.1.2 Perspektivní promítání

Perspektivní promítání respektuje optický model, který vyjadřuje lidské vidění reálného světa. Modeluje proporcionální zmenšování předmětů při vzrůstající vzdálenosti od pozorovatele. Neřeší problémy stereoskopie (napodobení optické soustavy páru očí pomocí dvou perspektivních obrazů vytvořených pro dvě různé pozice pozorovatele.) , ale poskytuje dobrý prostorový vjem na rovinné průmětně. Na obr.43 je ukázána projekce bodu $P(x, y, z)$ na bod P_p $(x_p ; y_p ; z_0)$ v průmětně π.



Obr. 43. Perspektivní projekce bodu P

Pozorovatel je v daném případě v počátku souřadného systému a projekční rovina je kolmá na osu z. Na základě trojúhelníkové podobnosti lze snadno odvodit, že

$$x_p = x\frac{z_0}{z}, \quad y_p = y\frac{z_0}{z}. \tag{45}$$

Vzdálené objekty za průmětnou jsou při zobrazení zmenšeny, objekty v průmětně zachovají velikost a objekty před průmětnou se zvětšují. Bod $P(x, y, z)$ se transformuje na bod P_p $(x_p ; y_p ; z_p)$ pomocí vztahu

$$\begin{bmatrix} x_p & y_p & z_p & w_p \end{bmatrix} = \begin{bmatrix} x & y & z & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & \frac{1}{z_0} \\ 0 & 0 & 0 & 0 \end{bmatrix}. \tag{46}$$

To znamená, že se perspektivní promítání vyjádří v homogenních souřadnicích pomocí lineární transformace. Při umístění pozorovatele v bodě $z = z_0$ a přemístění projekční roviny do počátku souřadného systému (ztotožnění s rovinou xy) platí vyjádření

$$x_p = x\frac{z_0}{z_0 + z} = \frac{x}{1 + \frac{z}{z_0}}; \quad y_p = y\frac{z_0}{z_0 + z} = \frac{y}{1 + \frac{z}{z_0}}, \tag{47}$$

a v maticovém tvaru

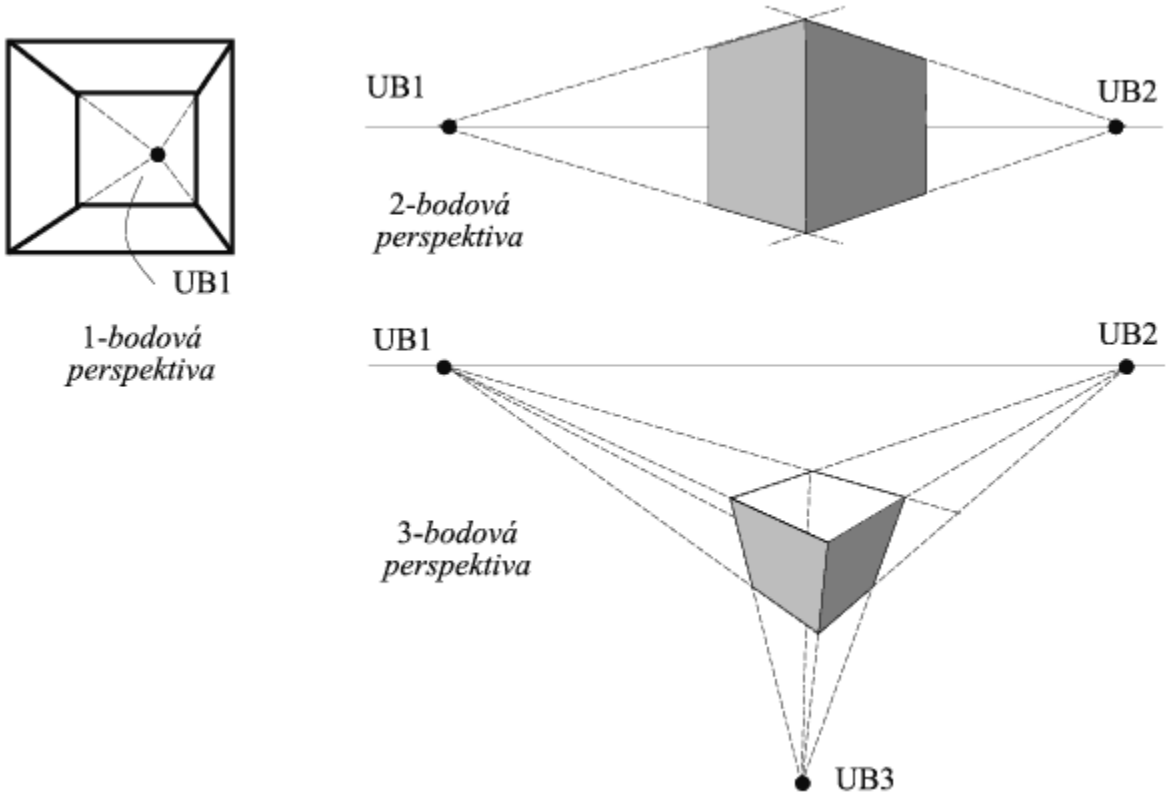
$$\begin{bmatrix} x_p & y_p & z_p & w' \end{bmatrix} = \begin{bmatrix} x & y & z & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & \frac{1}{z_0} \\ 0 & 0 & 0 & 0 \end{bmatrix}. \tag{48}$$

Poslední matici ve vztahu (48) lze vyjádřit jako součin matic

$$P_{per} = T_{per} P_{par}$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & \frac{1}{z_0} \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & \frac{1}{z_0} \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \tag{49}$$

Perspektivní projekci podle matice P_{per} lze chápat jako perspektivní transformaci objektů maticí T_{per} a následnou paralelní projekci do zvolené průmětny vyjádřenou maticí P_{par} . Transformace T_{per} zachová body v rovině z = 0 beze změny. To platí i pro obrazové body (1, 0, 0, 0) a (0, 1, 0, 0) na osách x nebo y. Rovnoběžky ve směru os x nebo y zůstanou rovnoběžné. Bod v nekonečnu na ose z (0, 0, 1, 0) je transformován do bodu (0, 0, 1, $\frac{1}{z_0}$) neboli (0, 0, z_0, 1). To znamená, že všechny rovnoběžky s osou z procházejí po perspektivní transformaci bodem (0, 0, z_0, 1). Tento bod se nazývá úběžník. Pokud projekční rovina protíná 2 nebo 3 osy souřadného systému, pak odpovídající perspektivní transformace určí 2 nebo 3 úběžníky na příslušných osách. V souladu s tím se používá pojmenování **dvoubodová** nebo **tříbodová perspektiva**. Příklady jednotlivých druhů perspektivní projekce jsou na obr. 44.



Obr. 44. Perspektivní projekce

Pokud protíná průmětna osy souřadného systému v úběžnících ($-x_0$, 0, 0), (0, $-y_0$, 0), (0, 0, $-z_0$), pak lze perspektivní transformaci popsat maticí

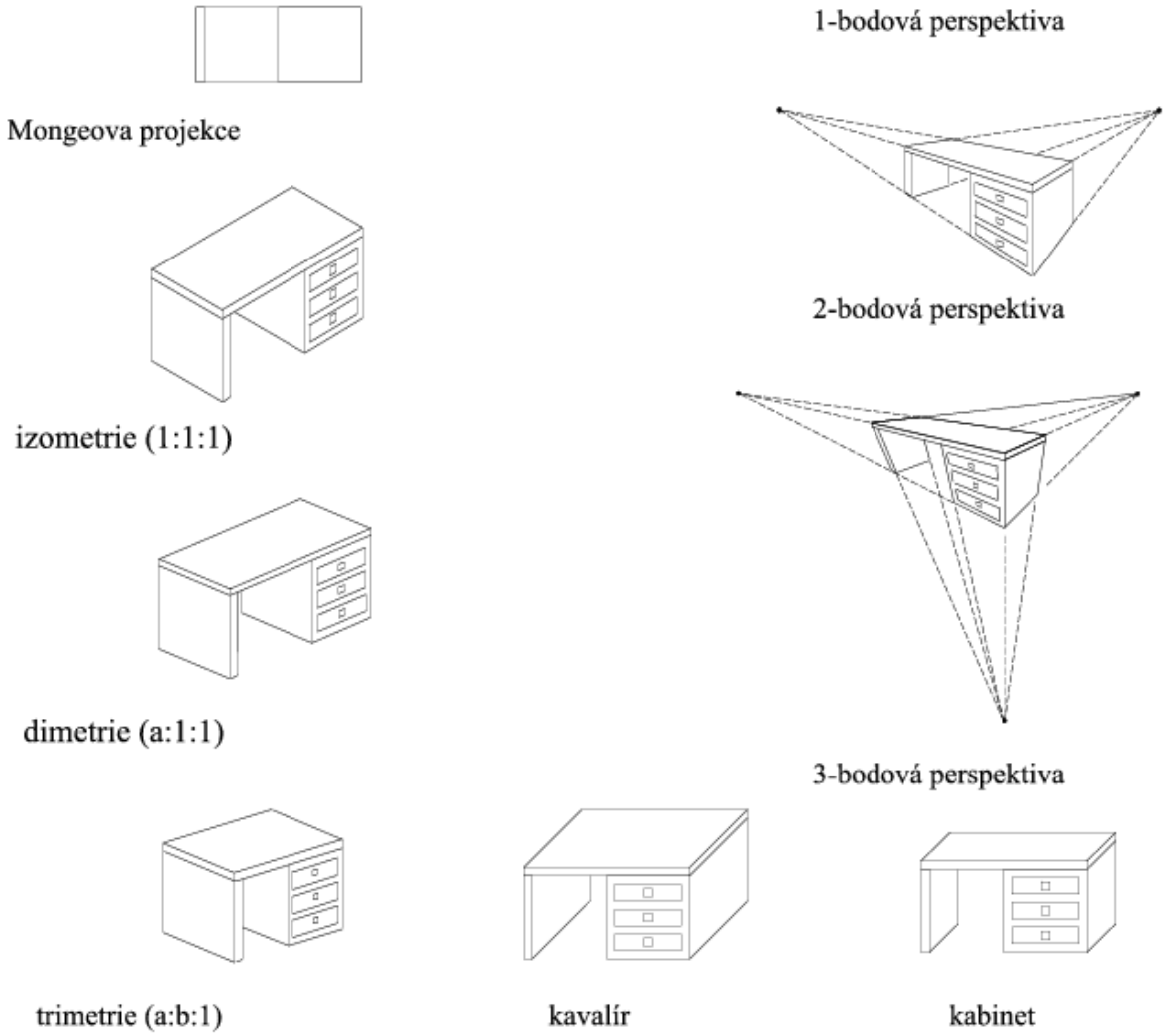
$$T_{per} = \begin{bmatrix} 1 & 0 & 0 & \frac{1}{x_0} \\ 0 & 1 & 0 & \frac{1}{y_0} \\ 0 & 0 & 1 & \frac{1}{z_0} \\ 0 & 0 & 0 & 0 \end{bmatrix}. \tag{50}$$

Celkové porovnání různých způsobů rovnoběžného a perspektivního promítání je společně uvedeno na obr. 45.

2.2 Modelování 3D Objektů

Trojrozměrné grafické objekty jsou většinou vyjádřeny pomocí matematického modelu. Pro reprezentaci hranic se nejčastěji používají dva způsoby. První z nich představuje specifikaci pomocí **objemového modelu**. V tomto případě se objekt popisuje parametricky. Tento způsob je vhodný pro následné zpracování a úpravu objektu. Využití je zejména ve strojírenské výrobě (CAD programy),

protože umožňuje fyzikálně simulovat vlastnosti výrobku. Nevýhodou tohoto popisu je, že modelování je omezeno pouze na základě použitých základních geometrických těles (primitivy), jako je koule, kvádr, válec, kužel a torus.



Obr. 45. Příklady promítání

Druhou metodou je **povrchová reprezentace**, ve které už není reprezentace odkázána na geometrické primitivy, protože je objekt konstruovaný jen pomocí polygonů svého povrchu. Povrch objektu se skládá z bodů (vertexů), které jsou spojeny úsečkami do mnohoúhelníků (polygonů). Tímto způsobem lze pomocí polygonální sítě reprezentovat prakticky libovolný předmět. Nejčastěji používaná je reprezentace s trojúhelníkovými polygony, jelikož je s nimi počítání nejjednodušší.

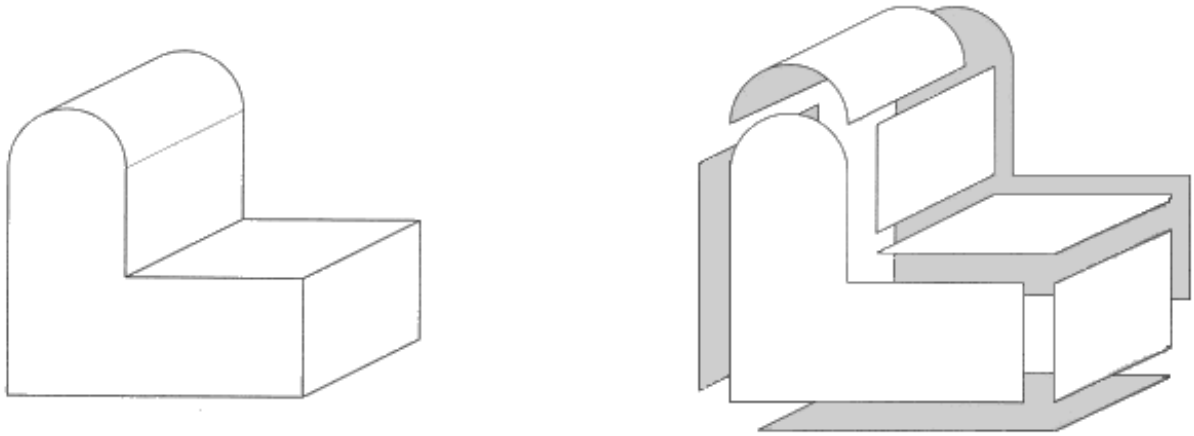
Přímé zobrazení analytických ploch bývá složité, pokud jsou kladeny vysoké nároky na kvalitu zobrazení. Nejvýhodnější se potom jeví zobrazení metodou sledování paprsku (viz kap. 2.6.1). Protože je tato metoda časově náročná, používá se k zobrazení zejména povrchová reprezentace, která se zpracovává běžnými technikami. Převodem objemového modelu na povrchový se navíc zmenší objem zpracovávaných dat. Protože algoritmy převodu z objemové na povrchovou reprezentaci bývají

velice rychlé, v paměti počítači se uchovává zpravidla jen objemová, protože je možné povrchovou reprezentaci vždy vygenerovat podle potřeby aplikace.

V následující kapitole jsou popsány nejčastěji používané reprezentace objektů. Jde zejména o hraniční reprezentaci, protože se jeví jako nejvíce výhodná z hlediska dalšího zpracování.

2.2.1 Hraniční reprezentace

Podstata hraniční reprezentace spočívá v popisu množiny hraničních bodů (*boundary representation, B-rep*). Informace o vnitřních bodech se neuchovávají a v případě potřeby se odvozují z popisu hranice. Popis tělesa je u hraniční reprezentace převeden na popis ploch (*face*), hran (*edge*) a vrcholů (*vertex*), které tvoří plášť tělesa – obr. 46.

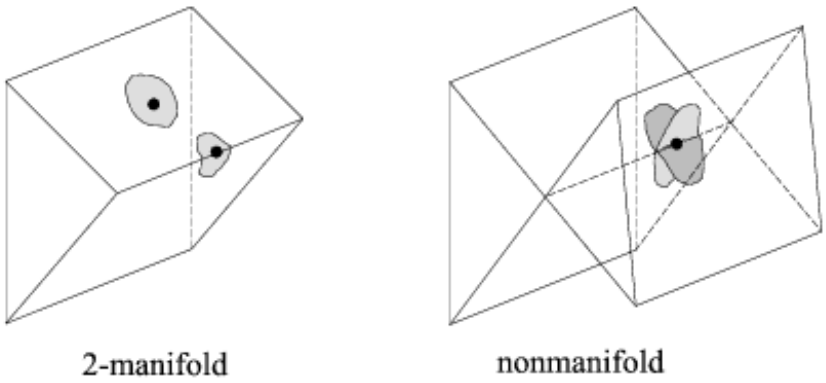


Obr. 46. Popis tělesa se převádí na popis pláště

Manifoldy

Popis vnitřními nebo vnějšími hraničními body se vztahuje i na objekty, které nelze ve skutečném světě vyrobit. Tělesa, která odpovídají reálnému modelu, se nazývají **manifoldy**. Definice říká, že těleso X se nazývá manifold, jestliže pro každý bod tělesa lze sestrojit otevřené kulové okolí, jehož průnik s X je homeomorfní s otevřenou koulí. Například těleso, které je definováno jako sjednocení 2 krychlí, které se dotýkají v jediném vrcholu, není manifold, patří mezi tzv. **nonmanifoldy**.

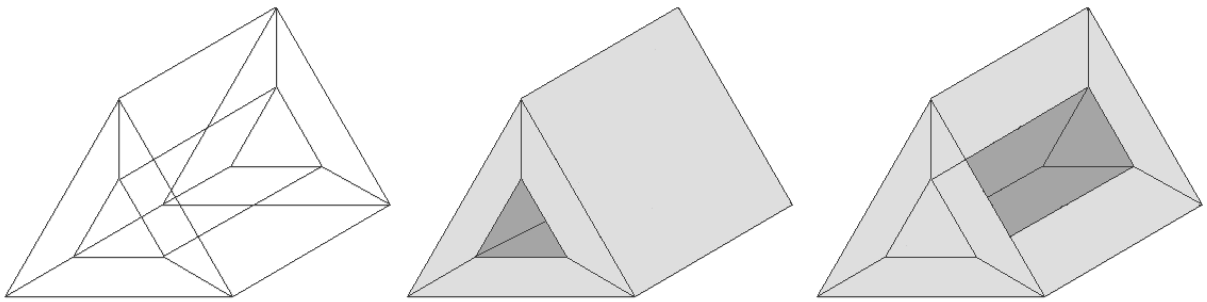
Modelovací systémy založené na hraničních modelech podporují často pouze tělesa, jejichž hranice tvoří 2-manifoldy. 2-manifold je tvořen vzájemně propojenými hraničními stěnami, kde každý bod stěny má v množině sjednocených stěn otevřené okolí, které je homeomorfní s otevřeným diskem. Tělesa, u kterých se v jedné hraně stýkají více než dvě hraniční stěny, tuto vlastnost nemají. Z definice manifoldu vyplývá, že se jednotlivé hraniční stěny a jejich okrajové křivky vzájemně neprotínají. Příklad 2-manifoldu a nonmanifoldu je na obr. 47. Hrana nonmanifoldu, na které je vyznačen bod, je z geometrického hlediska nekonečně tenká úsečka. Ve skutečnosti jsou zde ale hrany dvě a těleso je buď propojeno nebo rozpojeno.



Obr. 47. Tělesa 2-manifold a nonmanifold

2.2.1.1 Hranová reprezentace

Nejjednodušší metodou (a také nejstarší) pro popis povrchu tělesa je hranová reprezentace. Tělesa takto vytvořená se také označují jako drátové modely (*wire frames*). Při této implementaci se vytváří pouze seznamy vrcholů (zde jsou uloženy souřadnice) a hran (každá položka obsahuje dva ukazatele do seznamu vrcholů, které se mají spojit). Výsledkem je úsporná struktura obsahující minimum topologických informací, což jsou často nedostatečné informace pro jednoznačnou interpretaci tělesa. Příklad je uveden na obr. 48, kde jeden model může reprezentovat více různých těles.



Obr. 48. Nejednoznačný drátový model

2.2.1.2 Plošková reprezentace

Plošková reprezentace obsahuje stejné informace jako reprezentace hranová a navíc s sebou nese informace určené pro popis ploch. Tzv. **jednoduchá plošková reprezentace** má všechny tyto informace uloženy v datových strukturách. V praxi vznikají tři případy, přičemž jejich implementace závisí na konkrétní situaci (používají se dynamické seznamy, případně statická pole):

1. Plochy jsou uspořádány do sítě (*mesh*). Reprezentace se provádí dvojrozměrným polem ukazatelů do seznamu vrcholů.
2. Veškeré plochy mají stejný počet vrcholů (obvykle 3 nebo 4). Datová struktura pro popis plochy je tvořena seznamem, jehož každý člen je tvořen trojicí nebo čtveřicí ukazatelů na vrcholy.
3. Plochy mají různé uspořádání a velikost. V seznamu pak mají různé plochy různě dlouhé položky a každá z nich má různý počet ukazatelů na vrcholy.

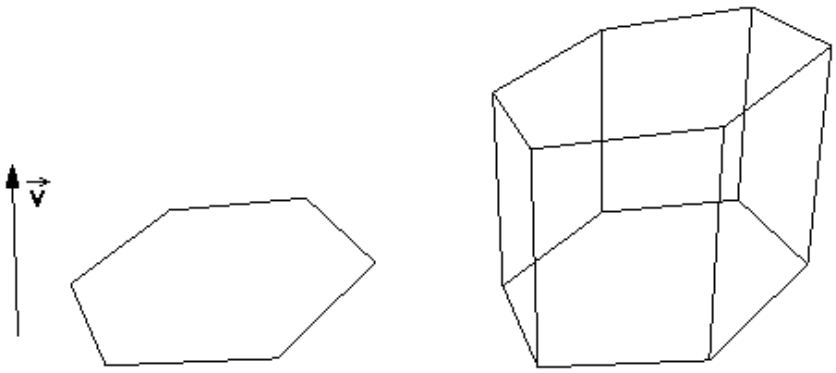
Proti hranové reprezentaci je jednoduchá plošková reprezentace jednoznačná a užívá se pro přímé kreslení hran i ploch. Přesto však nemusí obsahovat dostatek informací o objektu. Jde například o údaje typu hran, kdy se ztěží zjišťuje, zda je těleso manifold.

Z tohoto důvodu se velice rozšířila **strukturovaná plošková reprezentace**, která se skládá také ze tří seznamů (vrcholy, hrany, plochy). Zde s sebou nesou nejvíce informací seznamy hran. Každá hrana tak nese informace o všech elementech (plochy, hrany, vrcholy), do kterých zasahuje. Tento datový záznam se také nazývá okřídlená hrana, protože grafické znázornění s hranami, na které se napojuje, připomíná křidélka.

2.2.2 Šablonování

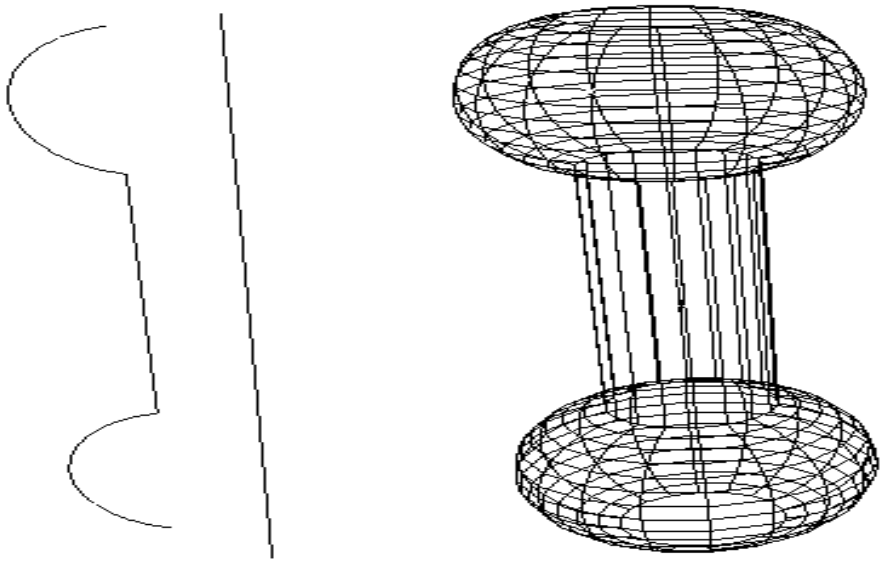
Šablonování je modelovací technika, při které se získává plocha tažením dvojrozměrného obrysu po trojrozměrné křivce - páteři (*spine curve*). Podle způsobu tvorby plochy se rozlišují 3 případy:

1. Translační šablonování – v tomto případě je obrys libovolný a páteří je úsečka. Tímto způsobem se vytvářejí přímkové plochy.
2. Rotační šablonování – také v tomto případě je obrys libovolný a trajektorií, po které je obrys tažen, je kružnice. Výsledkem je rotační plocha.
3. Obecné šablonování – u tohoto způsobu vytváření plochy jsou obrys i trajektorie libovolné. Vzniká tak zobecněná válcová plocha.



Obr. 49. Plocha (vpravo) vzniklá tažením z profilové křivky (vlevo) ve směru vektoru $\vec{v}$

Přímkové plochy se v jednom směru skládají z úseček. Výsledkem může být válcová či hranolová plocha. Přímková plocha může vznikat vytažením (obr. 49) nebo spojením dvou či více okrajů. Translační šablonování se pak v prvním případě provádí po profilové křivce, která nemění svůj tvar, ve druhém se mění s profilovou křivkou. Kombinace obou přístupů se nazývá potahování (*skinning*).



Obr. 50. Plocha (vpravo) vzniklá rotováním 2D profilu (vlevo) kolem úsečky

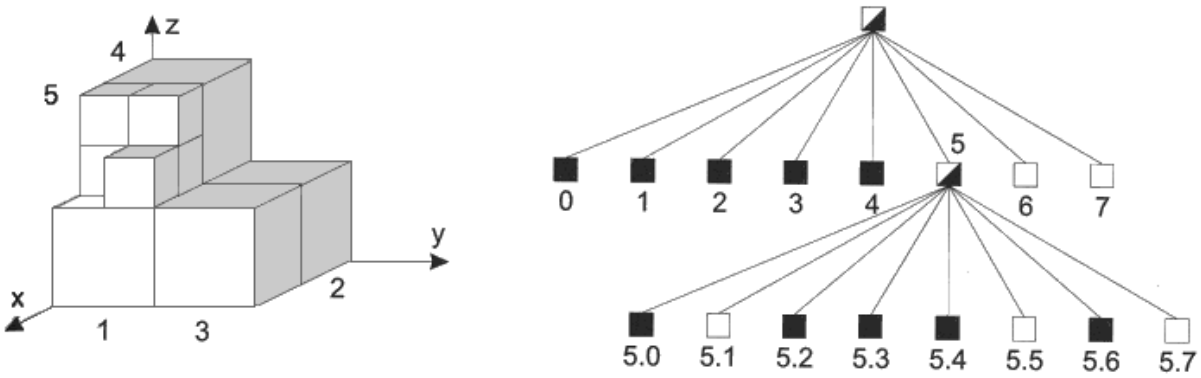
Jak již bylo uvedeno výše, u rotačního šablonování se rotační plocha získává tažením obrysu po kružnici, resp. rotací rovinné křivky okolo osy (obr. 50). Průnik rotační plochy s libovolnou rovinou kolmou na osu rotace je tedy kružnice. Pokud bude profilovou křivkou kružnice, která je posunutá o určitou vzdálenost od osy rotace, vznikne její rotací anuloid. Rotací úsečky rovnoběžné s osou rotace vzniká válcová plocha, rotací úsečky různoběžné s rotační osou otevřený kužel apod.

2.2.3 Obsazenost prostoru a oktálové stromy

Podobně jako se v rovině provádí reprezentace rastrového obrazu (viz kap. 1.3.3), lze v prostoru vyjádřit jeho obsazenost buď **trojrozměrným polem elementárních objemových jednotek** nebo pomocí **oktálových stromů**.

Reprezentace pomocí trojrozměrného pole se používá v případě, kdy je k dispozici dostatečné množství paměti. Scéna se rozdělí na potřebné množství objemových jednotek (nazývané **voxely**), které nabývají dvou hodnot – jsou obsazené nebo volné. Voxely jsou vlastně analogií pixelů ve dvojrozměrném prostoru a nejčastěji mívají tvar kvádru či krychle. Jejich nevýhodou je již zmíněná paměťová náročnost, která narůstá s detailnějším popisem objektu a tím pádem se zmenšujícími se velikostmi voxelů.

Oktálové stromy představují úspornější (rekurzivní) způsob popisu objektů v prostoru. Jsou analogií kvadrantového stromu používaného pro popis dvojrozměrného obrazu. Buňky uvnitř oktálového stromu mají velikost hrany, která vznikne postupným dělením dvěma z délky hrany největší oblasti v prostoru omezující celý objekt. Na obr. 51 vlevo je zobrazeno jednoduché těleso, které lze uzavřít v pomyslnou krychli. Ta se může v každém kroku dělit na 8 stejně velkých objemů (oktantů) třemi řezy v polorovině, přičemž dělení se provádí tehdy, pokud se v daném oktantu nachází menší detail objektu.



Obr. 51. Popis tělesa pomocí oktálového stromu

K zápisu oktálového stromu se používají následující informace, které se mohou zaznamenávat do uzlů stromu:

- prázdný uzel (*void, V*) – uzel, který leží mimo těleso,
- plný uzel (*full, F*) – uzel, který je zcela zaplněn tělesem,
- smíšený uzel (*mixed, M*) – uzel, který se dále dělí.

Očíslování (0 - 7) oktantu se provádí tak, aby vyjadřovalo ve svých bitech prostorové souřadnice (0. bit je určen pro souřadnici *x*, 1. bit pro *y* a 2. bit pro *z*). V počítači se pro zápis oktálového stromu používají lineární seznamy. Strom, který je na obr. 51 vpravo lze v rekurzivním tvaru, kdy je smíšený uzel nahrazen dvojicí závorek, zapsat jako

(F F F F F (F V F F F V F V) V V).

Velikost oktantu klesá se vzdáleností od kořene stromu. Tímto způsobem lze pomocí oktálového stromu úsporně zaznamenat i velice malé detaily. Nedostatkem této metody je náhrada všech tvarů v kvádry či krychle, což u oblých tvarů způsobuje sníženou kvalitu reprodukce.

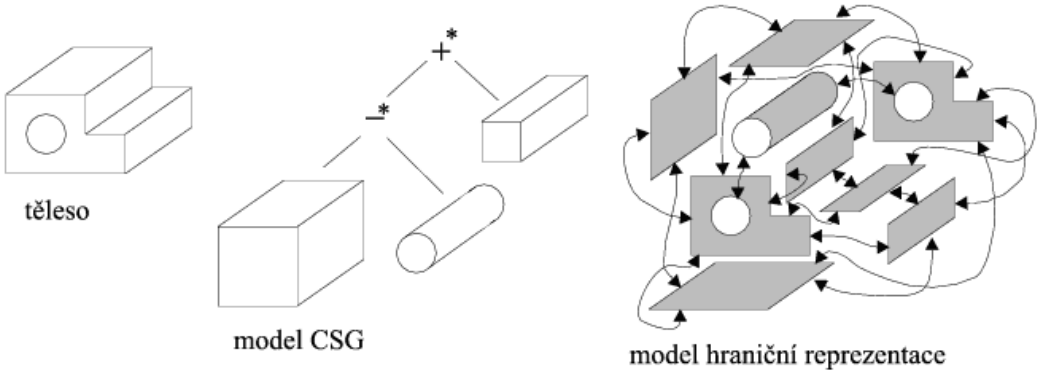
Přesto jsou oktálové stromy hojně používany nejen pro primární reprezentaci objektů, ale i jako pomocná datová struktura, která jiný popis objektů doplňuje. Staly se tak součástí mnoha jiných algoritmů, např. při zobrazovací metodě sledování paprsku (kap. 2.6.1) nebo konstruktivní geometrii těles. V těchto případech listy typu *F* nepředstavují objem vyplněný hmotou, ale oktant, ve kterém se nacházejí údaje o jiných (primárních) datových strukturách.

2.2.4 Konstruktivní geometrie těles

V aplikacích typu CAD je nejvýhodnější popisovat tělesa pomocí jednoduchých geometrických těles. Metoda se nazývá **konstruktivní geometrie těles** (CSG – *Constructive Solid geometry*) a tělesa se reprezentují stromovou strukturou (CSG stromem), která uchovává historii dílčích konstrukčních kroků. Výsledný objekt je potom složen pomocí množinových operací těchto jednoduchých objektů.

Množinové operace odpovídají konstrukčním postupům a mohou být prováděny s jednotlivými tělesy nebo celými CSG stromy. Patří mezi ně součet, rozdíl a průnik. Jako základní geometrická tělesa se používají kvádr, kužel, koule, válec, anuloid a poloprostor (používá se zejména pro efektivní provádění operací, jako je ořezávání). Výsledný vzhled pak ještě ovlivňují prostorové transformace (posun, otočení). Pravděpodobně nejznámějším a nejpoužívanějším opensource programem pro modelování pomocí CSG je Povray. [5]

Na obr. 52 vlevo je těleso, které lze velice jednoduše reprezentovat pomocí CSG (uprostřed). Od jednoho kvádru se odečte válec (vznikne otvor) a přičte kvádr s menšími rozměry. Výsledný objekt je tak složen ze tří těles a dvou matematických operací. Pokud by se takové těleso mělo reprezentovat pomocí hraniční (ploškové) reprezentace, jeho konstrukce by byla mnohem složitější (obr. 52 vpravo).



Obr. 52. Porovnání modelů CSG a hraniční reprezentace při popisu tělesa

2.3 Kódování objektů a zobrazování ve 3D

V současné době se pro kódování 3D objektů nejčastěji používají techniky VRML, OpenGL, DirectX. Zejména poslední dvě jmenované techniky se používají pro tvorbu 3D aplikací (především her), protože možnosti pro práci s 3D objekty rozšířily o další podpůrné funkce.

2.3.1 VRML

VRML (*Virtual Reality Modeling Language*) je jednoduchý multiplatformní jazyk pro kódování 3D objektů na internetových stránkách a vytváření trojrozměrných „světů“. Autorem tohoto

standardu je firma Silicon Graphics, která v roce 1995 předložila první verzi. Tato verze podporovala popis pro tvorbu statických prostředí. V současné sobě existuje ve druhé verzi (VRML 2.0), která přinesla rozšíření o popis dynamických světů a nástroje pro spolupráci více uživatelů ve virtuálním prostředí. Koncem roku 1997 byl jazyk VRML oficiálně přijat za standard ISO.

Jazykem VRML je v prostém textovém souboru popsána trojrozměrná scéna, složená z různých geometrických obrazců. Způsob popisu scény by se dal velmi zjednodušeně vyjádřit takto: "Na pozici o souřadnicích 0,0,1 vykresli červenou kouli o poloměru 0,3, posviť na ni modrým světlem, umístěným na souřadnicích 0,0,0." Jazyk VRML umožňuje podobně jako HTML definovat odkazy na další dokumenty, takže lze například procházet virtuální obrazárnou a kliknutím na plochu obrazu získat třeba informace o autorovi. Díky tomu, že po síti neputují přímo grafická data, ale pouze jejich textový popis, je přenos VRML souboru (přípona .WRL) poměrně rychlý. O to větší nároky jsou kladeny na hardware, který musí zobrazit třírozměrnou scénu a na základě podnětů od uživatele s ní dále pracovat. Pro experimenty s VRML jsou doporučovány počítače s výkonnějšími procesory a větší kapacitou paměti. Nutným doplňkem prohlížeče pro práci s VRML je příslušný plug-in nebo komponenta.

VRML se používá všude tam, kde je potřeba uživateli nabídnout možnost prohlédnout si nějaký objekt ze všech stran. Objektem mohou být modely vozidel, obrazárny. Pomocí tohoto jazyka může každý běžný uživatel osobního počítače připojeného k Internetu procházet městem nebo zahradou či zemí, která se mohla zrodit ve fantazii někoho jiného nebo se jedná o model skutečné krajiny. Touto cestou by v budoucnu mohlo být možné vybírat dovolenou u své cestovní kanceláře nebo si podle návodu v jazyce VRML smontovat skříň zakoupenou v nesloženém stavu přímo ve virtuálním obchodě.

2.3.2 DirectX

Microsoft DirectX je skupina nízkoúrovňového aplikačního programového rozhraní (**API** - *Application Programming Interface*) na vytváření her a ostatních náročnějších multimediálních aplikací. Obsahuje podporu pro dvojrozměrnou (2D) a trojrozměrnou (3D) grafiku, zvuk a hudbu, vstup a síťovou komunikaci pro aplikace jako jsou síťové hry. DirectX poskytuje standardizované vývojové prostředí pro programy určené pro OS Windows, vývojáři se tak nemusejí starat o hardware, pro který své programy píší - o komunikaci s ním se postará právě rozhraní DirectX.

Nejpodstatnější komponenty, které DirectX obsahuje jsou:

- ***DirectDraw*** - programové rozhraní, které zpřístupňuje přímý přístup do paměti zobrazovacího adaptéru s důrazem na kompatibilitu s Windows grafickým rozhraním (tzv. GDI - *Graphics Device Interface*). *DirectDraw* představuje hardwarově nezávislé řešení, přičemž používá dvě vrstvy, a to: **HAL** (*Hardware Abstraction Layer* - sada příkazů je na každé konfiguraci počítače stejná a pokud DirectX zjistí, že

zařízení podporuje daný příkaz hardwarově využije jeho funkce. Pokud zjistí opak, tento příkaz interpretuje softwarově pomocí dostupných prostředků) a **HEL** (*Hardware Emulation Layer* – k dispozici jsou všechny příkazy, ale všechny se vykonávají softwarově. Tento mód se používá hlavně při krokování programů). Tyto dvě metody se používají i ve všech ostatních rozhraních DirectX.

- ***Direct3D*** – vzniklo jako rozšířené rozhraní *DirectDraw* pro trojrozměrnou grafiku (3D). Programátor si může vybrat ze dvou typů, a to: *Immediate Mode* nebo *Retained Mode*. *Immediate Mode* využívají hlavně profesionální programátoři, protože poskytuje přímý přístup na hardware grafické karty. V tomto módu se pracuje s jednotlivými polygony a světly. *Retained Mode* - tento mód využijí hlavně začínající programátoři nebo ti kteří nepotřebují mít kontrolu nad každým zobrazeným pixelem na obrazovce. Tento mód poskytuje jednoduché používání objektů, celých animací světel kamer, kolizi objektů a mnoho jiných užitečných věcí. Od verze DirectX 8 došlo ke sjednocení *DirectDraw* a *Direct3D* pod komponentu *DirectX Graphics*.
- ***DirectInput*** - rozhraní pro vstupní zařízení jako jsou myš, klávesnice, joystick a ostatní herní zařízení včetně *force-feedback* zařízení (zařízení se zpětnou odezvou na dění ve hře).
- ***DirectPlay*** – rozhraní podporující síťové hry (protokoly IPX, TCP/IP).
- ***DirectMusic*** - rozhraní na přehrávání hudby.
- ***DirectSound*** - rozhraní pro práci se zvukem. Součástí jsou i 3D zvukové efekty.

Rozhraní DirectX bylo poprvé představeno již v roce 1995. Od té doby následuje v podstatě každý rok nová verze a nutno přiznat, že každá je lepší a lepší - a pro počítačové hráče stále více nezbytná. Většina her postupně začala DIRECTX striktně vyžadovat a bez něj (či lépe řečeno jeho minimální požadované verze) je nelze spustit. V současné době je nejnovější verze 9.0, která zejména přinesla nový jazyk HLSL (*High-Level Shader Language*), což je nový programový model, usnadňující programátorům práci s grafikou a grafickými efekty. Je založený na jazyku C a představuje nové, uživatelsky přívětivější prostředí pro vývojáře, které jim usnadňuje práci na mnoha frontách. Navíc je HLSL kompatibilní s grafickými procesory podporujícími DirectX, takže je samozřejmé, že naprogramované efekty budou běhat stejně na různém hardwaru. Mezi další novinky patří vylepšená animace v reálném čase, urychlené programování základních grafických procesů, nový vertex a pixel shader 2.0, plná integrace vývojového systému NET, urychlená hardwarová akcelerace pomocí DirectShow a výkonnější DirectPlay pro multiplayerové hry.

Dnes se už dá říci, že právě nové verze DirectX jsou "hnacím motorem vývoje nových grafických čipů". Hardwarovou podporu slibují takřka všichni dnešní hlavní výrobci grafických karet, zejména tedy ATi a nVidia.

2.3.3 OpenGL

OpenGL (*Open Graphics Library*) je nízkoúrovňová knihovna pro práci s trojrozměrnou grafikou. Od doby svého uvedení na počátku devadesátých let se stala široce uznávaným a podporovaným standardem na poli grafických aplikací, CAD/CAM/CAE inženýrských systémů, virtuální reality a tvorby her. OpenGL představuje jednotné API (*Application Programming Interface*) mezi programem a grafickým hardware. Hlavními rysem OpenGL je nezávislost na cílové platformě a použitém programovacím jazyku. V současnosti lze OpenGL nalézt na téměř všech platformách, z nichž ty nejvýznamnější jsou IRIX, Unixy, Windows 95/98/NT, DOS, Linux, Solaris, MacOS, OS/2, NeXTSTeP, OPENStep a BeOS. Trochu nečekaně lze OpenGL nalézt i na některých palmtopech. OpenGL využívá značné množství komerčních programů. Z vizualizačních programů například Kinetix 3D Studio MAX, Alias | Wavefront Maya, Caligari TrueSpace nebo LightWave 3D. Z oblasti CAD potom MicroStation, CATIA nebo SolidWorks. Konečně z her využívá OpenGL mimo jiné Quake 2, Unreal, Sin nebo Half Life.

Pro běžného uživatele je OpenGL standardem, díky kterému mohou aplikace plně využívat možností akcelerované grafiky. I v případě, že uživatel nemá v počítači zabudovánu výkonnou grafickou kartu, může provozovat grafické aplikace OpenGL podporující. Potom se ovšem všechny výpočty realizují softwarově, což je sice o poznání pomalejší, ale v praxi pořád ještě použitelné.

Z pohledu programátora představuje OpenGL soubor asi 150 funkcí, které umožňují specifikovat geometrické objekty ve dvou nebo třech dimenzích. Takto popsané objekty potom umožňuje dále zpracovávat. Autor programu přitom nemusí detailně znát konfiguraci počítače, na kterém jeho program poběží. Při značně zjednodušeném pohledu lze říci, že programátor pouze udává souřadnice vrcholů trojúhelníků společně s informací o mapování textury, vlastní vykreslování už za něj vyřídí OpenGL. Má přitom možnost zapínat a vypínat různé její dílčí funkce, a tak přímo ovlivňovat výslednou kvalitu vytvářeného obrazu. Komplexnější vykreslení scény znamená často velmi malý nárůst kódu.

Scéna se komponuje z elementárních objektů, což jsou právě a jenom body, čáry, polygony, výřezy obrázku (*pixel rectangles*) a bitmapy. Elementární objekty - primitiva jsou dány skupinou jednoho nebo více vrcholů. Vrcholy (vertexy) přitom znamenají body v trojrozměrném prostoru. Význam vrcholů v definici elementárního objektu může být různý. Podle typu potom může znamenat počáteční a koncový bod úsečky, vrchol mnohoúhelníku, apod. Veškeré další informace o scéně, jako například barva, normálový vektor nebo souřadnice textury jsou spjaty právě s vrcholy primitiv. Jednotlivé vrcholy, z nichž se elementární objekt skládá, se dají zpracovávat nezávisle v daném pořadí. Právě na proudovém zpracování obrazových elementů staví OpenGL svůj grafický výkon.

Koncept zobrazování OpenGL s sebou nese i jistá omezení. Například je obtížné základními prostředky pracovat s objektem, který je oblý. Parametricky popsané plochy (Bézierovy plochy, NURBS) se nejprve musí převést na síť mnohoúhelníků, a teprve potom se mohou zobrazovat. To, že

se jednotlivé komponenty obrazu zpracovávají odděleně, má vliv na výslednou kvalitu obrazu. Například výpočet osvětlení se vztahuje vždy lokálně k danému tělesu. Bez detailní simulace šíření světla ve scéně nejsou výsledky zcela přesvědčivé. Kvalita obrazu výsledné scény se tak pouze blíží *ray traycingu* (viz kap. 2.6.1), tedy metodě, která nejlépe aproximuje fyzikální model. Tyto principiální nedostatky OpenGL jsou vyváženy rychlostí a jednoduchostí výpočtu. OpenGL sama o sobě nenabízí prostředky k objektovému popisu scény na hierarchickém principu. K vyřešení některých těchto nedostatků se používají pomocné knihovny:

- GLU OpenGL Utility Library,
- GLUT OpenGL Utility Toolkit,
- GLAUX OpenGL Auxiary Library.

2.3.4 Porovnání DirectX a OpenGL

Pokud by se měly porovnat techniky DirectX a OpenGL, je situace poměrně nejasná a teprve budoucnost ukáže, který standard přetrvá nebo zda budou oba existovat vedle sebe zároveň.

Uvedením DirectX vyvolal Microsoft rozruch, a to jak pozitivní, tak i negativní. Vývojáři her pod Windows dostali ucelený balík komponent, *DirectDraw*, *DirectSound*, *DirectPlay*, *Direct3D*, *DirectShow*, …, který dodával nové možnosti. Ze jmenovaných součástí je nejdiskutovanější právě Direct3D. Většina specialistů z oboru se shoduje v tom, že nevznikl důvod k zavedení nového standardu. Microsoft se pustil do návrhu vlastního 3D API, přitom OpenGL plně dostačovala všem aplikacím a byla prověřená časem. Programování v Direct3D se OpenGL příliš nepodobá. Direct3D přitom disponuje v zásadě stejnými funkcemi jako OpenGL. Na rozdíl od Direct3D leží OpenGL přímo na hardwaru a volá přímo jeho služby. Ten, kdo OpenGL implementuje, musí vědět, pro jaký typ grafické karty ho píše a co umí. Rozhodnutí, která služba bude vykonávána na procesoru, je tedy na něm. Díky tomu, že se každá implementace z hlediska aplikace chová stejně, netestují se žádné ovladače. OpenGL předstírá, že umí úplně vše, záleží na implementaci, jakým způsobem a jak rychle bude služby poskytovat. OpenGL je přitom otevřený standard a implementace přitom musí projít náročnými testy, aby mohla nést označení OpenGL. Direct3D naproti tomu vyvíjí jen Microsoft a pouze on rozhoduje o tom, co se za touto značkou skrývá. Protože je OpenGL ve své koncepci striktně oddělena od jakéhokoliv okenního systému, běží téměř všude. Oproti tomu DirectX jsou pevně svázány s Windows.

OpenGL v *immediate módu* umí všechno, co Direct3D a navíc poskytuje další funkce, jako třeba odstraňování zadních stěn před vykreslením (*face-culling*), správu šablon a paměti textur, trojrozměrné textury, akumulační buffer (používá se třeba k navození efektu rychlého pohybu s rozmazáním – *motion blur*), podporu parametrických křivek a ploch, atd. OpenGL i Direct3D používají Goraudovo stínování (viz kap. 2.5.1.1). Direct3D má teoretické předpoklady i pro Phongovo

stínování, které by posunulo grafický výstup blíže k realitě, tato funkce však nebyla doposud implementována.

Protějškem display-list módu OpenGL je u Direct3D tzv. *retained mode*. Tady poskytuje Direct3D podstatně lepší služby než OpenGL. Objekty na scéně je možné seskupovat do hierarchických struktur, se kterými se dá pracovat na objektovém principu. Podobné funkce jsou u OpenGL dostupné pouze prostřednictvím určitých rozšíření. Za vymoženosti nového přístupu potom programátor zaplatí sníženým výkonem programu.

Přímé srovnání grafického výkonu OpenGL a Direct3D je docela komplikované, protože obě API mají trochu jiné konečné určení. OpenGL a Direct3D je navíc třeba srovnávat pouze pod Windows, protože do jiného prostředí nelze Direct3D implementovat. OpenGL poskytuje kvalitní zobrazení trojrozměrné scény, které je použitelné i pro seriózní aplikace (jiné aplikace než hry). Prioritou Direct3D je rychlost, nikoliv výsledek.zatím chybí rozsáhlejší výkonové testy, které by ukázaly, jak si vedou programy na různých konfiguracích počítače.

OpenGL se stala úspěšným průmyslovým standardem a její pozice zůstává do značné míry neotřesena. Významné je, že za osm let se standard změnil pouze minimálně, hlavní vylepšení jsou v implementacích. U DirectX jsme svědky rychlého vývoje verzí, které často nejsou ani zpětně kompatibilní. Zatímco současná verze OpenGL je 1.2, verzí DirectX je výrazně více.

2.4 Viditelnost objektů

V počítačové grafice se s prostorovými objekty často pracuje jako se samostatnými objekty. Každá z těchto entit se podle zvoleného způsobu promítání a individuálních parametrů transformuje do prostoru obrazu. Zejména z důvodu rychlosti zobrazování je třeba určit, který objekt (resp. jeho část) je či není zakrytý. Takové entity se nemusí zobrazovat, neboť jsou buď mimo oblast pro zobrazení nebo jsou překryty entitami dalšími. Jinými slovy jde o řešení viditelnosti.

Cílem algoritmů, které zjišťují viditelnost, je nalezení všech objektů či jejich částí, které jsou viditelné z místa pozorovatele. Tyto části se vyhodnocují a následně vykreslují. V některých technických aplikacích jsou algoritmy rozšířeny i o případné vykreslování skrytých částí (např. čárkovanou čarou apod.).

Algoritmy pro řešení viditelnosti jsou vždy závislé na konkrétní reprezentaci objektů v prostoru. Nejvýhodnější se zpracování viditelnosti provádí, pokud jsou objekty popsané hraniční (ploškovou) reprezentací. Většina algoritmů pracuje právě s rovinnými ploškami. U ostatních reprezentací se používají speciální metody.

Algoritmy viditelnosti se dělí do dvou základních skupin podle toho, v jakém tvaru poskytují výsledný soubor:

1. **Liniové algoritmy** (HLE – *Hidden Line Eliminiation*). Na výstupu se získá soubor úseček (vektorů), které představují viditelné hrany. Výhodou tohoto řešení je

skutečnost, že vektory se dají při vykreslování bez omezení opakovaně zvětšovat i zmenšovat, aniž by docházelo ke ztrátě kvality obrazu. Pokud by byla potřeba mít na výstupu i soubor zakrytých hran, lze s nimi pracovat obdobně.

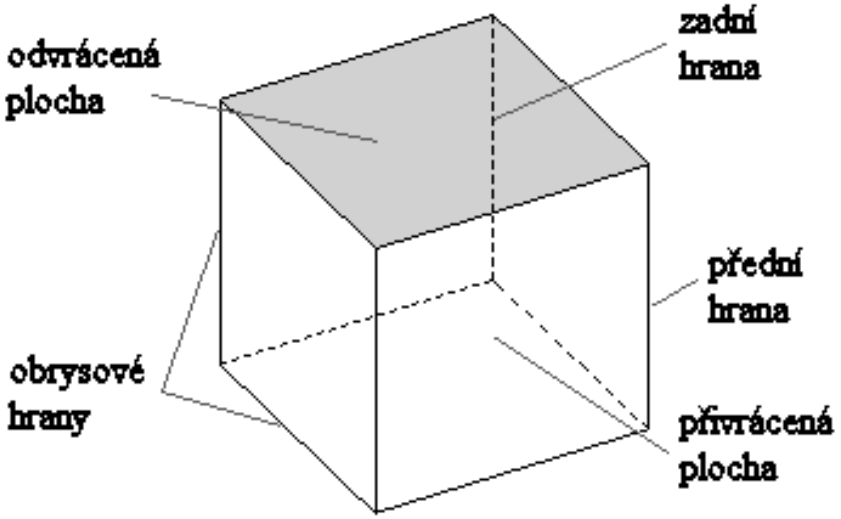
2. **Rastrové algoritmy** (HSE – *Hidden Surface Eliminiation*). Na výstupu se získá soubor pixelů, které představují konkrétní body dané scény v pevně daném rozlišení s barvou odpovídající viditelné ploše. Nevýhodou je pevný rozměr výsledného obrazu. Naopak výhodou je, že z těchto pixelů lze okamžitě vykreslit osvětlené, stínované plochy.

Realizace obou druhů algoritmů se od sebe samozřejmě liší. V kapitolách 2.4.2 a 2.4.3 jsou uvedeny konkrétní případy některých z nich.

2.4.1 Předzpracování dat

Protože je výpočet všech algoritmů časově náročný, snaží se na začátku téměř všechny algoritmy analyzovat objekty ve scéně a snižují počet zpracovávaných rovinných ploch tím, že vyloučí stěny těles, které jsou odvráceny od pozorovatele (jsou zakryté jinými plochami). Tato operace se nazývá předzpracování dat.

Způsob zjištění, která stěna je odvrácená, bývá následující. Předpokládá se, že u každé plochy je znám její normálový vektor. Orientace této plochy se určuje porovnáním směru tohoto vektoru se směrem pohledu. Úhel, který svírá normála odvrácené plochy se směrem pohledu, je totiž vždy ostrý. Pokud se provede skalární součin těchto dvou vektorů, získá se kosinus tohoto úhlu. Pokud je tedy znaménko kladné, úhle je ostrý a daná plocha je odvrácená.



Obr. 53. Rozdělení hran na přední, zadní a obrysové (horní podstava krychle není záměrně vykreslena)

Po rozdělení ploch se ještě někdy provádí vyhodnocení hran. Hrana, která je sdílená přivrácenými stěnami, se nazývá **přední** a hrana mezi dvěma odvrácenými stěnami se nazývá **zadní.**

Velice důležitým typem hrany u algoritmů viditelnosti je hrana **obrysová** (*contour edge*) a v liniových algoritmech řešení viditelnosti indikuje možnou změnu viditelnosti. Popsané typy hran u krychle bez horní podstavy jsou uvedeny na obr. 53.

2.4.2 Liniové algoritmy viditelnosti

Typickým přestavitelem liniových algoritmů viditelnosti je **Robertsův algoritmus**. Tato metoda je programátorsky nejjednodušší a je použitelná pro libovolné mnohostěny. Podstata algoritmu spočívá v postupném zpracování všech hran těles kromě hran zadních. Každá hrana se testuje, zda není zakrývána jiným tělesem. Pokud je hrana zakrytá jen částečně, dochází k rozdělení hrany na zakrytou a nezakrytou část. Každá nezakrytá část se stává novou, testovanou hranou. Výstupem algoritmu je seznam všech hran, které je třeba vykreslit.

Robertsův algoritmus se pro svoji jednoduchost stal základem mnoha složitějších metod, které tento algoritmu upravují, např. v řazení hran. Přitom původní myšlenka rozdělení potenciálně viditelných hran na elementární úseky, na nichž se nemůže viditelnost měnit, zůstala zachována. K nalezení úseků s neměnnou velikostí se používají obrysové hrany. Pouze tyto indikují případnou změnu viditelnosti a mohou se dělit na menší úseky.

U nekonvexních mnohostěnů je Robertsův algoritmus pomalý. Urychlení přináší např. **Appelův algoritmus**, který zkoumá potenciálně viditelné úseky hran více důsledněji. Tyto hrany se porovnávají s obrysovými a vzájemné průsečíky se zaznamenávají a hodnotí, zda jsou vstupní nebo výstupní. Seřazené průsečíky pak určují viditelnost jednotlivých úseků dané hrany.

2.4.3 Rastrové algoritmy viditelnosti

2.4.3.1 Paměť hloubky (z-buffer)

Paměť hloubky (*Z-buffer, depth-buffer*) patří mezi nejefektivnější a nejčastěji používané rastrové metody viditelnosti objektů prostorové scény. Má větší nároky na paměť, protože si vytváří dvojrozměrné pole, jehož rozměry jsou totožné s rozměry zobrazovacího okna, obsahuje *z*-ové souřadnice vypočítaných bodů, které leží nejblíže pozorovateli, a jehož průmět leží v odpovídajícím pixelu rastru. Tato metoda je v současné době hardwarově implementována ve všech 3D grafických kartách.

Podstata algoritmu je následující. Obrazová paměť se vyplní barvou pozadí a paměť hloubky hodnotou $-\infty$. Pak se každý objekt scény převádí do rastrové podoby a tím se získají body se souřadnicemi *x*, *y* a *z*. Protože obrazovka je jen dvojrozměrná, objekty, které jsou překryty, by nemělo být vidět. Z-buffer slouží právě k tomu, aby se nestalo, že vzdálenější objekt, který je vykreslovaný jako poslední, nepřekryl objekty, které jsou blíže k místu pozorovatele. Do pole se tak ukládají *z*-ové

souřadnice příslušných bodů objektu a v případě, že nová hodnota je menší než ta, která je v paměti hloubky uložena, přepíše se původní hodnota.

Pořadí zpracovávaných ploch může být libovolné, přičemž každá plocha se zpracovává pouze jednou. Doba zpracování tedy roste lineárně s počtem ploch ve scéně, ale závisí i na jejich velikosti a rozměru obrazu. Při implementaci lze algoritmus paralelizovat například rozdělením paměti hloubky na pruhy.

2.4.3.2 Řádková paměť hloubky (scan-line z-buffer)

Tento algoritmus je podobný metodě paměti hloubky s tím rozdílem, že pro určení bodů (pixelů) nejbližších pozorovateli není vymezena rozsáhlá paměť hloubky, ale pouze jeden řádek (řádek hloubky). Charakteristickým rysem tedy je menší paměťová náročnost na úkor rychlosti zpracování.

Viditelnost se vyhodnocuje vždy pouze v jediném řádku, po jehož zpracování se přechází na další a tímto způsobem se zpracovávají všechny řádky obrazovky. Na začátku algoritmu se daný řádek v obrazové paměti vyplní barvou pozadí a řádek hloubky hodnotou $-\infty$. Každá plocha, která je protínána daným řádkem, se rozloží na pixely a pro každý z nich se stanovuje hloubka z. Pokud je tato z-hodnota větší než odpovídající položka v řádku hloubky, pak se obarví příslušný pixel v obrazové paměti barvou této plochy a položka v řádku hloubky se aktualizuje touto z-hodnotou.

Problémem této metody je, že každá plocha se zpracovává několikrát, což snižuje rychlost algoritmu. Zlepšení lze dosáhnout seřazením ploch podle souřadnic y s udržovaným seznamem aktivních hran těch ploch, které jsou protínány aktuálním řádkem. Seznam aktivních hran lze dále předzpracovat tak, aby vznikly úseky, ve kterých se nemění viditelnost. Tyto úseky lze přímo vykreslit na obrazovce, aniž by jednotlivé pixely byly testovány vůči řádkové paměti hloubky.

2.4.3.3 Malířův algoritmus

Malířův algoritmus (*painter's algorithm*) je založen na myšlence vykreslování všech ploch postupně odzadu dopředu obdobně, jako kdyby malíř nanášel na obraz nejprve barvy pozadí a přes ně kreslil objekty v popředí. Objekty, které jsou nejdále od pozorovatele se tak kreslí jako první a nejbližší objekty jako poslední. Tím se zajistí překrytí (překreslení) vzdálenějších objektů bližšími a tím i správná viditelnost.

Na obr. 54 vlevo by se tak nejprve vykreslil šestiúhelník, pak obdélník se zakulacenými rohy a nakonec hvězda, která je nejblíže pozorovateli. Otázkou zůstává, na základě jakého kritéria lze polygony ve 3D srovnávat, protože vzdálenost objektu nemusí být konstanta v každém jeho bodě. V úvahu připadají tři možnosti, podle toho, co se považuje za vzdálenost objektu od pozorovatele. Může to být minimální, maximální a nebo průměrná vzdálenost ze všech jeho bodů. Z důvodu rychlosti se často používá jedna průměrná souřadnice pro celý objekt. Vylepšit toto utřídění lze tak, že se zjistí

krajní souřadnice *z* každého objektu a podle jejich hodnoty se objekty setřídí. Plochy se ještě nevykreslují, protože některé objekty se mohou s jinými prolínat. Z tohoto důvodu se objekty zpravidla ještě testují s některými dalšími v jeho blízkosti. Pokud se na základě těchto testů prokáže, že polygon leží za ostatními, vykreslí se. V opačném případě dojde k jejich výměně s polygonem, kde test překrývání dopadl záporně.

Některé testy jsou založeny na porovnávání dvou souřadnic, jiné vyžadují složitější výpočty. V následujícím testu překrývání jsou testované objekty (plochy) označeny jako *P1* a *P2* a jejich krajní souřadnice ($z1_{min}$, $z1_{max}$) a ($z2_{min}$, $z2_{max}$). Testy se vykonávají postupně a při splnění jednoho se mohou ostatní vynechat:

1. Pokud se nepřekrývají průměty polygonů v rovině průmětny, je jejich pořadí vykreslování nezávislé.
2. Pokud platí $z1_{max} > z2_{min}$, pak je polygon *P1* zcela za polygonem *P2*.
3. Pokud jsou všechny vrcholy polygonu *P1* pod rovinou určenou polygonem *P2*, potom *P1* leží za polygonem *P2*.
4. Pokud jsou všechny vrcholy polygonu *P2* nad rovinou určenou polygonem *P1*, potom *P1* leží za polygonem *P2*.
5. Ve všech ostatních případech je nutné najít v průmětně libovolný průsečík obou ploch a podle odpovídajících *z*-souřadnic určit pořadí vykreslovaných ploch.

Obr. 54. Modelování scény pomocí malířova algoritmu bez a s cyklickým překrytím

Asi největší a jedinou výhodou malířova algoritmu je jeho jednoduchost na pochopení a naprogramování. Naopak nevýhod je zde hned několik – objekty je nutné zobrazovat podle vzdálenosti od pozorovatele, což je velice časově náročné zejména v případech složitějších scén (desítky tisíc polygonů). Dalším nedostatkem je neefektivnost, protože se spousta objektů vykresluje úplně zbytečně, protože jsou překryté jinými objekty (tento nedostatek lze odstranit kreslením objektů zepředu dozadu a vypočítané pixely už nepřekreslovat). Třetí nevýhodou je ta skutečnost, že

v určitých případech nedává správné výsledky, protože uvedené testy nejsou zcela dostatečné pro složitější případy vzájemného překrývání více ploch (viz obr. 54 vpravo). Tento stav lze odhalit, protože vlivem cyklického přesunu se na místo testovaného objektu dostane takový objekt, který již byl testován. Tuto situaci lze řešit rozdělením polygonu na menší části, což má za následek další snížení efektivity algoritmu.

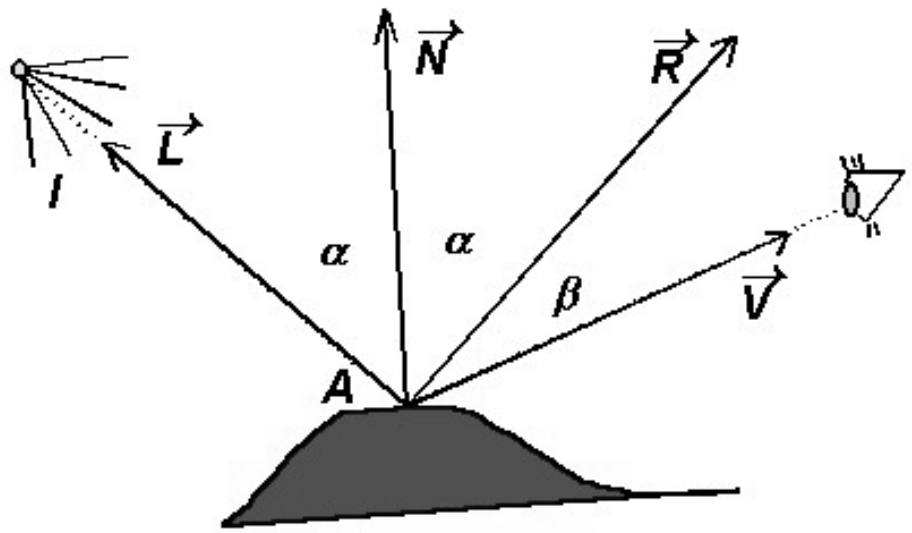
2.5 Zobrazovací modely a stínování

Lidské oko vnímá okolní svět prostřednictvím světla, které dopadem na oční sítnici přináší informace, ze kterých si lze vytvořit představu o barvě, umístění a tvaru objektů v prostoru. Viditelné světlo je frekvenční pásmo elektromagnetického spektra v oblasti $4{,}3*10^{14}$- $7{,}5*10^{14}$ Hz. Mezi těmito krajními frekvencemi dokáže lidské oko rozlišit až 400 000 různých barev.

Po dopadu světelného paprsku na povrch tělesa jsou některé frekvence odraženy a některé pohlceny. Kombinace frekvencí, které se nacházejí v odraženém světle, vytváří výslednou barvu, kterou oko vnímá. Pokud převládají nízké frekvence, je objekt vnímán jako červený. Naopak pokud převládají nejvyšší frekvence, předmět se jeví jako fialový.

Povrch tělesa odráží světlo do všech směrů. Matematická funkce, která vyjadřuje intenzitu paprsku rozptýleného světla v závislosti na jeho směru, intenzity a vlnové délky dopadajícího paprsku, se nazývá odrazová funkce a je základem zobrazovacích modelů. Pomocí odrazové funkce lze stanovit takové vlastnosti, jako je barva nebo lesklost objektu.

Světlo při odrazu mění svoje barevné složení a tím dává informaci o daném povrchu objektu. Počítačem simulovaný obrázek modelu prostorového objektu musí zohledňovat všechny fyzikální a optické vlastnosti povrchu, které jsou při určování výsledné realističnosti rozhodující.



Obr. 55. Odraz paprsku na povrchu tělesa

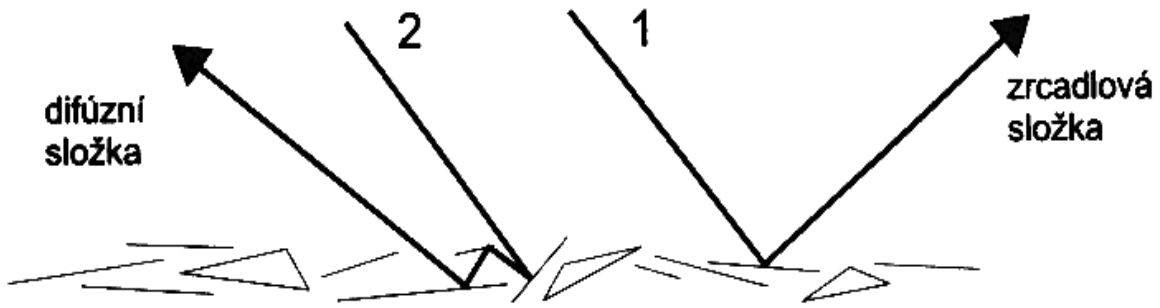
Na obr. 55 světelný paprsek ze zdroje I dopadá do bodu A a po odraze se světlo rozptyluje do všech směrů. Jednotkový vektor N kolmý na povrch v bodě A se nazývá normálový. Směrové jednotkové vektory L, V a N leží na přímkách dopadajícího paprsku světla, směru pohledu a normály povrchu. Povrch tělesa není nikdy dokonale plochý. Bývá tvořen drobnými ploškami, jejichž velikost a tvar jsou dány složením konkrétního materiálu.

Při sledování určitého bodu na povrchu se vnímá velké množství individuálních paprsků, které do tohoto bodu přicházejí, ale jejichž předchozí trajektorie mohou být rozdílné podle lámání a odrazu. Intenzita odraženého světla se dá proto vyjádřit jako součet dvou složek

$$I_V = I_S + I_D . \tag{51}$$

Složka I_S se nazývá zrcadlová (S – *specular*) a na obr. 56 je označená číslicí 1. Její hlavní vlastností je směrovost. Směrovost je tím vyšší, čím je povrch tělesa hladší. Světlo odražené od dokonalého zrcadla má pouze tuto složku. Zrcadlová složka je příčinou vzniku odlesků na tělesech, přičemž odlesky mohou mít jinou barvu, než má povrch tělesa (např. lesklý povrch černých brýlí).

Složka I_D se nazývá difúzní (D – *diffuse*) a na obr. 56 je označená číslicí 2. Pro ni je charakteristické to, že nezávisí na směru pohledu. Po vícenásobném odrazu a lomu je totiž směr paprsku náhodný se stejnou pravděpodobností všech směrů. Velikost této složky závisí jen na kosinu úhlu dopadu. Difúzně odražené světlo přináší informace o tom, co se nazývá barvou povrchu.



Obr. 56. Mikroskopická úroveň při odrazu světla

2.5.1 Zobrazovací techniky a stínování

Zobrazovací techniky počítají nasvícení objektu od světelných zdrojů a následně provádí vystínování objektu. Významným způsobem tak ovlivňují reálný vzhled prostorové scény. Existují dvě skupiny zobrazovacích technik – **lokální**, kde se nasvícení každého objektu provádí samostatně, a **globální**, kde se současně počítá osvětlení všech objektů ve scéně a počítá se i s polohou objektů vůči sobě.

V této části jsou vysvětleny lokální zobrazovací techniky. Globální zobrazovací metody, které berou v úvahu vzájemnou interakci objektů, jsou probrány v kapitole 2.6.

2.5.1.1 *Lokální zobrazovací techniky*

Jak již bylo řečeno, tyto techniky počítají nasvícení každého objektu od světelných zdrojů samostatně. Neberou tedy v úvahu, že mezi objektem a světelným zdrojem může být jiný objekt, který upravuje nebo zamezuje průchodu světelného paprsku. Do této skupiny patří **plošné stínování**, **Gouraudovo stínování** a **Phongovo stínování**. Každá z těchto metod řeší stínování svým vlastním způsobem a liší se kvalitou stínování a rychlostí algoritmu.

Plošné stínování

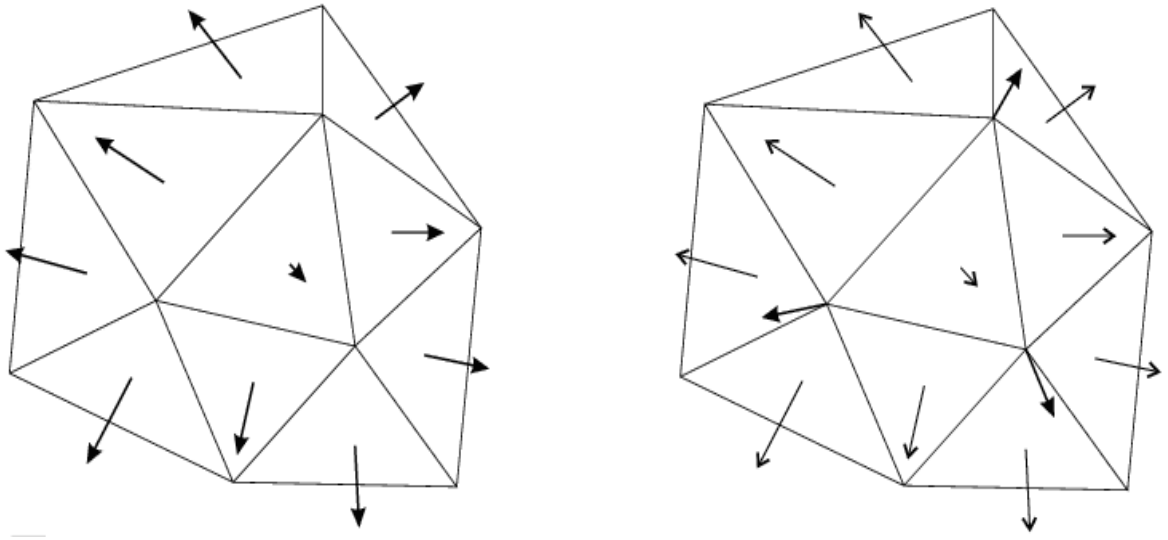
Plošné stínování, nazývané také někdy konstantní stínování (*flat shading*) je nejjednodušší a zároveň nejrychlejší. Používá se pro zobrazování rovinných ploch nebo ploch, které jsou aproximovány rovinnými ploškami. Pro každou plochu je obvykle dán normálový vektor (obr. 57 vlevo), podle kterého se určuje příslušný barevný odstín, který se po rasterizaci přiřadí všem bodům této plochy.

Algoritmus pracuje následovně: vypočítají se normálové vektory pro všechny plošky, ze kterých je objekt složen. Pak se pro každou složku určí odchylka normálového vektoru a vektoru paprsků světelného zdroje. Čím více je ploška ke světelnému zdroji přivrácena, tím více ji světelný zdroj ovlivňuje. Na základě odchylky se přiřadí každé plošce jedna barva, kterou bude vyplněna.

Tento typ stínování je vhodný pro rychlé zobrazování těles (tzv. *preview*) a naopak není vhodný tam, kde je potřeba kvalitní zobrazení.

Gouraudovo stínování

Gouraudovo stínování (*Gouraud shading*) je někdy též nazýváno jako Interpolace barvy. Jeho podstata spočívá v přiřazení barvy ke krajním bodům plošky a pro ostatní body plošky provede interpolaci barvy tak, aby se dosáhlo plynulého přechodu barev.



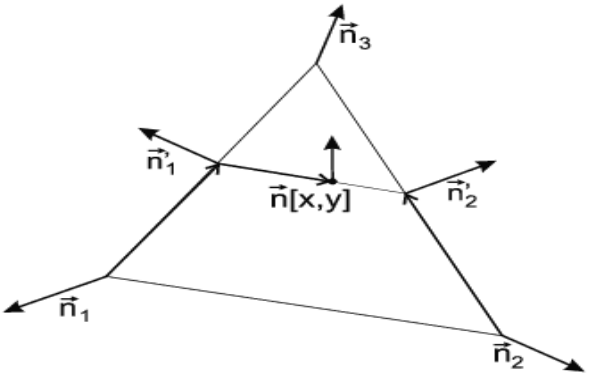
Obr. 57. Normálové vektory u Plošného (vlevo) a Gouraudova (vpravo) stínování

Algoritmus pracuje následovně: na začátku se vynulují vektory ve všech vrcholech a vypočítají se postupně normálové vektory pro všechny plošky, ze kterých je objekt složen. Každá normála plošky se přičte k vektorům ve všech vrcholech této plošky (obr. 57 vpravo). Poté se vektory vypočítané ve vrcholech normalizují a určí se jejich odchylka s vektorem paprsků světelného zdroje. Poté se již provádí samotná interpolace (nejprve po hranách).

Tento algoritmus je sice pomalejší než Plošné stínování, ale umožňuje dobře zobrazit i stínované objekty. V současné době již bývá i hardwarově implementován do většiny akcelerátorů a také bývá pravděpodobně nejčastěji používanou metodou stínování. Nevýhodou je, že nebere v úvahu vliv odlesků světla.

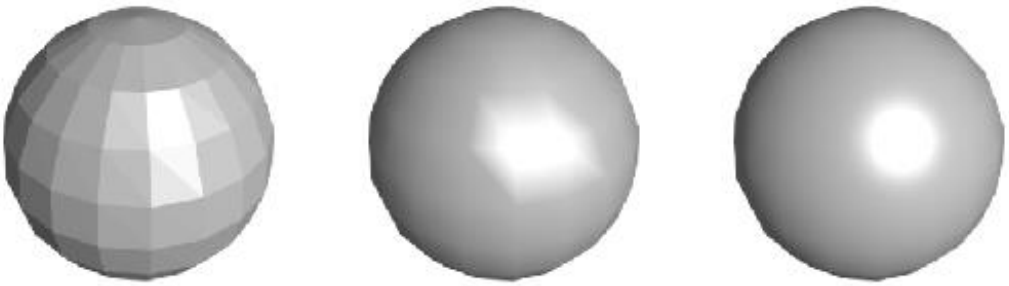
Phongovo stínování

Phongovo stínování (*Phong shading*) je někdy též nazýváno jako Interpolace normály a snaží se určit přesně barvu pro každý bod plošky. Podstata algoritmu je následující: provede se výpočet normál pro všechny plošky, ze kterých je objekt složen, a podobně jako v Gouraudově stínování se provádí výpočty normál ve vrcholech. Pak se ale nepřiřazují barvy do vrcholů, nýbrž se provádí interpolace normály pro jednotlivé body plošky. V okamžiku, kdy jsou pro všechny body na plošce určeny normály, určí se odchylky těchto normál od vektorů světelných paprsků. Podle těchto odchylek se každému bodu přiřadí příslušná barva.



Obr. 58. Určení barvy bodu v plošce u Phongova stínování

Algoritmus Phongova stínování je sice početně nejnáročnější (a tím i nejpomalejší), ale dosahuje nejlepšího zobrazení objektu. Porovnání s předchozími metodami demonstruje obr. 59.



Obr. 59. Vystínovaná koule plošným (vlevo), Gouraudovým a Phongovým algoritmem (vpravo)

2.5.2 Světelné zdroje

Světelný zdroj je objekt, který emituje světelné záření. Dalším charakteristickým rysem světelných zdrojů je ta skutečnost, že jim v počítačové grafice nepřísluší žádný geometrický tvar (samy o sobě se nezobrazují). Navenek se tak projevují pouze charakteristikou světla, které vysílají.

V reálném světě mívají světelné zdroje rozmanité podoby – žárovka, zářivka, LED dioda, plápolající oheň, sluneční korona apod. V počítačové grafice se používá několik kompromisních druhů světel, které do jisté míry aproximují nejčastější případy reálných světelných zdrojů. Toto opatření však komplikuje takové výpočty, jako jsou vržené stíny, proto pokud se používá více druhů světelných zdrojů, musí se u každého z nich definovat způsob jeho šíření v prostředí apod.

U některých světelných zdrojů se uvažuje aproximace útlumu intenzity světla procházejícího homogenním prostředím. Intenzita světla I klesá se vzdáleností r od zdroje světla podle vztahu

$$I = \frac{1}{k_1 + k_2 r + k_3 r^2}\,. \tag{52}$$

Koeficienty k_1, k_2 a k_3 jsou parametry pro konstantní, lineární a kvadratickou závislost útlumu na vzdálenosti r.

Bodový zdroj

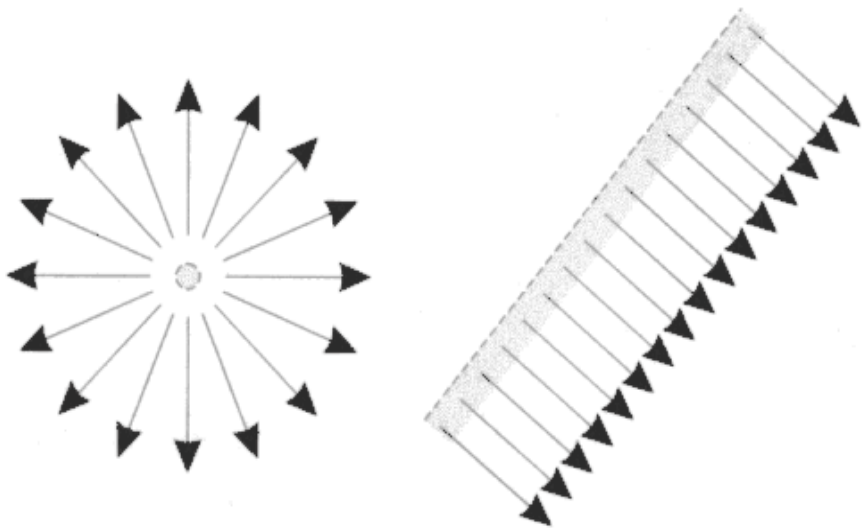
Je nejčastěji používaným světlem v počítačové grafice (obr. 60 vlevo). Světlo se šíří rovnoměrně a se stejnou intenzitou do všech směrů. Produkuje pouze ostré hranice stínů a nachází svoje uplatnění v metodě sledování paprsku a ve Phongově zobrazovacím modelu (viz předchozí kapitola). Útlum u tohoto světelného zdroje se zpravidla počítá podle vztahu (52). Bodový světelný zdroj je jednoznačně určen svoji intenzitou a polohou.

Rovnoběžný zdroj

Tento typ světelného zdroje (obr. 60 vpravo) lze chápat jako bodový zdroj ležící v nekonečnu. Světelné paprsky, které se z tohoto zdroje šíří, jsou rovnoběžné. Je určen intenzitou světelného zdroje v nekonečnu a směrem, proto se předpokládá, že paprsky světla se šíří kolmo k této ploše. Intenzita obvykle neklesá se vzdáleností, ale zůstává v celém prostoru konstantní. Příkladem zdroje rovnoběžného světla může být slunce, jehož záření dopadá na Zemi.

Plošný zdroj

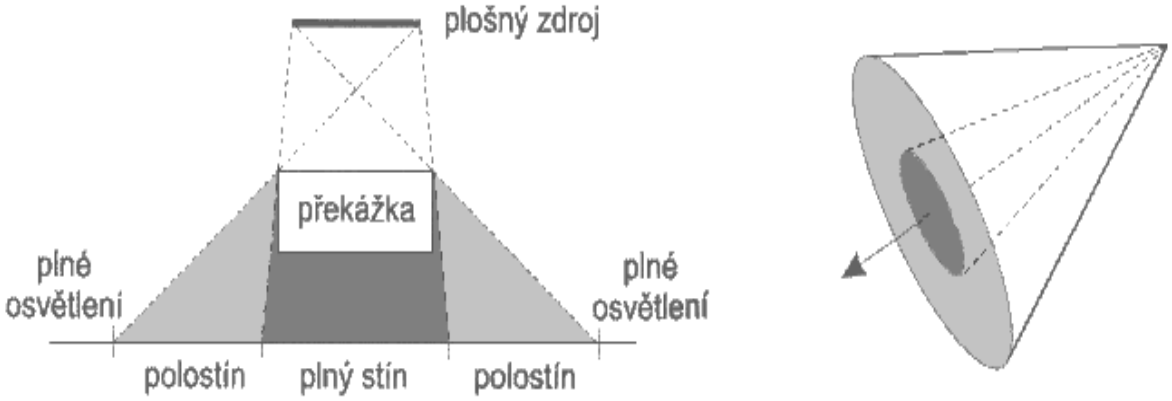
Plošný zdroj (obr. 61 vlevo) se nejvíce podobá reálným zdrojům, jako je například zářivka nebo okno, kterým proudí světlo do místnosti. Je definovaný jako orientovaný polygon charakterizovaný vlastní zářivostí. Jeho nevýhodou jsou vysoké nároky na výpočet, což je způsobeno realisticky vypadajícími stíny, které se pro jiné typy světelných zdrojů vytváří jen obtížně. Možnost, která se nabízí, je náhrada plošného zdroje několika zdroji bodovými.



Obr. 60. Šíření paprsků z bodového zdroje (vlevo) a rovnoběžného zdroje (vpravo) světla

Reflektor

Reflektor (*spot light*) je směrově závislý zdroj světla, který je určený polohou a orientací. Lze si ho představit jako kužel, v jehož vrcholu je světelný zdroj. Světelná intenzita je maximální ve směru, ve kterém svítí, a kolmo k tomuto směru exponenciálně klesá. Intenzita světla z reflektoru obvykle klesá se vzdáleností podle vztahu (52). Realističtější reprezentace reálného reflektoru je dvojice souosých kuželů se společným vrcholem (obr. 61 vpravo). Vnitřní kužel představuje svazek jasného světla, jehož intenzita klesá pouze se vzdáleností od světelného zdroje, ale ne se vzdáleností od osy kuželu. Vnější kužel určuje hranici, za kterou světlo vůbec nedopadá a v oblasti mezi pláštěm vnitřního a vnějšího kužele se světlo tlumí s ohledem na vzdálenost od osy.



Obr. 61. Šíření paprsků z plošného zdroje světla (vlevo) a reflektoru (vpravo)

2.6 Globální zobrazovací metody

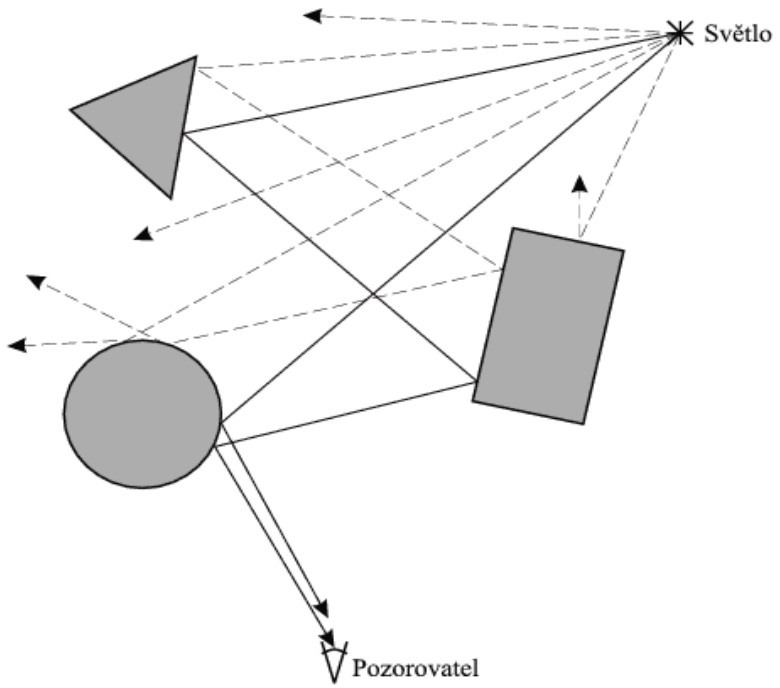
V předchozí kapitole se uvedené zobrazovací metody vyznačovaly tím, že se objekty ve scéně z hlediska osvětlení neovlivňují. Pro každý objekt byl individuálně vyhodnocen zobrazovací model tak, jako by byl tento model ve scéně sám a byl nasvícen ze všech světelných zdrojů.

Globální zobrazovací metody počítají vzhled všech objektů ve scéně současně, včetně toho, jak objekty navzájem stíní nebo jinak ovlivňují světelné paprsky. Berou tedy v úvahu vzájemnou interakci jednotlivých objektů. Tyto metody patří v počítačové grafice mezi časově nejnáročnější, ale zároveň poskytují velice kvalitní obrazy prostorových scén, které se jen stěží dokáží odlišit od reálného světa. Z tohoto důvodu jsou také nazývány metody pro **fotorealistické zobrazování** (*photoralistic rendering*).

Do této skupiny patří dvě techniky, které se liší svým přístupem ke světlu. První z nich je **částicový pohled na světlo**, kde se vychází z pohledu, že světlo šíří jednotlivé fotony a při výpočtu se uvažují jednotlivé paprsky, ve kterých se světlo šíří. Druhý pohled je **vlnový**, který vychází ze skutečnosti, že se světlo šíří ve vlnoplochách.

2.6.1 Částicový pohled na světlo

V těchto metodách se nejčastěji používají techniky **sledování paprsku** (***ray tracing***) a **vrhání paprsku** (***ray casting***). Oba pojmy pocházejí z optiky, ve které se používají pro určení vlastností optických systémů s čočkami.



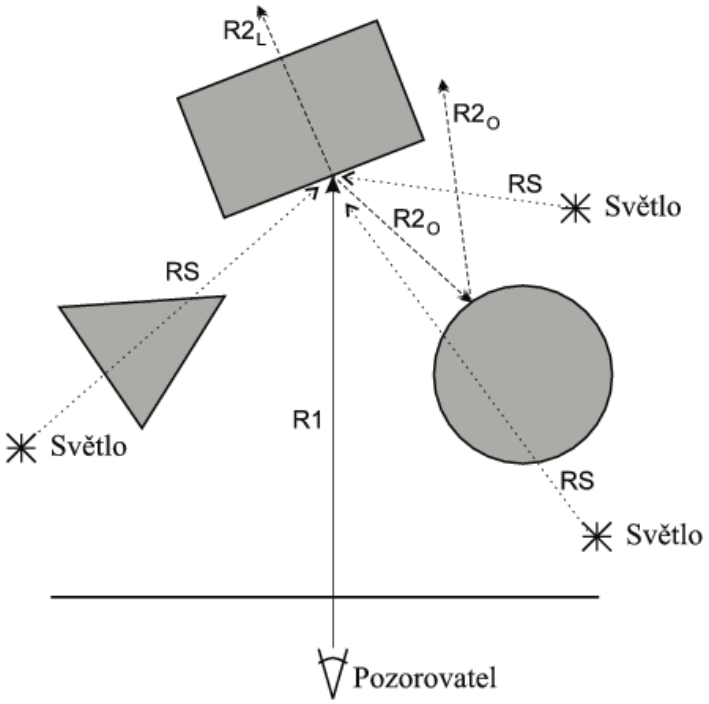
Obr. 62. Šíření paprsků světla ve scéně

Jak vyplývá z fyzikální podstaty světla, paprsky se šíří od světelných zdrojů různými směry (obr. 62). Některé z paprsků zasáhnou povrch objektů ve scéně, jiné prostor scény opustí. Podle optických vlastností těchto objektů se paprsky odrážejí, případně lomí a ovlivňují osvětlení v dalších místech scény.

Postupovat při výpočtech scény tímto způsobem by bylo značně zdlouhavé, protože spousta paprsků pozorovatele mine. Z tohoto důvodu se v počítačové grafice používá obrácený postup sledování šíření světla. Paprsek se promítne ze stanoviště pozorovatele přes bod v rovině obrazovky (pixelu) do scény. Výsledkem tohoto postupu je odpověď na otázku, co je v daném bodě scény vidět a jaké informace o světle tento paprsek přináší. Proto se tato metoda také někdy nazývá **zpětné sledování paprsků**.

Ray casting se od *ray tracingu* odlišuje tak, že se zobrazuje pouze bod na povrchu nejbližšího tělesa zasaženého paprskem. V tomto bodě se stanoví barva pomocí některého z jednoduchých lokálních zobrazovacích technik (viz kap. 2.5.1.1). Vzhled takto vzniklých obrázků není příliš odlišný od obrazů vzniklých jednoduššími zobrazovacími metodami (např. Phongovým stínováním), význam má však zejména pro zobrazování modelů v CSG reprezentaci (kap. 2.2.4).

Naproti tomu u *ray tracingu* sledování paprsku nekončí po nalezení nejbližšího tělesa, ale pokračuje sledováním dalších paprsků odvozených podle odrazivosti a průhlednosti nalezeného tělesa. V tomto případě se opět vidí povrch nejbližšího tělesa, na jehož barvě se ovšem podílejí i podíly světla zjištěné ostatními paprsky. Tato metoda dokáže zobrazit na povrchu tělesa i zrcadlové obrazy jiných těles a zpracovat vržené stíny.



Obr. 63. Sledování jednoho primárního paprsku světla (R1)

($R2_O$ – sekundární paprsek odražený, $R2_L$ – sekundární paprsek lomený, R_S – stínový paprsek)

Pro jednotlivé paprsky ve scéně (obr. 63) se zavedlo následující označení:

- **Primární paprsek (***primary ray***)** je vysílán z místa pozorovatele bodem na obrazovce (pixelem) do scény. Celkový počet primárních paprsků je tak dán rozlišením obrazového okna.
- **Sekundární paprsek (***secondary ray***)** je vytvořen po dopadu primárního nebo sekundárního paprsku na nějaký objekt ve scéně. Reprezentuje stav, kdy se předchozí paprsek od objektu odrazil nebo pronikl dovnitř poloprůhledného objektu. V daném bodě tedy mohou vzniknout až dva sekundární paprsky (odražený a lomený).
- **Stínový paprsek (***shadow ray***)** je vyslán z bodu, kam dopadl primární nebo sekundární paprsek směrem ke světelnému zdroji. Jeho úkolem je zjistit, zda mezi tímto bodem a světelným zdrojem není nějaká překážka zastiňující těleso. Pokud není žádná překážka nalezena, je tento světelný zdroj zahrnut do vyhodnocení zobrazovacího modelu v daném bodě.

Vlastní algoritmus realizace metody sledování paprsku lze rekurzivně zapsat takto:

1. Vyšli primární paprsek a nalezni průsečík *P* tohoto paprsku s nejbližším tělesem ve scéně.
2. Pokud takový průsečík neexistuje, paprsek opustil prostor scény - stačí přiřadit barvu pozadí a program skončí.
3. Z bodu *P* vyšli stínové paprsky ke všem zdrojům světla. Pokud stínový paprsek dojde až do zdroje, označ jej jako nezakrytý.
4. Vyhodnoť příspěvky osvětlení v bodě *P* od všech nezakrytých světelných zdrojů.
5. Pokud hloubka vnoření rekurze nepřekročila maximální hloubku sledování, urči sekundární paprsky (odražený a lomený paprsek) a pro ně znovu rekurzivně použij tuto funkci.
6. Výsledná barva paprsku se stanoví jako součet příspěvků osvětlení barvy odraženého a lomeného sekundárního paprsku.

Jak již bylo řečeno, touto metodou lze dobře zobrazit stíny objektů ve scéně. Dále jsou optimálně řešeny odrazy objektů ve scéně, zrcadlové odrazy světla a řešení průhlednosti objektů. Obecně je však tato metoda velmi pomalá, protože mají paprsky různé počátky a ve scéně se pohybují různými směry. Hledání průsečíku paprsku s nejbližším tělesem přitom u základního algoritmu zabírá více než 90 % času. To je důvodem, proč se ve většině programů používají nejrůznější metody, které dávají výsledky rychleji. Klasický *ray tracing* se používá pouze u speciálních objektů ve scéně, které mají nastavenou průhlednost nebo zrcadlo. Souhrnu všech použitých metod pro reálné zobrazení scény (včetně *ray tracingu*) se říká **rendering**.

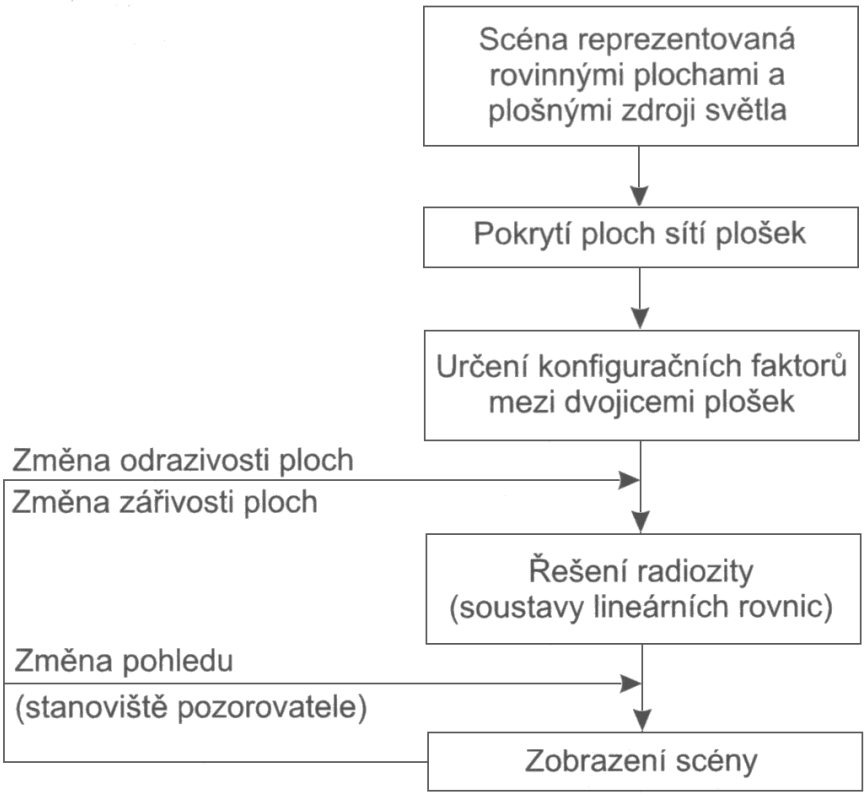
2.6.2 Vlnový pohled na světlo

Výsledné obrazy metody sledování paprsku uvedené v předchozí kapitole ještě neposkytují fyzikálně věrnou simulaci reálného světa. Klasický *ray tracing* povoluje pouze bodové zdroje světla a nehodí se tak k vyjádření nepřímého difúzního osvětlení. Vlnový pohled na světlo umožňuje korektně simulovat šíření světla scénou a dosáhnout tak fotorealistické věrnosti počítaných obrazů.

Metoda, která toto provádí, se nazývá **radiační** (*radiosity method*) a využívá poznatky z oblasti výpočtu tepelného záření pro výpočet záření světelného. Jak tato metoda pracuje? Schéma postupu je na obr. 64. Předpokládá, že se všechna tělesa ve scéně skládají z plošek nebo jsou ploškami alespoň aproximována. Tyto plošky se chovají jako ideální difúzní zářiče (vyzařují dopadající energii do všech směrů bez ohledu na to, odkud přichází). Zobrazovaná scéna je přitom chápána jako izolovaná od svého okolí. Ve scéně je několik světlých plošek, které samy září světelnou energii (tzv. primární zářiče). Zbylé plošky jsou na počátku tmavé. Primární zářiče emitují energii a ta dopadá na ostatní plošky ve scéně. Tyto plošky část energie pohltí a zbytek emitují zpět do scény. Tím se z nich stávají sekundární zářiče. Radiační metoda si klade za cíl spočítat pro všechny plošky ve scéně množství emitované energie po ustálení tohoto děje. Barva každé plošky se potom spočítá z intenzity energie, kterou emituje. Metoda vychází z termodynamického modelu pro výpočet sálavého tepla. Předpokládá, že se zobrazovaná scéna skládá z plošek a tvoří energeticky uzavřený objem. Složitější tvary jsou těmito ploškami aproximovány. Všechny plochy jsou ideálními difúzními primárními nebo sekundárními světelnými zářiči (tj. rozptylují světlo do všech stran bez ohledu na to, odkud na plochu dopadá). Intenzita energie je pro všechny body dané plošky konstantní.

Klíčovou roli zde hraje otázka, jak jedna ploška ve scéně ovlivňuje ostatní a jaká část světelné energie z jedné plošky dopadá na jinou plošku. Tento přenos energie lze popsat integrální rovnicí. Pro celou scénu se takto dostane soustava integrálních rovnic. Jejich sestavením a vyřešením je možné získat hledané emise jednotlivých plošek.

Radiační metoda je založena na fyzikálně přesnějším modelu šíření světla scénou než metoda rekurzivního sledování paprsku. Přesto má několik vážných nedostatků, které dosud brání jejímu většímu rozšíření. Časové nároky této metody nejsou o nic menší než u rekurzivního sledování paprsku. Původní metoda měla též obrovské paměťové nároky. Naštěstí dnes již existují její modifikace, které tyto paměťové nároky citelně zmenšily. To jsou ale zatím pouze technické nedostatky. Radiační metoda má však i jiná slabá místa. Místo toho, aby se popisovala samotná tělesa ve scéně například pomocí CSG stromu, musí se celá zobrazovaná scéna popsat pomocí plošek, ze kterých se skládá. Dále radiační metoda předpokládá, že intenzita vyzařované energie je pro celou plošku konstantní a že se všechny plošky chovají jako ideální difúzní zářiče nebo reflektory. Postihuje sice dobře plošné zdroje světla (oblaka, okno), ovšem bodové zdroje světla korektně postihnout neumí (při zachování rozumných paměťových nároků). Dále je velmi obtížné postihnout zrcadlový obraz či lom světla při průchodu průhledným tělesem.



Obr. 64. Postup při zobrazování pomocí radiační metody

Z předchozího je zřejmé, že tato metoda má své výhody oproti jiným, jako je *ray tracing*. Má ovšem i nevýhody. Bližším studiem obou metod si lze povšimnout, že se obě tyto metody (radiační metoda a *ray tracing*) navzájem dobře doplňují. Hlavní nevýhody jedné metody odstraňuje druhá a naopak. Je tedy pochopitelné, že kromě odstraňování nevýhod jednotlivých metod jejich vlastními prostředky, je patrná snaha o kombinaci těchto metod. Radiační metoda umí dobře postihnout difúzní světlo a je nezávislá na pozici pozorovatele. *Ray tracing* vyniká v zobrazování lesklých a průhledných těles. Základní způsob, jak spojit tyto dvě metody, spočívá v následujícím postupu. Nejprve se pomocí radiační metody vypočítá difúzní světlo. Potom je možné začít scénou "procházet". Vždy při změně pozice pozorovatele se nejprve vykreslí směrově nezávislá difúzní složku světla. Pokud se při průchodu scénou zastaví (nebo je-li k dispozici dostatečně výkonný hardware), spustí se algoritmus *ray tracingu* a přidají se k obrázku zrcadlové odrazy (stačí se zaměřit jen na ta tělesa, která jsou lesklá, a nemusí se použít *ray tracing* pro celý obrázek). Takto je možné v reálném čase procházet scénou a při pohledu na některý objekt podrobněji se stačí na chvíli zastavit.

Tento způsob spojení uvedených metod je relativně jednoduchý. Nelze však od něj očekávat zcela reálné obrázky. Především zde chybí zachycení zrcadlového odrazu v první fázi, v rámci radiační metody. Například nevznikne světlé místo tam, kam se odráží od zrcadla primární světelný zdroj (známé "prasátko"). Nemůže se tedy ani podílet na změně světelných poměrů ve svém okolí. Tyto jevy je spíše možné postihnout zachycením směrové povahy světla přímo v radiační metodě. Ze

všech těchto uvedených nedostatků vzniklo několik verzí radiační metody, jejichž bližší popis je uveden např. v [13].

2.7 Textury

Textura je v podstatě vlastnost povrchu, která fyzickým způsobem neovlivňuje geometrický tvar objektu, na který se nanáší. Představuje tedy další krok ke zvýšení prostorovému vjemu syntetických scén. Důležitým pojmem je i **hypertextura**. Zatímco textura určuje vlastnost bodu na povrchu objektu, hypertextura definuje vlastnosti okolí tohoto bodu ve směru normálového vektoru, tedy směrem ven z objektu. Vhodnou aplikací hypertextury je tvorba modelů mlhavých objektů, hořícího dřeva, atmosféry Země, srst medvěda apod.

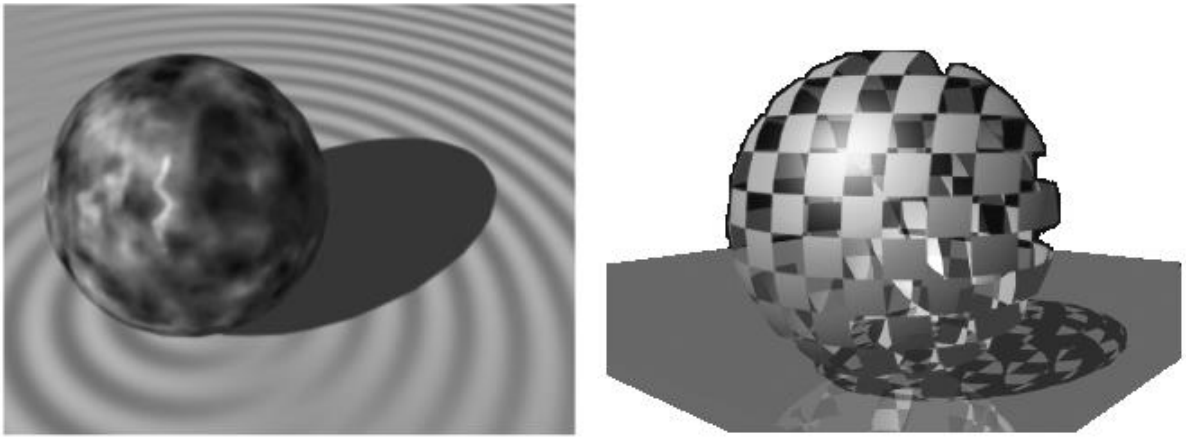
Textura může být určena v zásadě dvěma způsoby. Prvním z nich je definice pomocí tabulky, která ve dvojrozměrném případě představuje obrázek, v případě třírozměrném pak představuje pole voxelů, tj. rovnoměrně rozmístěných bodů ve třírozměrném prostoru, ve kterých je uložena hodnota barvy. Typickým a nejčastěji aplikovaným postupem je "nanesení" obrázku na modelovaný objekt. K tomu, aby bylo možné tuto operaci provést, je nutné určit kam a jakým způsobem se obrázek na objekt položí. Pro tento účel se na povrchu objektu zavádí lokální souřadnicový systém, který je dvojrozměrný. Každý souřadnicový systém má svůj počátek a tomuto bodu odpovídá i nějaký bod textury. Je-li určen počátek souřadnicového systému, určí se kam ukazuje na texturovaném objektu kladný směr osy u, čímž se souřadnicový systém zorientuje. Poté se již může začít textura nanášet na povrch objektu. Tento krok se (zjednodušeně řečeno) provádí tak, že se postupně vypočítávají souřadnice u a v pro body na povrchu zkoumaného objektu a určují se odpovídající body textury. Orientace souřadnicového systému na povrchu objektu určuje orientaci textury.

Tímto způsobem se však s takto definovanou třírozměrnou texturou příliš často nepracuje především ze dvou důvodů. Tím důležitějším je obtížná možnost získání takovéto textury. Zatímco dvojrozměrný snímek se snadno pořídí pomocí scanneru či fotoaparátu, třírozměrné snímky se získávají komplikovaně. Druhým důvodem jsou extrémní paměťové nároky třírozměrných textur reprezentovaných pomocí tabulky. Z těchto důvodů je pro popis třírozměrných textur výhodnější druhá možnost reprezentace, kterou je definice podle procedury. Procedurální textury jsou paměťově velice šetrné, neboť se v podstatě jedná o jednoduché programy, které na žádost aplikace poskytují odpovídající hodnotu textury v daném bodě. Většina procedurálních textur je založena na fraktálních technikách, které umožňují efektivní popis textur různých mlh, šumů, mraků ... atp. Procedurální třírozměrné textury se používají pro modely mramoru a dřeva.

Jiný způsob dělení textur je podle vlastností povrchu, který popisují:

- **Barva povrchu** je určena koeficientem difúzního odrazu. Toto je nejčastější případ definice textury.

- **Odraz světla** se může měnit s místem povrchu. Způsobuje odraz okolních objektů (lesklost) na povrchu textury. Proto také bývá označována jako mapování okolního prostředí (*enviroment maping*).
- **Změna normálového vektoru** opticky mění tvar povrchu, bez toho aby se změnila geometrie objektu. Výsledkem je povrch, který vypadá geometricky změněný. Příkladem může být hrbolatá textura (*bump maping*).
- **Drsnost** povrchu se též může měnit s místem. Obdobně jako u změny normálového vektoru opticky mění tvar povrchu.
- **Průhlednost** nemusí být na celém povrchu konstantní, ale může být proměnlivá se změnou místa. Přitom dochází ke změně geometrických vlastností objektu.

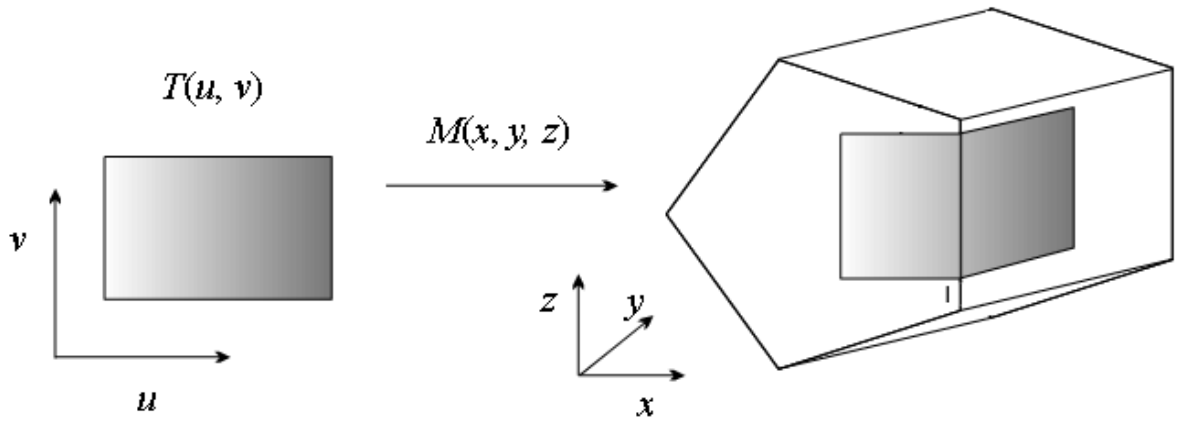


Obr. 65. Příklad hrbolaté (vlevo) a průhledné (vpravo) textury

2.7.1 Mapování textur

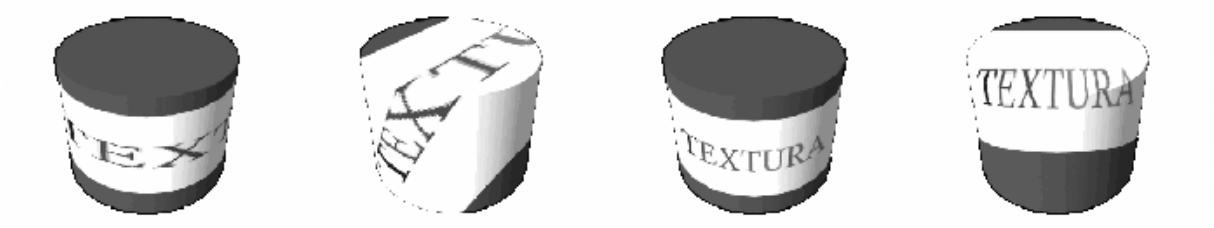
Mapováním (*texture mapping*) se nazývá proces nanášení textury na povrch tělesa. Způsoby mapování se liší podle definice textury, struktury tělesa, na které se textura nanáší, a mapované veličiny na povrchu tělesa.

Rovinná textura může být definována jako funkce $T(u, v)$, která přiřazuje bodům $[u, v]$ v rovině různé hodnoty. Pro aplikaci se používá funkce $M(x, y, z)$, která přiřazuje každému bodu na povrchu tělesa bod z definičního oboru textury T. Funkce M je označována jako **inverzní mapování** (*inverse mapping*). Při vykreslování tělesa jsou známy souřadnice $[x, y, z]$ jednotlivých bodů na povrchu a hledají se k nim informace o mapované (zobrazované) textuře (obr. 66). Použití textury si lze představit jako polepení tělesa tapetou. Funkce T odpovídá tomu, co je na papíru nakresleno a funkce M způsobu, jakým je nalepen.



Obr. 66. Mapování rovinné textury

Způsob umístění textury na objekt tedy určuje poloha textury (obr. 67 - texturou lze otáčet, posouvat ji po povrchu, měnit její měřítko atd.) nebo funkce *M*. Pro jednoduché objekty, jako je koule, válec či kvádr, lze tyto funkce vyjádřit přesně, pro jiné objekty (třeba fraktály či implicitní tělesa) tyto funkce vůbec neexistují. Pro naprostou většinu objektů tyto souřadnice existují, ale nechovají se tak, jak by bylo potřeba, nebo je obtížné je nalézt. Pro takovéto objekty se potom používají jiné mapovací techniky. Druhým problémem je, že textura nemusí na těleso přesně "padnout". Může ji někde kus přebývat nebo naopak kus chybět. Jednoduchým řešením bývá opakování textury, jinde je zase výhodné texturu natáhnout či stlačit. Typickým problémem při mapování textur jsou póly koule, rohy a hrany krychle ... apod.



Obr. 67. Různé možnosti mapování jedné textury

Při znalosti mapovací funkce mezi texturou a povrchem objektu se nemusí modifikovat pouze difúzní složku odrazu, ale je možné rovněž měnit průhlednost povrchu objektu. Korespondence mezi barvou v obrázku, který reprezentuje texturu, a povrchem objektu lze určit několika způsoby. Při částečné průhlednosti se používá intenzita barvy v obrázku k řízení průhlednosti odpovídajících bodů například tak, že maximální intenzita má význam totální průhlednosti a poloviční intenzita průhlednosti poloviční. Černobílé obrazy umožňují generování "okousaných", či například rzí poničených objektů. Lze například stanovit, že černá barva v černobílé textuře odpovídá bodu, který je neprůhledný, a barva bílá koresponduje s průhledným bodem. Takto mapované textury vytvářejí modely zkorodovaných objektů, zatímco zcela pravidelné černobílé obrazy dovolují jednoduše

vytvořit z koule model stínítka na lampičku nebo například z elipsy model tenisové rakety. Obrázek děrované koule na obr. 65 vpravo vznikl právě tímto způsobem.

2.7.2 Promítané textury

Společnou vlastností textur uvedených v předchozích odstavcích je jejich nezávislost na směru, ze kterého se na příslušný objekt dívá pozorovatel. Například při otáčení koule, na které je nanesena textura znázorňující zemský povrch, se vytvoří dojem rotující zeměkoule. Rotací objektu, na který je nanesena průhlednostní textura, se získá otáčející se děravý objekt. Tato vlastnost však neplatí pro tzv. promítané textury (*projected texture, chrome mapping*). Proč tomu tak je, je patrné z toho, jak se tyto textury získávají. Promítané textury se mapují tak, že texturovaný objekt se považuje za zrcadlovou plochu a textura se v něm jednoduše odráží. Při otáčení takovéto koule se bude jevit jako by byla v naprostém klidu. Pokud se ale pohne pozorovatel, textura nebo texturovaný objekt, povrch objektu se vizuálně pozmění.

Tento druh textur se výpočetně mapuje poměrně náročným způsobem. V prvním kroku se textura - obrázek nanese pomocí *u* a *v* mapovací funkce na vnitřní povrch nějakého uzavřeného tzv. obklopujícího objektu, nejčastěji se používá koule, krychle nebo válec. Poté se objekt, na který má být textura nanesena, umístí do středu obklopujícího objektu a po této přípravné fázi následuje vlastní mapování textury. Mapování probíhá tak, že se v poloze pozorovatele vytvoří obraz všech viditelných bodů objektu, na který je nanášena textura. Tento objekt je považován za ideální zrcadlo, a proto se na místo, kam dopadl pohled, namapuje ten bod z obklopujícího objektu, který se získá zrcadlovým odrazem pohledu od povrchu objektu. Z tohoto popisu je zřejmé, že obraz, který je mapován na vyšetřovaný objekt, závisí na poloze pozorovatele a na poloze objektu vzhledem k obklopující obálce. Vzhledem ke schopnosti této textury věrně modelovat zrcadlové odrazy bývá tento postup často používán jako simulace metody rekurzivního sledování paprsku, tedy všude tam, kde je zapotřebí vyrábět kvalitní obrázky a na jejichž výpočet není dostatek času. Zatímco při sledování paprsku se vždy vypočítává mnohonásobně odraz jediný, texturovaný objekt je umístěn uvnitř uzavřeného objektu a je tedy jisté, že odražený paprsek musí vždy zasáhnout nějaký bod textury.

2.7.3 Hrbolaté textury

V počítačové grafice hraje klíčovou úlohu kolmice k povrchu objektu (normála, normálový vektor). Zcela zásadní vliv má tento vektor při výpočtu osvětlení vyšetřovaného bodu. Z polohy pozorovatele, z polohy světel a ze směru kolmice k povrchu v určitém bodě lze vypočítat barvu tohoto bodu, vržené stíny aj. Modulací normálového vektoru lze získat tzv. hrbolatou texturu (*bump texture, bump mapping*). Čím je povrch objektu hrbolatější, tím komplikovaněji se mění směr normály mezi dvěma sousedními body. Hrbolatou texturu lze získat například tím způsobem, že při výpočtu

osvětlení objektu se změní směr normály k povrchu tak, jako by se jednalo o hrbolatý povrch. Textura tedy v tomto případě neurčuje barvu, ani průhlednost, ale členitost povrchu. Hrbolaté textury se prozradí na okrajích objektů, na kterých jsou naneseny.

Jak je patrné z obrázku 65 vlevo, koule, která vypadá jako hodně členitá, má zcela zřetelně kruhový obrys. To je patrné i na stínu, který je od koule vržen na rovinu, ležící pod ní. Přestože rovina vypadá jako zvlněná, stín je na ní zcela hladký. To je způsobeno tím, že rovina není skutečně zvlněná, ale je zvlněná pouze zdánlivě, vlivem modulace normály. Tyto textury jsou velice populární a pro svou jednoduchost jsou velmi často používány. Jejich nevýhodou je, že při velmi velkém zvětšení působí nepřirozeně. Lze se s tím setkat například u akčních počítačových her, při pohledu na nějakou zeď hodně zblízka.

2.8 Fraktály

Běžná tělesa a útvary v okolí člověka se dají popsat nebo zobrazit jako jistý konečný počet parametrů, které tato tělesa charakterizují. Pro základní geometrické tvary, například pro krychli, kouli, válec, prstenec, úsečku, přímku či rovinu, jsou známy vzorce a vztahy, ze kterých lze vypočítat jejich geometrické charakteristiky, například délku, plochu či objem. Jako samozřejmost se přitom bere to, že výsledek je vždy stejný, i když se počítá v libovolných jednotkách. Tyto veličiny (objem, povrch atd.) lze počítat i pro poněkud složitější útvary, které vzniknou kombinací konečného počtu elementárních útvarů. Lze třeba spočítat délku *Bézierovy křivky* nebo objem láhve vzniklé rotací této křivky okolo osy rotace. Takové objekty se dají snadno vytvořit v některém modelovacím programu.

Jiný způsob získávaní modelů představuje **simulace**. Tento pojem představuje napodobování určité vlastnosti nebo skupiny vlastností. Toto se v současné době děje zejména pomocí počítače a pro vytváření modelů reálného světa pomocí simulace se použil pojem **procedurální modelování.** Představuje tedy způsob získání modelů pomocí algoritmů. Charakteristické pro procedurální modelování je to, že pro malou změnu vstupních dat lze získat širokou škálu výstupních dat.

Fraktální geometrie je obor, který se zabývá simulací objektů tvarově podobných objektům reálného světa. Nezáleží přitom na změně měřítka – například větev stromu je podobná malé větévce, malý kámen je podobný hoře apod. Takové množiny lze poměrně snadno získat rekurzivními postupy.

Pro tato tělesa, která lze označit jako *normální* nebo *běžné,* platí, že všechny jejich parametry mohou být zadány v libovolné jednotce, aniž by se změnily vlastnosti tělesa. To znamená, že nezáleží na měřítku, ve kterém se lze na těleso dívat. Všechny tyto objekty mají jednu společnou vlastnost. Každému útvaru lze přiřadit jisté celé číslo, které se nazývá **počet rozměrů** nebo také **dimenze** daného útvaru. Takže úsečka, přímka či jiná křivka (například parabola, sinusovka či Bézierova křivka) má dimenzi rovnu 1. To znamená, že je jednorozměrná a tudíž poloha bodu je na ní definována pouze jedním číslem - souřadnicí. To, že má křivka dimenzi rovnu jedné, neznamená, že je

zobrazována v jednorozměrném prostoru. Dimenze udává jen počet parametrů, které jsou nutné pro definování bodu na křivce. Například křivka

$$\begin{aligned} x &= sin(t^2)cos(t) \\ y &= log(\sqrt{t}) \\ x &= \sqrt{sin(t)} \end{aligned} \quad . \tag{53}$$

má dimenzi rovnu jedné, ale je zobrazována v trojrozměrném prostoru. Jakákoliv hladká plocha (kruh, trojúhelník, n-úhelník) má dimenzi rovnu 2, to znamená, že poloha bodu musí být definována pomocí dvou souřadnic. Například sedlová plocha

$$\begin{aligned} x &= u \\ y &= v \\ z &= uv \end{aligned} \tag{54}$$

má dvě proměnné *u* a *v*, které definují jakýkoliv bod na ploše, zatímco *x*, *y* a *z* jsou výsledné souřadnice bodu v prostoru pro dané *u* a *v*.

Tímto způsobem definované plochy mají určitý obsah, ale jejich objem je nulový, protože mají nulovou tloušťku. Krychle, koule, válec nebo celý běžný prostor kolem nás mají dimenzi 3, protože poloha bodu je v nich jednoznačně určena třemi souřadnicemi. Podobně lze samozřejmě pokračovat dále, ale s dalšími dimenzemi nejsou zkušenosti – prakticky možná ani neexistují.

2.8.1 Nekonečně členité útvary

Tak jak bylo uvedeno v předchozí kapitole, lze se obvykle u běžných těles setkat s dimenzemi 1, 2 nebo 3. Byly však objeveny i zvláštní geometrické útvary, pro které toto rozdělení na celočíselné dimenze není dostatečné. Některé tyto útvary nejsou jen abstraktní objekty vzniklé ve fantazii matematiků, ale mají své přirozené vzory přímo v přírodě.

Klasickým příkladem takového útvaru je břeh řeky či potoka nebo pobřeží nějakého reálného ostrova. Je-li linie pobřeží zobrazena na nějaké mapě (nebo leteckém snímku), má tato mapa určité měřítko, například 1:1 000 000. Pomocí kružítka lze délku pobřeží přibližně zjistit a přepočítat na kilometry, protože je známo měřítko mapy. Jestliže bude kružítko roztaženo tak, aby vzdálenost mezi jeho hroty byla jeden centimetr a měřítko mapy je 1:1 000 000, pak to znamená, že při jednom kroku kružítka se na mapě posune o 1 000 000 centimetrů, což po přepočtu dává deset kilometrů.

Pokud však bude k dispozici přesnější mapa, která má měřítko například 1: 10 000 a bude-li se opět měřit délka stejného úseku pobřeží, dostane se délka odlišná - větší. To znamená, že se změnou měřítka se změnila délka toho samého objektu - v tomto případě pobřeží. Důvod se zdá být jasný - při zmenšení měřítka se vidí i detaily pobřeží, které nebyly na mapě s větším měřítkem viditelné. Samozřejmě, že při cestě pěšky okolo ostrova bude délka ještě větší, ovšem za předpokladu, že cesta

povede přesně na hranici pobřeží a nebude se nikde zkracovat. Se zmenšujícím se měřítkem by délka dále rostla a při délce měření blížící se limitně k nule by délka rostla dokonce až do nekonečna. Z toho vyplývá zajímavý fakt, že ostrov o konečné ploše má nekonečnou délku pobřeží.

Výše uvedený příklad není pouhou myšlenkovou abstrakcí, ale vyskytl se reálně při měření obvodu ostrova Korsika. Čím byla větší přesnost tohoto měření, tím větší byla naměřená délka. Tato skutečnost nebyla dlouhou dobu známa, a proto se délky státních hranic lišily až o násobky očekávaných délek.

Měření obvodu ostrova Korsika prováděl Richardson, který také jako první zjistil, že obvod ostrova je závislý na délce tyče, kterou se měří. Richardson také empiricky odvodil vztah

$$K = N(\varepsilon)\varepsilon^{D}, \tag{55}$$

kde K je délka celkového počtu $N(\varepsilon)$ úseček nutných k pokrytí dané křivky (např. pro pokrytí úsečky délky 1 je potřeba $N(\varepsilon) = 3$ úsečky délky jedna třetina, čtyři úseky délky jedna čtvrtina apod.). Délka pobřeží se ukázala být závislá na konstantě D, jejíž význam byl vysvětlen až později a nazván jako **Hausdorffova dimenze** (viz dále).

Také délka řek se musí měřit odlišným způsobem, než jaký byl běžně používán. Lze změřit délku řeky tak, že se postupně změří délka pravého a levého břehu a výsledná vzdálenost se zprůměruje. I v tomto případě by se při velmi přesném měření délka řek pohybovala v astronomických jednotkách a limitně by se blížila nekonečnu. Délka řeky ale nemůže být nekonečná, protože jestliže se na řece pluje konstantní rychlostí, dopluje se vždy na konec řeky v konečném čase. Je tedy nutné měřit například po pevně daných krocích nebo přímo v toku řeky.

Dala by se najít spousta dalších případů objektů, které mají konečný objem, ale jejichž plocha může být nekonečná, jako například povrch planety. Jiným příkladem je plocha plic nebo lidského mozku, které mají relativně malý objem. Ve všech těchto případech jde o praktickou aplikaci fraktálů v přírodě.

2.8.2 Hausdorffova dimenze a soběpodobnost

Měřením délky geometricky hladké křivky, která má dimenzi 1 se dostane při měření v různých měřítkách vždy stejné konečné číslo. Měřením délky ostrova se při zmenšování měřítka toto číslo stává nekonečně velkým. Pobřeží tedy v rovině zabírá více místa než hladká křivka. Nezabírá však všechno místo (nevyplňuje celou rovinu). Jeho skutečná dimenze je tedy větší než dimenze křivky (ta je rovna jedné) a současně je menší než dimenze roviny (ta je rovna dvěma). Z toho jasně vyplývá, že dimenze takového útvaru není celočíselná. Toto neceločíselné číslo se nazývá **Hausdorffovou (fraktální) dimenzí**.

Hodnota Hausdorffovy dimenze udává, jakou rychlostí délka těchto útvarů (či odpovídající veličina při větším počtu rozměrů) roste do nekonečna. Jestliže se bude Hausdorffova dimenze a

topologická dimenze lišit velmi málo, bude takový objekt málo členitý. Bude-li Hausdorffova dimenze ostře větší než dimenze topologická, bude objekt velmi členitý.

Fraktální dimenze D nějakého objektu se určí ze vztahu (55) dosazením za $K = 1$ a limitním zmenšováním $\varepsilon \to \infty$. Obecně platí podmínka

$$Ns^D = 1, \tag{56}$$

odkud lze odvodit vztah

$$D = \frac{\log N}{\log \frac{1}{s}}. \tag{57}$$

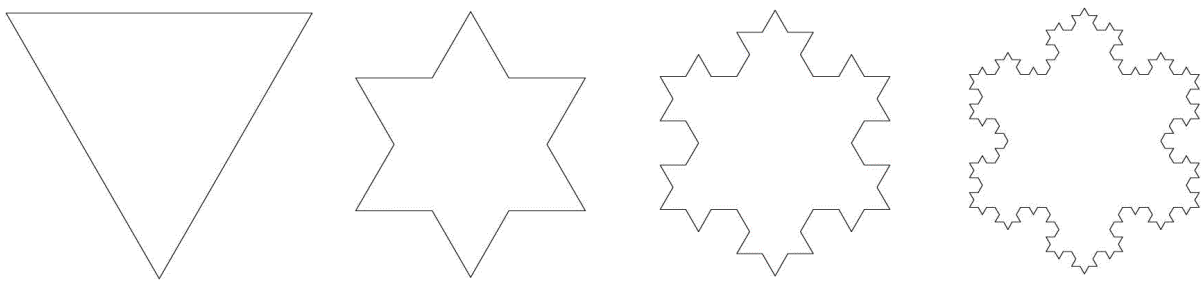
V (56) a (57) proměnná s značí měřítko. Pro některé jednoduché objekty je možné na základě vyjádření s vypočítat D

$$\text{úsečka: } s = \frac{1}{N} \Rightarrow D = 1, \tag{58}$$

$$\text{čtverec: } s = \frac{1}{\sqrt{N}} \Rightarrow D = 2, \tag{59}$$

$$\text{krychle: } s = \frac{1}{\sqrt[3]{N}} \Rightarrow D = 3. \tag{60}$$

Je mnoho uměle vytvořených geometrických útvarů (ale i přírodních objektů), které na první pohled nemají podobný tvar, ale ve skutečnosti si jsou vzájemně podobné v různých velikostech. Říká se o nich, že jsou **soběpodobné**. Pokud se některá část daného objektu zvětší, bude se podobat původnímu útvaru, a to tvarem a rozměry. Jedním z takových soběpodobných útvarů je sněhová vločka Kochové (v roce 1904 ji vytvořila švédská matematička Kochová jako příklad spojité, prostorově ohraničené soběpodobné křivky, která je nekonečně dlouhá).



Obr. 68. Čtyři iterace vločky Kochové

Její konstrukce je podle obr. 68 následující: na začátku je rovnostranný trojúhelník s délkou strany 1. V prvním kroku se na každou stranu umístí jednotková úsečka, která se následně rozdělí na

třetiny a střední část se nahradí dvěma jednotkovými úseky. Tím se vytvoří „zub“. V dalším kroku se provede totéž pro každou ze 4 částí křivky a dá se tak pokračovat stále dál. Stálým zjemňováním je po nekonečně mnoha iteracích délka okraje vločky nekonečná. Hausdorffova dimenze se vypočítá podle dosazením do vztahu (57) hodnoty $N = 4$ a $s = \frac{1}{3}$

$$D = \frac{log\,4}{log\,3} = 1{,}262\,. \tag{61}$$

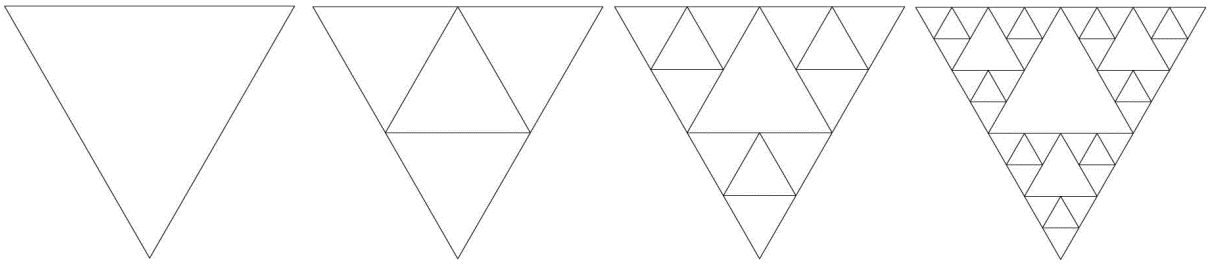
2.8.3 Typy fraktálů

Stejně jako v každé vědní disciplíně i ve fraktální geometrii se po určitém čase začaly rozlišovat jednotlivé typy fraktálů. Toto rozlišení není zavedeno jen kvůli systematičnosti, ale i proto, že jednotlivé typy fraktálů jsou vhodné pro řešení určitého okruhu problémů, které se vyskytují v různých oborech. Také způsob vytváření (generování) se liší podle typu fraktálu. Nejhrubší rozlišení fraktálů je následující:

- L-systémy,
- systémy iterovaných funkcí – IFS,
- dynamické systémy,
- nepravidelné fraktály (statisticky soběpodobné) .

2.8.3.1 L-systémy

L-systémy (Lindenmayerovy systémy – obr. 69) jsou skupinou fraktálů definovaných pomocí přepisovacích gramatik. Název pochází z anglického sousloví LOGO (*like turtle*). LOGO byl programovací jazyk, ve kterém se pomocí jednoduchých příkazů daly pomocí želvy kreslit různé obrazce. Podstatou tvorby L-systémů je přepisování řetězců podle určitých pravidel. Každý symbol v řetězci má přiřazen jistý geometrický význam, například transformaci či generování objektu.



Obr. 69. Ukázka L-systému - čtyři iterace Sierpinského koberečku

Zajímavé obrazce se začnou tvořit, jestliže se v LOGu použije iterace, což odpovídá iteraci v gramatice (na pravé straně pravidla je nonterminální symbol shodný se symbolem na levé straně pravidla). S pomocí L-systémů lze generovat fraktály, které se podobají rostlinám, stromům a dalším přírodním útvarům. Poslední aplikace také směřují k využití těchto fraktálů při generování textur.

Z toho, jak se fraktál generuje, je zřejmé, že jde o deterministický postup a výsledný fraktál je tedy též deterministický. Těmto fraktálům se také někdy říká graftály.

2.8.3.2 Systémy iterovaných funkcí – IFS

Systém iterovaných funkcí IFS je generativní metoda tvoření fraktálů. Tato metoda je vhodná jak pro generování fraktálů, tak i pro kompresi bitmapových obrazů, zavede-li se tzv. *inverzní úloha*. Zajímavé je, že tato metoda může být jak deterministická, tak i náhodná. Oba způsoby však paradoxně vedou ke stejnému výslednému fraktálu, použije-li se dostatečný počet iterací.

Obr. 70. Příklad IFS fraktálu – list kapradiny

2.8.3.3 Dynamické systémy

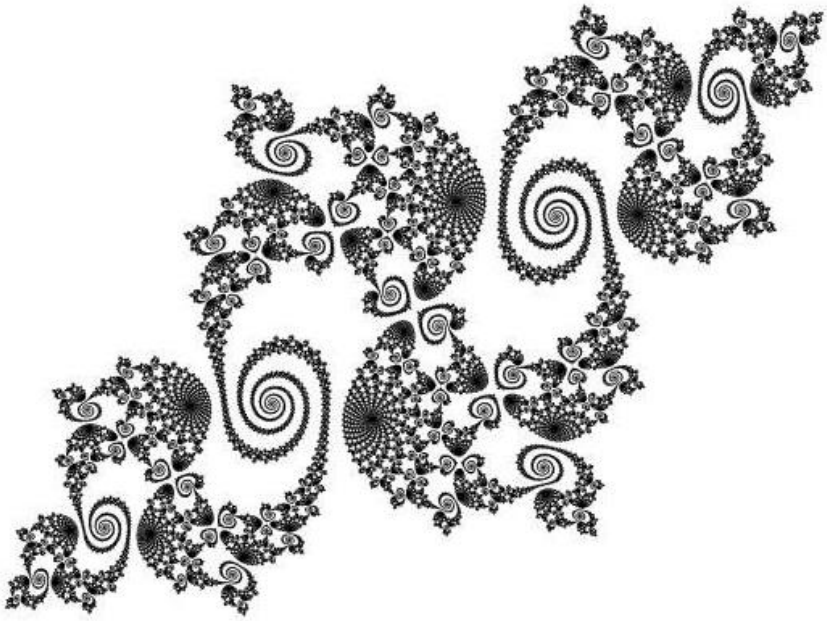
Dynamické systémy jsou pravděpodobně tím typem fraktálů, který má v technické praxi nejširší uplatnění. Dynamický systém je matematický model, který je závislý na nějaké proměnné, většinou to bývá na čase. Dynamický systém vychází z počátečních podmínek a je jimi v čase determinován.

Existují dynamické systémy, které se po určitém čase neustálí v pevném stavu, ale ani nedivergují. Tento případ, který připomíná iracionální čísla, má většinou fraktální dynamiku a označuje se termínem *deterministický chaos*.

Typickým příkladem dynamického systému s fraktální dynamikou je výpočet populačního růstu, který má tu zajímavou vlastnost, že volbou jediného parametru lze určit, zda systém bude ustálený, oscilující nebo chaotický. Při zkoumání dynamiky populačního růstu byly také objeveny některé konstanty, které mají obecnou platnost, podobně jako například konstanta π nebo základ přirozených logaritmů *e*.

Dynamické systémy existují i v komplexní rovině. Tady jsou asi nejvíce známy Juliovy množiny (obr. 71) a Mandelbrotova množina.

Znalost toho, zda nějaký systém je ustálený či zda směřuje k chaosu, je velmi důležitá pro další výpočty tohoto systému. Pro chaotický systém (nebo fraktálně dynamický systém) nelze předpovědět stav systému v budoucnosti, aniž by se musel simulovat celý vývoj systému. Taktéž platí, že dynamické systémy jsou obecně velmi závislé na počátečních podmínkách, takže i nepatrná změna počátečních podmínek má za následek odlišné chování systému v budoucnosti.



Obr. 71. Příklad dynamického systému - Juliova množina

2.8.3.4 Nepravidelné fraktály

Zatímco všechny předchozí skupiny fraktálů byly v určitém smyslu symetrické, nepravidelné fraktály vnáší při generování fraktálu do algoritmu náhodu. Tento typ fraktálů také umožňuje nejlepší popis přírodních objektů. Zatímco při generování stromu klasickým L-systémem nebo IFS koláží lze získat jako výsledek perfektně symetrický strom, je skutečnost už na první pohled zcela jiná. Strom rostoucí v přírodě má nepravidelné délky a tloušťky větví, strom rostoucí v lese má jiný tvar než tentýž druh stromu rostoucí osamoceně apod. Proto je vhodné zavést do generování fraktálů náhodu.

Způsob, jakým se náhodnost bude podílet při generování fraktálů, bude vždy určovat tvar fraktálu i jeho Hausdorfovu dimenzi. Pro generování náhodných čísel se používá například bílý šum.

Náhodné fraktály lze vytvářet více způsoby:

- pomocí simulace Brownova pohybu (jak v ploše, tak i v prostoru),
- metodou přesouvání středního bodu,
- spektrální syntézou.

Simulace Brownova pohybu vytváří fraktální objekt, jehož Hausdorffova dimenze je úměrná absolutní velikosti změny při jednom kroku iterace. Tato metoda se používá například při generování toků řek. Není příliš vhodná pro trojrozměrné objekty.

Mezi typické metody generování tohoto typu fraktálů je metoda přesouvání prostředního bodu, a to jak na ploše, tak i v prostoru. Tato metoda se velmi často používá v počítačové grafice k vygenerování a následné vizualizaci přírodní krajiny. Volbou maximální odchylky při posunu prostředního bodu lze potom měnit celkový ráz krajiny od pouště až po vysokohorskou krajinu.

Další často používanou metodou je spektrální syntéza, která vychází z Fourierovy řady. Metoda spočívá v tom, že se náhodně vygenerují Fourierovy obrazy, které mají spektrální hustotu úměrnou zadané Hausdorffově dimenzi. Potom se provede inverzní Fourierova transformace s těmito koeficienty. Výsledkem je fraktální objekt. Pomocí této metody lze generovat hory či povrchy planet. Výhodou je, že lze přímo zadat požadovanou Hausdorffovu dimenzi výsledného objektu.

2.9 Animace

Někde mezi nepohyblivým světem stálých obrázků a světem obrázků, které se pohybují v reálném čase, leží svět počítačové animace. Tradiční kreslené filmy představují trochu víc než sérii uměleckých děl v buňkách, z nichž každá obsahuje nepatrně změněné polohy animovaného předmětu. Pokud je velké množství těchto buněk zobrazeno velkou rychlostí postupně za sebou, jeví se animovaný obrázek lidskému oku jako pohyblivý. Počítačem animovaná sekvence se chová naprosto stejným způsobem. Série obrázků je tvořena jedním předmětem, přičemž každý obrázek představuje pohled na animovaný předmět z mírně odlišné perspektivy. Jsou-li tyto obrázky puštěny na obrazovce ve správném pořadí a správnou rychlostí (tzv. snímkovací kmitočet), pak se zdá, že se předmět pohybuje. Počítačem provedená animace je ve skutečnosti kombinací obou druhů obrázků - statického i pohyblivého. Každý snímek animované sekvence je nehybný obrázek, který vyžaduje kompresi a uložení. Soubor s animovaným dílem však musí uchovávat data stovek nebo tisíců animovaných snímků a navíc musí poskytovat informace nezbytné pro přehrávání snímků v příslušném zobrazovacím režimu a se správným snímkovacím kmitočtem. Většina souborů pro animaci používá kompresního schématu delta, který je formou proudového kódování, které ukládá a komprimuje pouze informace o rozdílu mezi dvěma obrázky (což je vhodnější než komprese celého snímku). Mezi nejpoužívanější animační formáty patří FLI/FLC, MPEG a AVI. Popis prvních dvou formátů je uveden v kapitole 1.4.

Pokud se hovoří o počítačové animaci, jedná se většinou o pohyb. Pohyb může mít mnoho zcela odlišných forem - od změny polohy kamery v jinak statické scéně, přes prosté posouvání objektu po stole a plápolání vlajky ve větru až po tekoucí vodu či oheň. V počítačové animaci není k dispozici jednotný popis různých forem pohybu, proto existuje i velké množství algoritmů, pomocí kterých každý po svém řeší dílčí úlohy počítačové animace. Některé úkoly lze řešit snadno a je možné je i jednoduše ovládat - například pohyb bodu po křivce, orientaci kamery aj. Jiné jsou však natolik komplikované, že se pro ně musejí hledat složité algoritmy, a jejich výsledek je často i těžko odhadnutelný.

Protože se počítačová grafika zabývá reálným světem, je možné aplikovat spoustu postupů z jiných vědních oborů, čímž se počítačová animace přímo setkává se simulací chování různých přírodních jevů. Z tohoto důvodu se počítačová grafika musí učit např. z fyziky, kde existují modely takového chování. Nejde však ani tolik o to, aby výsledky byly fyzikálně přesné, ale zejména o to, aby byly vizuálně přesvědčivé. Počítačová grafika tedy hledá taková zjednodušení těchto postupů, aby byl jejich výpočet maximálně efektivní z hlediska časové výpočetní náročnosti a zároveň realistický. Podle způsobů, jak jednotlivé algoritmy řeší úlohy pohybu, lze rozdělit počítačovou animaci na **nízkoúrovňovou** a **vysokoúrovňovou**. Animace na vyšší úrovni je založena na animaci nízkoúrovňové tak, že dílčí úlohy jsou chápány jako jakési bloky, které se nemusejí řešit a z nichž se skládají pohyby komplikovanější. Jinými slovy, na vyšší úrovni se zabývá například tím, jak má nějaká ruka uchopit objekt, například virtuální postava virtuální sklenici. Řešení kolizí, výpočet dynamiky pohybu atd. řeší algoritmy nižší úrovně. Nejde tedy o dva oddělené světy, ale vyšší úroveň je přímo založena na úrovni nižší. Mezi vyšší animační techniky patří zejména inverzní a přímá kinematika.

Animace neslouží pouze pro zábavu dětí a dospělých. Animované sekvence používají programátoři systémů CAD při rotaci 3D objektů, aby je mohli pozorovat z různých úhlů. Dále se používají při sledování chování složitých objektů v pohybu metodou sledování paprsku (viz kap. 2.6.1). A rovněž matematické údaje, získané sledováním z letadla nebo satelitu, mohou být předloženy ve formě animované sekvence zachycující pohled za letu. V případě zvláštních filmových efektů mají také hlavní slovo počítače.

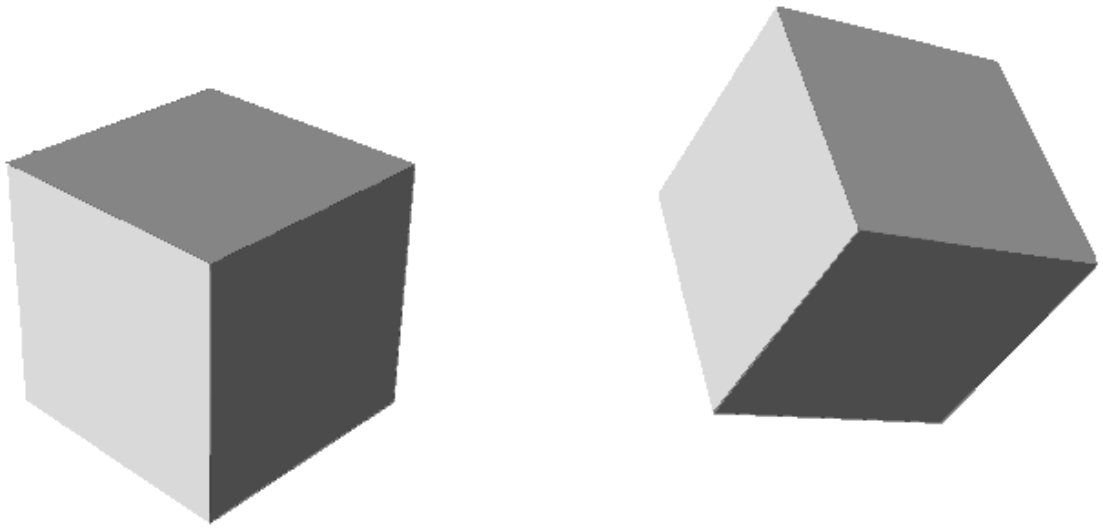
2.9.1 Nízkoúrovňová animace

2.9.1.1 Klíčování

Jedním z ústředních pojmů počítačové animace je **klíčování** (*keyframing*). Podstata je taková, že se definují tzv. klíčové snímky, ve kterých dochází k nejdůležitějším změnám animace. Zbývající snímky ležící mezi nimi se nazývají mezisnímky (*inbetweens*). Původně veškeré snímky ve

dvojrozměrné animaci zpracovávali lidé. Vzhledem k tomu, že práce s mezisnímky byla hodně rutinní a zdlouhavá, vznikly programy, které mezisnímky vytváří automaticky na základě snímků klíčových.

Postup, kdy animátor zadává určité klíčové body a program dodělává zbývající práci, se postupem času začal používat i v trojrozměrné počítačové animaci. Rozšíření proti dvojrozměrnému případu je však v tom, že animátor zadává klíčové pozice v podstatě čehokoliv - polohy objektů, velikosti úhlů, barvy, textury, průhlednosti aj. Úkolem programu je pak automatické generování příslušných mezipoloh. Výsledkem tohoto postupu je popis scény, který pak slouží jako vstup nějakého programu, který na základě údajů o poloze a vlastnostech kamery celou scénu zobrazí. Tato metoda není ničím jiným než zadáváním klíčových parametrů (*key parameters*), proto se všechny tyto případy souhrnně nazývají klíčování. Obrázek 72 ukazuje případ, kdy byly zadány dvě klíčové polohy krychle (druhá poloha vznikla jejím posunutím a pootočením). Program může automaticky vygenerovat jednu z možných trajektorií pro posun. Tento problém evidentně není jednoznačný, neboť libovolnými dvěma body může vést nekonečně mnoho křivek. Platí tedy pravidlo, že čím více klíčových poloh se zadá, tím více se výsledek přibližuje tomu, co chtěl animátor ve skutečnosti vytvořit. Samozřejmě platí, že v klíčových polohách je možné ručně ladit spoustu různých parametrů.



Obr. 72. Dvě klíčové polohy krychle

2.9.1.2 Dráha a orientace

Druhým způsobem, jímž lze určit jednoduchý pohyb, jakým je například trajektorie nějakého objektu, je přímá definice příslušné křivky. Křivka se namaluje v grafickém editoru a nadefinuje se, že konkrétní objekt má sledovat její dráhu. To samozřejmě není jednoznačné, a tak se na objektu vždy musí definovat speciální bod, kterému se obvykle říká **pivot**. Tento bod je počátkem lokálního souřadnicového systému animovaného objektu a z jeho polohy se určuje například i vzdálenost mezi ním a ostatními objekty ve scéně. Pivot odpovídá bodu, který přesně sleduje zadanou trajektorii. Zatímco poloha bodu je zadána zcela jednoznačně, v případě objektu zbývá ještě spousta parametrů,

které se musejí určit, aby byl pohyb jako celek jednoznačný. Objekt se totiž může kolem této trajektorie libovolně otáčet, a tak je nutné určit jeho orientaci. Jak již bylo řečeno, s pivotem je určen lokální souřadnicový systém, tedy osy *x, y* a *z*. Pokud je potřeba dráhu pohybujícího se objektu určit jednoznačně, musí se udělat totéž na křivce a oba tyto systémy k sobě nějak vztáhnout.

2.9.1.3 Animační křivky

Protože možných interpolací je nekonečně mnoho, je nutné mít k dispozici prostředky, jak počítačem určenou křivku dolaďovat. Pokud se animuje například pohyb nějakého objektu, např. cyklisty na kole v zatáčce, musí být k dispozici nástroje, s jejichž pomocí je možné doladit úhly natočení, zachytit fakt, že cyklista po projetí zatáčkou zrychlí atd. Prostředky, které se k tomuto používají, se nazývají animační křivky a každý profesionální animátor ví, že nejvíce práce je právě s nimi. Tímto způsobem se neurčuje pouze pohyb, ale změna libovolného parametru, například barvy či prolínání textury na povrchu objektu. Animační program by měl poskytovat širokou škálu možností pro editaci animačních křivek. Nejčastějším požadavkem je možnost zrychlení a zpomalení efektu. Program by měl rovněž umět navázat pohyb tak, aby v něm nevznikaly skoky, i když tento „rychlostní skok“ je v poslední době (např. v reklamách) velice často používaný. Většina programů, které se pro trojrozměrnou animaci používají, poskytuje pro tento případ buď Bézierovy křivky nebo častěji různé druhy spline křivek (viz kap. 1.1.3.2).

2.9.2 Vysokoúrovňová animace

Animace na úrovni animačních křivek je nesmírně pracná a poměrně složitá. Vyžaduje určitou dávku zkušeností a lze ji do určité míry považovat za jakousi počítačovou alchymii. Programátoři, kteří tvoří animační programy, si jsou tohoto faktu vědomi, a proto je z jejich produktů zřetelná snaha o přenesení těžiště práce na vyšší úroveň. V programech se objevují nástroje, které automatizují rutinní práci, a tak dovolují daleko větší míru kreativity animátora.

Jedním ze základních prostředků, které se označují jako prostředky vyšší úrovně počítačové animace, je možnost seskupování (hierarchizace) objektů. Klasickým příkladem je postavička člověka, která se skládá z rukou a nohou, ty se skládají z prstů, ty z článků atd. Jednotlivé části jsou na sebe navázány a pohyb jedné části může ovlivňovat pohyb částí zbývajících. Ve trojrozměrném modelovacím programu se tedy nejprve vytvoří jednotlivé části a tyto se postupně na sebe navážou. Podstatnou roli při navazování samozřejmě hraje již výše zmíněný lokální souřadnicový systém se svým počátkem - pivotem. Při pokusu uchopení za palec na ruce a pokusu postavičce zatřást rukou, musí program nějakým způsobem reagovat.

Výše zmíněná úloha je klasickým problémem, kterým se zabývá robotika, a proto nebylo nijak složité nalézt řešení tam. Základním krokem, který se vždy provádí, je zjednodušení úlohy. Z celého

objektu se ponechá pouze jakási kostra (*skeleton*) a pracuje se pouze s ní. Obalit kostru plochami a plochám přiřadit textury jsou jiné úlohy, které animátora v případě řešení pohybu příliš nezajímají. Kostra se skládá ze základních článků, kterým se říká segmenty, a proto se celé kostře také říká **segmentová struktura**.V počítačové grafice se takovéto objekty chápou jako složené z posloupnosti pevných částí, které jsou mezi sebou spojeny, a v každém spojení je možné s oběma segmenty libovolně otáčet. Posuvné (prizmatické) spojení nebývá v počítačové grafice obyčejně uvažováno, množství případů, které se popisuje pouze pomocí otáčení, je údajně dostačující. Segmentová struktura bývá na jednom konci pevně zakotvena. Je-li její druhý konec volný, říká se takové struktuře otevřená segmentová struktura, a bodu, který je na jejím volném konci, koncový efektor. Příkladem segmentové struktury je již zmíněná lidská paže natahující se po sklenici. Na jednom konci je paže pevně svázána s tělem, koncovými efektory jsou prsty.

Pokud je potřeba určit polohu pevného tělesa v prostoru jednoznačně, je k tomu nutný zvolený souřadnicový systém a celkem šest čísel. Tři hodnoty určují souřadnice tělesa v prostoru a tři jeho natočení vzhledem k souřadnicovým osám. Tyto veličiny se jmenují stupně volnosti (*DOF - degrees of freedom*) a jednoznačně charakterizují polohu libovolného tělesa. Přidá-li se k takovému objektu nové těleso, zvýší se počet stupňů volnosti. Pokud se budou tělesa svazovat pevnými linkami (bude se tedy vytvářet segmentová struktura), bude počet stupňů volnosti nižší než při prostém seskupování.

Ale zpět ke zmíněné úloze. Hlavním úkolem je vyřešit problém, jak v segmentové struktuře, která reprezentuje model lidské paže, určit natočení všech úhlů tak, aby paže uchopila virtuální sklenici. K automatickému řešení tohoto problému se v počítačové animaci používají dva základní postupy - přímá a inverzní kinematika.

2.9.2.1 Přímá a inverzní kinematika

Tato úloha není jednoduchá hned z několika důvodů. Předně - řešení nemusí vůbec existovat, například paže na sklenici vůbec nedosáhne a systém by v těchto případech měl poskytovat nějakou rozumnou odezvu, například varovné hlášení, změnu barvy animovaného objektu aj. Dalším, ale daleko závažnějším problémem je, že řešení může existovat více, případně jich může existovat nekonečně mnoho. Systém musí umět nějakým rozumným způsobem reagovat. Složitější je, když se například virtuální paže uchopí za prst a posouvá se s ním, může se program někdy dostat do takové situace, že zvolené řešení pro jeden bod a zvolené řešení pro druhý bod jsou velice vzdálená, a v animaci se pak objevují neočekávané kroky. Jak tedy algoritmy pro kinematiku fungují?

Výpočet polohy koncového efektoru pomocí **přímé kinematiky** spočívá v postupném určení natočení všech segmentů struktury od prvního (pevně zakotveného) až po poslední. V případě virtuální figurky se nejprve stanoví úhel natočení v rameni, poté v lokti a nakonec v zápěstí. Pokud bude figurka působit nepřirozeně a bude se muset například změnit úhel natočení ramene, změní se tím i tvar celé ruky, a tak se musí postupovat znovu od posledního změněného místa. To je největší

nevýhodou přímé kinematiky. Algoritmy využívající přímé kinematiky jsou jednoduché z hlediska implementace. Využívání tohoto způsobu řízení modelu je však neintuitivní a pro animátora je zadávání jednotlivých parametrů pro každý segment často zdlouhavé. Z těchto důvodů je přímá kinematika používána spíše v případech, kdy je animace řízena nějakým vnějším popisem. Pro interaktivní animování pohybu je výhodnější tzv. inverzní kinematika.

Při řízení modelu pomocí **inverzní kinematiky** jde o opačný postup než v předcházejícím případě. Cílem je nalézt úhly natočení jednotlivých segmentů na základě informace o poloze koncového efektoru. Často uváděným příkladem takového postupu je stanovení úhlů natočení kloubů při provádění dřepu. Koncový efektor (poloha chodidla) stejně jako poloha osoby jsou známy a algoritmus inverzní kinematiky určí úhly ohnutí kloubů. Inverzní kinematice se také říká cílem řízený pohyb (*goal directed motion*). Algoritmů inverzní kinematiky existuje několik a jednotlivé programové balíky obyčejně nabízejí více možností než jeden jediný. Problematické případy, kdy dojde k oscilacím v řešení či k nějakým jiným patogenním stavům, jsou obtížně předvídatelné a ještě hůře se jim zabraňuje. Jedním z řešení tohoto problému je omezení možností pohybu celé struktury. Toho lze docílit přirozenou cestou například tak, že se do systému zavedou určitá omezení (*constraints*), třeba zákaz otáčení kloubu za určitou mez, zákaz překročení určité vzdálenosti pro určité segmenty, je možné systému zadat, aby některá napojení řešil pouze v rovině, která je přesně zadána apod.

2.9.2.2 Snímače pohybu

Přes veškerou snahu animátorů i programátorů, kteří se animaci pohybu věnují, nevypadá umělý pohyb na obrazovce počítače příliš dobře. Vše vyhlíží příliš matematicky stroze a nepomáhá ani dodání náhodných čísel. Existují i velmi dobré výsledky, ale obyčejně je jich docíleno za cenu nesmírné námahy v ruční fázi animace. Jedním z řešení metodou hrubé síly (*brute fouse solution*) je použití tzv. snímačů pohybu (*motion capture, motion sampler*).

Snímače pohybu jsou čidla detekující svou polohu a orientaci v prostoru a přenášející tento údaj do počítače. Figurant si na sebe upevní několik čidel a na daný povel provede určité pohyby, které jsou snímány do počítače a přenášeny na syntetickou postavu. Každé čidlo odpovídá přesně určitému bodu na postavičce a pokud někde zbývají nějaké volné segmentové struktury, o zbytek se postará algoritmus inverzní kinematiky. Výsledkem je dokonale věrohodný pohyb syntetické postavičky ve virtuálním prostředí v počítači. Snímání pohybu je dnes poměrně běžné například ve filmu nebo při vytváření akčních her.

Nejčastěji se používají dva principy snímání pohybu - magnetický a optický. Není snadné rozhodnout, která z těchto zařízení jsou výhodnější, s jistotou lze však říci, že magnetické snímače jsou daleko levnější. Na snímání pohybu člověka se používá přibližně třináct až osmnáct snímačů, které jsou umístěny po dvou na kotnících, na kolenou, na ramenou, na rukou, na dlaních a po jednom

na hlavě, hrudi a klíně. Vzorkovací frekvence, tj. okamžiky, ve kterých jsou polohy zaznamenávány, bývají až 144 Hz. Poměrně přesné vzorkování ve svém důsledku produkuje rozsáhlé soubory dat (122 kB za vteřinu, 7 MB za minutu). Tato data jsou navíc zatížena šumem a nepřesnostmi, takže se užívají různé filtrovací metody a komprese. Se snímači pohybu není problémem vytvořit si knihovnu různých gest, pohyby mezi sebou různě míchat atp.

3. GRAFICKÝ HARDWARE

Každý počítač je složen z jednotlivých komponent, z nichž každá vykonává určitou funkci. To se týká i grafických periférií, z nichž některé jsou pro práci s grafikou nutností, jiné práci uživatele zjednodušují a ulehčují. V této kapitole je pojednáno o nejčastěji používaném grafickém hardware s popisem základních vlastností. Více informací lze nalézt např. v [8] nebo [9].

Periferie lze z hlediska komunikace s uživatelem rozdělit do dvou velkých skupin.

- Vstupní – nejčastěji zpracovávají vstupní grafickou informaci *x*-ové a *y*-ové souřadnice zadaného bodu nebo jejich přírůstek. Tyto informace se transformují do číselné formy, kterou počítač dále zpracovává.
- Výstupní – převádí naopak počítačem specifikovaný grafický element (např. pixel) na výstupní zařízení. Nemusí jít jen o polohu, ale i např. o barvu nebo odstín.

Další dělení periferií je podle způsobu vytváření grafické informace.

- Vektorové – na uchování grafické informace potřebují kapacitně menší paměť než rastrové periferie. Tato zařízení zobrazují grafické prvky jako jednoduché objekty (např. úsečka). V podstatě se kreslí vektor od začátku do konce prvku. Na stejném principu fungují i vektorové grafické formáty souborů.
- Rastrové – jsou paměťově náročnější. Jejich podstatou je plocha daná maticí bodů (pixelů). Rastrové zařízení zvýrazní příslušný pixel (nastaví jas a barvu). Výsledný obraz se skládá postupným skládáním těchto pixelů. Na uchování zobrazované informace na monitoru se používá videopaměť, jejíž velikost je dána rozlišením obrazovky a množstvím barev.

3.1 Vstupní grafická zařízení

3.1.1 Myš

Myš patří mezi základní periferní zařízení. Její využití v jednotlivých aplikacích spočívá v maximální míře omezit používání „nepohodlné“ klávesnice. Základní funkce myši je založena na přenosu pohybu ruky po vodorovné podložce na obrazovku počítače.

Fyzicky představuje myš malou krabičku, vybavenou dvěma nebo více tlačítky (nebo válečkem), jejichž stiskem se vyvolává konkrétní funkce ve spuštěné aplikaci. Velké množství myší obsahuje ve spodní části kuličku, která se při jejím pohybu odvaluje. Pohyb kuličky se přenáší na dva otočné válečky (horizontální a vertikální směr) a pohyb válečků se opticky převádí na elektrické impulsy, které se posílají dále do počítače. Tento typ myší se nazývá mechanicko-optický.

Existují další variace, jako např. ultrazvuková nebo optická myš. Zejména optické myši se díky srovnatelným cenám s klasickými stále více uplatňují. Jejich výhoda spočívá zejména v odstranění potřeby „čistit kuličku a válečky“, které jsou při častém používání na nečistotu náchylné. Princip optické myši spočívá v nahrazení kuličky dvojicí vysílače a přijímače. Tyto prvky jsou založené nejčastěji na bázi LED diod (červená barva). Pohyb se pak snímá změnou odrazu vysílaného světla od podložky. V současné době existují i myši, které umožňují trojrozměrný vstup. Třetí souřadnice je dána vzdáleností myši od podložky (zdvih). Aplikace na jejich využití však nejsou příliš rozšířené.

Důležitým údajem u myší je taky rozlišovací schopnost udávaná v DPI (počet bodů na palec). Mechanicko-optické myši dosahují běžně přesnosti 200 – 2 400 DPI, mechanické a optické až do 6000 DPI.

Myš lze k počítači připojit přes sériový či paralelní port (dříve), dnes se lze častěji setkat s konektory PS/2 či USB. Vyrábějí se i myši bezdrátové, jejichž přenos signálů je pomocí infračervených paprsků nebo rádiových vln, vzhledem k vyšší ceně nejsou však tolik rozšířené.

3.1.2 Scanner

Jde o zařízení, které snímá tištěné obrazy do počítače. Jeho principem je převod odstínu snímané barvy z předlohy na číselnou hodnotu, kterou může počítač dále zpracovávat. Rozdělit je lze do dvou skupin na

- *ruční*, které jsou lacinější. To je dáno jednodušší konstrukcí, která způsobuje i menší přesnost a rozlišovací schopnost (zpravidla 150 - 300 DPI). Při snímání uživatel sám pohybuje scannerem,
- *automatické* jsou přesnější a mají vyšší rozlišovací schopnost (600 - 1200 DPI i více). Pohyb snímací hlavy je prováděn bez zásahu uživatele. Jsou dražší a patří mezi ně stolní, bubnové nebo filmové scannery.

Scannery snímají obraz rastrově a výsledný obraz je tak složen z jednotlivých bodů. Množství těchto bodů je dáno již zmíněnou rozlišovací schopností. S narůstajícím rozlišením roste cena a časová náročnost snímání (i případného zpracování), z tohoto důvodu některé scannery podporují tzv. softwarovou interpolaci, která dokáže rozlišení zvýšit. Podstatou je vkládání dalších bodů mezi dva sousední a jejich barva závisí na barvě bodů, mezi které se nový bod vkládá. Při narůstajícím počtu vkládaných bodů ovšem klesá výsledná kvalita obrazu.

Nasnímaná data lze pak dále upravovat v grafických editorech a ukládat v různých grafických formátech. Existuje i speciální programové vybavení nazývané OCR (*Optical Character Recognition*), které dokáže graficky snímat i text, který lze zanalyzovat a převést do znakové podoby. K dispozici jsou i programová vybavení, která dokáží rastrové obrazy převést do vektorové formy.

3.1.3 Optické pero

Používá se jako grafické vstupní zařízení ve spojení s grafickým displejem. Pero bývá spojeno s počítačem pomocí kabelu a může mít ještě potvrzovací tlačítko. Po přiložení pera k obrazovce se snímá poloha pera. Princip je následující: uvnitř pera je fotocitlivá součástka. Poloha se zjistí pomocí synchronizace aktivačního pulsu, ke kterému dochází při průchodu elektrického paprsku místem, kde se nachází pero.

3.1.4 Trackball

Z hlediska konstrukce si lze toto polohovací zařízení představit jako obrácenou mechanicko-optickou myš, jeho oblast užití je tedy prakticky totožná. Buď bývá v samostatné krabičce nebo se zabudovává do krytu klávesnice (např. u notebooku). Na horní straně krabičky je kulička, kterou se pohybuje prsty. Kulička se odvaluje a její otáčení se snímá a vysílá do počítače. Podobně jako u myší, mívá trackball dvě nebo tři tlačítka.

3.1.5 Tablet

Jde o zařízení, které je podobné myši a komunikace s aplikacemi, ve kterých se používá, je stejná jako u myší. Dosahuje vyšší polohovací přesnosti a rozlišení. To je dáno konstrukcí, která se zpravidla skládá z ploché snímací podložky (deska, pod kterou je umístěn elektromagnetický obvod), pohyblivého snímacího zařízení a řídicího elektronického obvodu.

Pohyblivé snímací zařízení mívá tvar myši či pera a snímá signály z elektromagnetických obvodů podložky. Řídicí elektronický obvod bývá často zabudován přímo v podložce. Proud procházející sítí podložky snímá impulsy z pohyblivé části, pomocí kterých se zjišťuje poloha.

3.1.6 Joystick

Je oblíbené vstupní zařízení především u domácích počítačů, o jehož rozšíření se zasloužily zejména počítačové hry. Nazývá se též křížový ovladač. Pro uživatele je nejdůležitější ovládací páka, jejíž vychylováním v příslušném směru se ovládá kurzor. Joystick bývá doplněn i několika tlačítky, které mohou mít speciální funkce závislé na aplikaci. Podle konstrukce lze joysticky rozdělit na spínací, kdy se při vychýlení spíná pouze kontakt, podle něhož se detekuje směr. Druhou skupinu tvoří joysticky potenciometrické. V tomto případě dochází při vychýlení ke změně odporu potenciometru v příslušném směru.Tato hodnota se převádí do číselné formy počítače. Potenciometrické joysticky jsou tak schopné snímat nejen směr vychýlení, ale i rychlost změny vychýlení a velikost výchylky.

3.2 *Výstupní grafická zařízení*

3.2.1 Tiskárna

Tiskárna je výstupní zařízení počítače, pomocí kterého lze data v elektronické podobě převést do tištěné podoby. Počítač tiskárně říká nejen „co se má tisknout“ (údajová část), ale i „jak se to má tisknout“ (řídicí znaky). Pomocí nich se definuje např. font a velikost písma, řádkování apod. Mezi základní rozhodující parametry při výběru tiskárny patří kvalita tisku (DPI), cena tiskárny a tisku, rychlost, obsah tisku (černobílý, barevný). Nejrozšířenější typy tiskáren jsou:

- *Jehličkové* (*maticové*) – donedávna patřily mezi nejrozšířenější. Hlavním důvodem byla cena jak tiskárny, tak i tisku. Tisková hlava je tvořena určitým počtem jehel (1,9,18,24, 48). Počet jehel určuje rychlost a kvalitu tištěného dokumentu. Jehly vytváří na papíře znaky přes pásku, která je podobná pásce na psacím stroji. Pro barevný tisk jsou určeny pásky se základními pruhy barev (tyrkysová, fialová, žlutá, černá). Nevýhodou těchto tiskáren je hlučnost.
- *Inkoustové* – jsou v současné době velice rozšířené. Jsou vhodné pro tisk běžných textových a grafických dokumentů. Cena samotných tiskáren nebývá velká, náklady se spíše promítají do tisku, protože je inkoust poměrně drahý. Tisk inkoustem využívá buď princip piezoelektrický (vytváří se tlaková vlna, která vystříkne inkoust) nebo termický (kapičku inkoustu ovládá bublinka páry).
- *Laserové* – jsou určeny zejména pro rychlejší a kvalitnější tisk. Nevýhodou je vyšší nákupní cena. Rozdíl vzhledem k předchozím typům tiskáren je takový, že laserové tiskárny tisknou ne po znacích, ale po celých stranách. Tištěný obraz se vytváří pomocí laserového paprsku na speciálním válci. Na místech, kde prochází paprsek zůstává elektrostatický náboj, na kterém se uchytí práškové barvivo (toner). Ten se z válce přenese na papír a tepelnou úpravou se na něm připevní.

3.2.2 Plotter

Je také nazývaný souřadnicový zapisovač. Grafický výstup se provádí na papír pomocí pera. Pro své vlastnosti se hojně používá v CAD systémech nebo pro zaznamenávání grafů závislostí při reálných měřeních. K počítači se často připojují přes sériový nebo paralelní port.

Rozdělení plotterů lze provádět z různých hledisek – podle řízení (analogové, číslicové), podle způsobu uložení a pohybu papíru (stolní, válcové), podle používaného pera (fixové, tryskové, tušové, s kuličkovým perem) a podle počtu barev (jednobarevné, vícebarevné).

Ovládání pera spočívá v jeho pohybu podle jednotlivých os a dále zdvih nebo spuštění na papír, u barevného plotteru i výměna per podle požadovaných barev.

V praxi také bývá často nahrazeno pero ostrým nožem, pomocí kterého se vyřezává požadovaný obraz. Příkladem může být vyřezávání reklamy na samolepící folie. Dále se lze setkat s případy, kdy se pero nahrazuje osvětlovací (laserovou) hlavou a místo papíru se užívá fotocitlivý film. Uplatnění tohoto zařízení lze nalézt při tvorbě předloh k výrobě desek s plošnými spoji.

3.2.3 Monitor

Monitor je jednoznačně základní výstupní zařízení, na kterém lze vidět textové i grafické výsledky. Jeho rozšíření je v současné době stále hojné přes stále se rozšiřující uplatňující LCD obrazovky. Vzhledem k dlouhodobému vývoji lze říci, že v současné době lze zakoupit kvalitní monitor za rozumnou cenu.

Většina monitorů je založena na principu katodové trubice (*CRT – Cathode Ray Tube*). Obrazovka je založená na emitující katodě a stínítka. Při vytváření obrazu emituje katoda svazek paprsků, které prochází zaostřovacím systémem a pomocí elektromagnetického pole jsou vychylovány na stínítko obrazovky. Zde paprsky emitují rychle slábnoucí světlo. Pro zachování kvalitního obrazu je třeba jeho stálé obnovování (jeden z nejdůležitějších parametrů monitoru). Vychylování se provádí pomocí dvou párových destiček, které vychylují elektrony v horizontálním a vertikálním směru. Pro vytváření barevného obrazu se používají tři katody, speciální luminisenční vrstva a stínící maska. Na stínidle jsou pravidelně rozložené trojice luminisenčních bodů, které reprezentují jednotlivé složky RGB. Tyto body se pro lidské oko jeví jako jeden výsledný bod. Vychylovací destičky standardně používají vychylovací úhel 90º. Některé novější modely používají úhel 100 stupňový, který umožňuje zmenšit rozměry, ovšem klade vyšší nároky na zobrazování.

Na rozdíl od LCD monitorů, kde uvedená úhlopříčka deklaruje skutečný rozměr obrazovky, u klasických 17 palcových monitorů jen zřídka přesáhne 16 palců. 15 palcový monitor mívá ve skutečnosti okolo 14 palců , 14 palcový mívá 13,3 palce.

Pro určení obnovovací frekvence je důležitá hodnota horizontálního kmitočtu (udává se v kHz a určuje kolik řádků je schopný monitor vykreslit za sekundu). Hodnota vertikálního kmitočtu udává, jaké frekvence monitor zvládne.

U CRT monitorů se lze setkat se třemi základními typy obrazovek:

1. **Bodová maska (delta)**. Toto je nejstarší typ obrazovek. Pod stínítkem je umístěna děrovaná fólie, ve které se oddělují obrazové body. Kvalita výsledného obrazu je úměrná vzdálenosti bodů v této masce. Čím je vzdálenost menší, tím je lepší výsledný obraz. V současné době se tato vzdálenost pohybuje v rozmezí 0,25 - 0,28 mm.
2. **Štěrbinová mřížka (trinitron)**. Je založena na novější technologii. K vytváření obrazu se používá mřížka s napnutými dráty se svislém směru. Pro zajištění stability obsahuje dva dráty, jejichž stíny jsou na obrazovce zřetelně viditelné (v jedné a ve

dvou třetinách výšky obrazovky). Předností obrazovek je kvalita barev, vyšší kontrast a jas.

3. **Štěrbinová maska (Inline)**. Jde o nejnovější technologii, která se snaží kombinovat výhody předchozích typů obrazovek. Otvory v masce jsou obdélníkového tvaru.

Při koupi nového monitoru je třeba vzít v úvahu zejména únavu a námahu očí. Z toho důvodu by měl mít monitor při práci obnovovací frekvenci alespoň 85 Hz. Dále by měl mít některý z certifikátů MPR-II, TCO' 95, TCO' 99 a EPA EnergyStar. Obraz by měl být kvalitní, stabilní a neblikající, mít malé rozestupy bodů (0,28 mm dobré, 0,26 mm výborné, 0,24 mm nejlepší), regulaci moaré (nežádoucí rušivé obrazce vlnového charakteru vznikající interferencí mezi obrazovými prvky definovanými obrazovým signálem a skutečným rozložením obrazových prvků definovaných na sítnici maskou nebo mřížkou obrazovky) a horizontální kmitočty u 15 palcového monitoru alespoň 70 kHz, u 17 palcového 86 kHz a u 19 palcového 96 kHz.

Důležitým kritériem je i reálné rozlišení, které lze na daném monitoru používat vzhledem k úhlopříčce:

Velikost obrazovky	640x480	800x600	1024x768	1280x1024	1600x1200
14’’	doporučené	použitelné	nepoužitelné	nepoužitelné	nepoužitelné
15’’	použitelné	doporučené	použitelné	nepoužitelné	nepoužitelné
17’’	použitelné	použitelné	doporučené	použitelné	nepoužitelné
19’’	použitelné	použitelné	doporučené	doporučené	použitelné
21’’	použitelné	použitelné	použitelné	doporučené	použitelné

Velká pozornost se u monitorů věnuje jejich škodlivým vlivům na zdraví člověka. Jde zejména o časté bolesti hlavy a zhoršování zraku. Tyto problémy jsou často způsobeny nesprávným pracovním prostředím a špatným nastavením zobrazovacího systému.

3.2.4 LCD

LCD (*Liquid Crystal Display*) je plochý displej založený na využití změn optických vlastností tzv. tekutých krystalů v závislosti na změnách elektrického pole, které na ně působí. Mezi výhody LCD patří nízká spotřeba elektrické energie, nízká hmotnost, malá hloubka, technologicky méně náročná výroba, přímé adresování každého bodu obrazovky a na rozdíl od CRT monitorů je vidět téměř 100 % plochy obrazu. K nevýhodám patří nízký kontrast, malý úhel pozorování, problémy se změnou rozlišení (dané adresováním každého bodu) a zobrazení na LCD monitoru je pomalejší, proto se rychlé změny obrazu mohou jevit rozmazaně.

Základní princip je velice jednoduchý. Materiál tekutého krystalu (olejovitého nebo koloidního charakteru) je vložený mezi dvě skleněné destičky, které mají z vnější strany nanesený průsvitný polarizátor a z vnitřní strany vyrovnávací přípravek, který způsobuje, že molekuly krystalů jsou seřazeny rovnoběžně se sklem. Protože jsou polarizátory vzájemně natočené o 90 stupňů, molekuly krystalu vytvoří devadesátistupňový spirálovitý útvar, dopadající světlo se polarizuje a otáčí se o 90 stupňů. Vychází tedy ve stejné rovině jako vstupuje. Po přivedení napětí na příslušnou elektrodu se molekuly krystalu působením elektrického pole vzniklého nad elektrodou seřadí kolmo na sklo. Oblast nepropouští světlo – jeví se jako černá.

V současné době existuje několik typů krystalů a způsobů řízení zobrazovačů LCD. Mezi nejznámější a nejpoužívanější patří nematické krystaly (*TN – Twist Nematic, STN – SuperTwist Nematic*). Kromě nich existují ještě smektické a fotoelektrické krystaly. Základní způsoby řízení jsou dva – multiplexní (jednodušší, levnější, ale horší kvalita) a aktivní maticové zobrazování (větší kontrast).

3.2.5 Grafická karta

Grafická karta (adaptér) je zařízení, které zajišťuje výstup textových a grafických dat na zobrazovací zařízení. Nejjednodušší typy adaptéru pouze mapují část operační paměti vyhrazené při zobrazování (tzv. videopaměť). Dokonalejší karty mají vlastní paměť, která umožňuje jednoduché objekty zobrazovat samostatně. Nazývají se grafické akcelerátory.

Základem každé grafické karty je grafický čip. Ten má na starosti všechny grafické operace, zpracování a vykreslení obrazu. Protože dříve karty nepodporovaly žádné urychlovací funkce, musel všechny potřebné výpočty vykonávat procesor. Postupem času nastal s vývojem procesorů i vývoj grafických karet, aby i na běžném PC bylo možné dosáhnout dobrého výkonu v oblasti 3D grafiky. Přitom bylo nutné přihlédnout k následujícím vlastnostem: karta v sobě musela integrovat všechny funkce běžné VGA karty, funkce 2D GUI akcelerátoru (podpora Windows od verze 95), funkce na urychlení 3D grafiky a s rozšiřováním videosouborů i hardwarovou podporu přehrávání digitalizovaného videa ve formátu AVI, MPEG 1 a MPEG 2. Kromě toho karta musela disponovat potřebným 2D výkonem ve standardních MS DOS režimech VGA a SVGA.

Mezi základní parametry grafické karty patří rychlost, kvalita zobrazování, poměr cena/výkon, paměť a rozhraní, přes které se připojuje do počítače. Rychlost se ve hrách udává obvykle počtem obrázků za sekundu (FPS). Vyjadřuje to, kolik pixelů je daná karta schopna vykreslit. Rychlost 3D enginu se obvykle udává v počtu polygonů, které je schopen 3D grafický čip vypočítat a zobrazit za jednu sekundu. Na to, aby grafická karta využívala 3D funkce, je třeba mít příslušný software, který je dokáže využít. Za tímto účelem byla vyvinuta grafická rozhraní Open GL a Direct 3D (viz kap. 2.3) nebo dnes již historický GLIDE. Pokud 3D akcelerátor chybí, musí všechno počítat procesor, což přináší znatelný pokles rychlosti a kvality.

Většinou platí nepsané pravidlo, že čím více paměti má videokarta, tím lépe. Vše závisí na použití. V případě 15 palcového monitoru stačí pro 2D 2 MB RAM. Na 3D je však potřeba paměti několikanásobně víc. Je třeba počítat nejen s pamětí na zobrazování (v DirectDraw bývají dokonce dvě – *front* a *back buffer,* některé aplikace využívají i tzv. *triple buffering*), ale i např. se *z-bufferem,* kde se uchovávají textury. Na 4 MB 3D kartě je maximum 800 x 600 ale pokud je potřeba mít v tomto rozlišení i 32-bitový *z-buffer* a textury o velikostech 2048 x 2048 pixelů, tak je třeba 16 a víc MB paměti na grafické kartě.

Volná paměť se využívá hlavně na uchovávání textur, které mají být zpracovány. Výhodou je, pokud karta dokáže textury komprimovat - např. Savage, GeForce apod., čímž lze do paměti na grafické kartě těchto textur vložit více a nemusí se nacházet v hlavní paměti. Pravidlo, čím víc paměti, tím lepší karta, nemusí byť vždy pravdivé. Např. RIVA TNT s 8 MB RAM je pomalejší než s 16 MB RAM, ale např. SAVAGE4 s 8 MB je téměř stejně rychlé jako s 32 MB RAM za předpokladu, že se intenzivně nevyužívá celá RAM grafické karty.

V současné době se karty vyrábí zejména pro AGP slot. Ten byl navržen pro nárůst propustnosti mezi CPU a jádrem grafické karty, zejména pro velikost textur, které je třeba zobrazit v jedné sekundě.

3.2.5.1 Základní typy grafických karet PC

MDA (*Monochrome Display Adapter*) byl dodáván k prvním počítačům PC. Pracoval pouze v textovém režimu a umožňoval zobrazit 80 x 25 znaků. Font byl uložen v ROM paměti, proto ji bylo nutné pro implementaci národního prostředí vyměnit.

CGA (*Color Graphics Adapter*) byl první skutečnou grafickou kartou s kapacitou paměti 16 kB. Grafické informace se zpracovávaly ve dvou režimech: 320 x 200 pixelů ve 4 barvách a 640 x 200 monochromaticky. Pro práci v textovém režimu měl uživatel možnost 80 (40) sloupců a 25 řádků v 16 barvách. V grafickém režimu se nepoužíval BIOS, proto horní část znaků v ASCII tabulce mohla být snadno implementována uživatelem a šlo tedy používat národní prostředí softwarově.

HGC (*Hercules Graphics Card*) vycházel z adaptéru MDA s rozšířením grafického režimu. Podporoval monochromatické rozlišení až 720 x 348 pixelů. Prakticky v grafickém režimu odpovídal každému pixelu jeden bit, proto byl snadný zápis i čtení z obrazové paměti. Její kapacita byla 32 kB a na PC byla mapovaná od adresy B0000 H.

EGA *(Enhanced Graphics Adapter*) s sebou přinesl vyšší rozlišovací schopnost: 640 x 350 pixelů při 16 barvách z palety 64 možných barev. Adaptér měl tabulku znaků jednak v ROM paměti, ale šlo umístit znaky i do RAM a snadno je používat s implementací národního prostředí. S větší pamětí podporoval až dvě videostránky (v jednom okamžiku se jedna stránka zobrazuje, zatímco se do druhé zapisuje), což zvýšilo grafický výkon.

VGA (*Video Graphics Array*) kromě emulace předchozích typů karet přinesl dva nové režimy: 320 x 200 pixelů ve 256 barvách a 640 x 380 v 16 barvách, přičemž paleta barev se mohla vybírat až

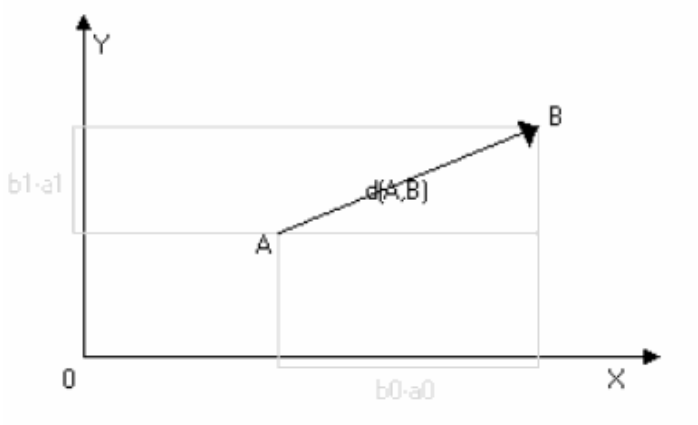
z 2^{24} odstínů. Režimy VGA se postupně staly standardem a postupně docházelo k inovacím zejména ve zvýšení rozlišovací schopnosti. Rozšířenými se tak staly režimy SVGA (*Super VGA*) s rozlišením 800 x 600 a vyšším.

4. MATEMATIKA V POČÍTAČOVÉ GRAFICE

4.1 Vektory

Vektor se zpravidla značí šipkou nad písmenem a jeho souřadnice se píší do kulatých závorek. Vektor má konstantní směr a může se přesouvat v daném souřadnicovém systému. Obyčejně se v rovině označuje $\vec{u} = (u_0, u_1)$, kde u_0 je *x*-ový koeficient vektoru (b_0-a_0) a u_1 je *y*-ový koeficient vektoru (b_1-a_1). Takový vektor (obr. 73) lze nakreslit jako úsečku od bodu *A* se souřadnicemi (a_0, a_1) do bodu $B = (b_0, b_1)$.

X-ový jednotkový vektor je $\vec{i}$ = (*1, 0*) a *y*-ový jednotkový vektor je $\vec{i}$ = (*0, 1*). V prostoru po přidání z-ové souřadnice budou jednotkové vektory (*1, 0, 0*), (*0, 1, 0*) a (*0, 0, 1*).



Obr. 73. Vektor *AB* a jeho délka *d*

4.1.1 Velikost vektoru

Délka vektoru $\vec{u}$ umístěného do bodů $A = (a_0, a_1)$ a $B = (b_0, b_1)$ se dá jednouše odvodit podle Pythagorovy věty a vypočítá se jako

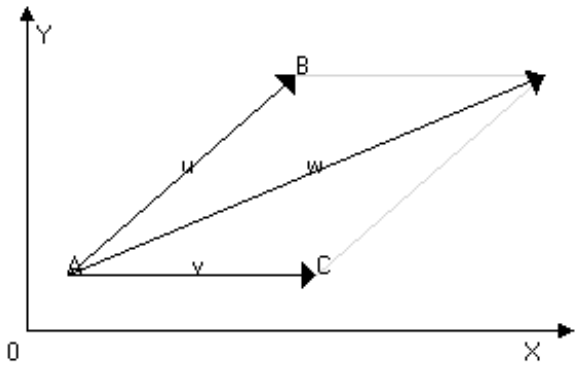
$$\vec{u} = d(A,B) = \sqrt{(b_0 - a_0)^2 + (b_1 - a_1)^2}\ . \qquad (62)$$

4.1.2 Sčítání a odčítání vektorů, opačný vektor

Vektory se sčítají a odečítají po jednotlivých souřadnicích a výsledkem je opět vektor (obr. 74). Součet je komutativní a asociativní, rozdíl není komutativní ani asociativní. Opačný vektor je $\vec{u} = (-u_0, -u_1, -u_2)$.

Příklad na sčítání a odčítání vektorů $\vec{u} = (u_0, u_1, u_2)$ a $\vec{u} = (v_0, v_1, v_2)$:

$$\vec{w} = \vec{u} \pm \vec{v} = (u_0 \pm v_0, u_1 \pm v_1, u_2 \pm v_2). \tag{63}$$



Obr. 74. Vektor *AB* a jeho délka *d*

4.2 Matice

Maticí se nazývá speciálně zvolená tabulka reálných nebo komplexních čísel

$$\begin{pmatrix} a(1,1) & a(1,2) & ..... & a(1,n) \\ a(2,1) & a(2,2) & ..... & a(2,n) \\ ..... & ..... & ..... & ..... \\ a(m,1) & a(m,2) & ..... & a(m,n) \end{pmatrix}. \tag{64}$$

Číslo *a(i,j)* je prvek matice, která obsahuje *n* sloupců a *m* řádků, pro které platí, že $1 \leq i \leq m$ a $1 \leq j \leq n$. Rozměrem matice je $m \times n$, což se zapisuje jako $\boldsymbol{A}(m \times n)$. První index *i* prvku matice *a(i,j)* ukazuje, ve kterém řádku se prvek nachází, a podobně index *j* ukazuje na sloupec matice. Pokud má matice počet řádků a sloupců stejný, nazývá se čtvercová.

Bude-li existovat matice typu $1 \times n$, lze ji nazvat horizontální nebo řádkový vektor

a = | a(1) a(2) . . . a(n) |.

Matice je diagonální, pokud jsou všechny prvky nad a pod diagonálou nulové. Diagonála je množina prvků matice se stejnými indexy sloupce a řádku.

Speciálním případem diagonální matice je jednotková matice, jejíž všechny prvky na diagonále jsou rovné 1. Obyčejně se značí písmenem *I*.

$$\text{Jednotková matice } I = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}. \tag{65}$$

Matice, jejíž všechny prvky jsou nulové se nazývá nulová matice. Označuje se **O**.

Matice ***B*** je inverzní k matici ***A*** pokud platí : ***A*** x ***B*** = ***I***, nebo ***B*** = ***A*** ^ (-1).

4.2.1 Operace s maticemi

4.2.1.1 Sčítání matic

Maticové sčítání je definováno pro matice stejného typu tak, že ke každému prvku *b(i,j)* matice *B* je přičtený prvek *a(i,j)* matice *A*, pro $1 \le i \le m$ a $1 \le j \le n$.

Příklad : Sčítání matic 3 x 3

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 0 & 1 & -2 \\ 2 & 3 & 0 \end{pmatrix},\ B = \begin{pmatrix} -1 & 2 & 1 \\ 5 & 6 & -3 \\ 2 & -4 & 1 \end{pmatrix},\ A + B = \begin{pmatrix} 0 & 4 & 4 \\ 5 & 7 & -5 \\ 4 & -1 & 1 \end{pmatrix}. \tag{66}$$

4.2.1.2 Násobení matice skalárem

Skalárem se rozumí jedno číslo z definičního oboru. Operace se provádí tak, že se každý prvek matice vynásobí skalárem.

Příklad:

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 0 & 1 & -2 \\ 2 & 3 & 0 \end{pmatrix},\ k = 2,\ A * k = \begin{pmatrix} 2 & 4 & 6 \\ 0 & 2 & -4 \\ 4 & 6 & 0 \end{pmatrix}. \tag{67}$$

4.2.1.3 Násobení dvou matic

Násobení dvou matic je poněkud obtížnější, než násobení matice skalárem. Pokud budou existovat matice

$$A = \begin{pmatrix} a(1,1) & a(1,2) & a(1,3) \\ a(2,1) & a(2,2) & a(2,3) \\ a(3,1) & a(3,2) & a(3,3) \end{pmatrix}, B = \begin{pmatrix} b(1,1) & b(1,2) \\ b(2,1) & b(2,2) \\ b(3,1) & b(3,2) \end{pmatrix}, C = A * B = \begin{pmatrix} c(1,1) & c(1,2) \\ c(2,1) & c(2,2) \\ c(3,1) & c(3,2) \end{pmatrix}, \tag{68}$$

potom prvky *c(1,1) = a(1,1)*b(1,1)+a(1,2)*b(2,1)+a(1,3)*b(3,1),*

*c(2,1) = a(2,1)*b(1,1)+a(2,2)*b(2,1)+a(2,3)*b(3,1),*

*c(1,2) = a(1,1)*b(1,2)+a(1,2)*b(2,2)+a(1,3)*b(3,2)* … atd.

Zjednodušeně řečeno prvek na pozici *x, y* ve výsledné matici se vypočítá součtem součinů prvků řádku *x* prvního činitele (matice *A*) a prvků sloupce *y* druhého činitele (matice *B*). Nutnou podmínkou pro násobení tedy je, aby počet sloupců první matice se rovnal počtu řádků druhé matice. Výsledná matice bude mít rozměry, které lze popsat obecně vztahem

$$A(m,n) * B(n,o) = C(m,o)\,. \tag{69}$$

4.3 Transformace

Geometrické transformace patří mezi nejčastěji používané operace v počítačové grafice. Transformace lze rozdělit na lineární a nelineární. Mezi lineární patří posunutí, otáčení, změna měřítka, zkosení a operace vzniklé jejich skládáním.

Transformace lze aplikovat na jednotlivé body objektu, který tak mění svoji polohu, nebo na souřadnicový systém. S prvním případem se lze setkat v počítačové trojrozměrné animaci, pohybu objektu po dráze, druhý případ nachází uplatnění při projekci nebo řešení viditelnosti.

4.3.1 Homogenní souřadnice

Pro zjednodušení výpočtů transformací se používá reprezentace pomocí homogenních souřadnic. Výhodou je snadná reprezentace souřadnic pomocí matic. Uspořádaná čtyři čísla [*x*, *y*, *z*, *w*] se nazývají pravoúhlé homogenní souřadnice bodu *P* s kartézskými souřadnicemi [*X*, *Y*, *Z*] platí-li

$$X = \frac{x}{w}, Y = \frac{y}{w}, Z = \frac{z}{w}, w \neq 0\,. \tag{70}$$

Bod je svými homogenními souřadnicemi určen jednoznačně. Souřadnice *w* se nazývají váhou bodu. Často se volí *w* = 1.

Matice 4 x 4 reprezentující lineární transformaci bodu *P* = [*x*, *y*, *z*, *w*] na bod *P'*= [*x'*, *y'*, *z'*, *w'*] bude v dalším textu označena *A*. Pro převod souřadnic pak platí

$$P' = [\,x', y', z', w'\,] = PA = [\,x, y, z, w\,] \begin{bmatrix} a_{11} & a_{12} & a_{13} & 0 \\ a_{21} & a_{22} & a_{23} & 0 \\ a_{31} & a_{32} & a_{33} & 0 \\ a_{41} & a_{42} & a_{43} & 1 \end{bmatrix}. \tag{71}$$

Obdobně by se dala odvodit matice v rovině (rozměry 3 x 3).

4.3.2 Dvojrozměrné geometrické transformace

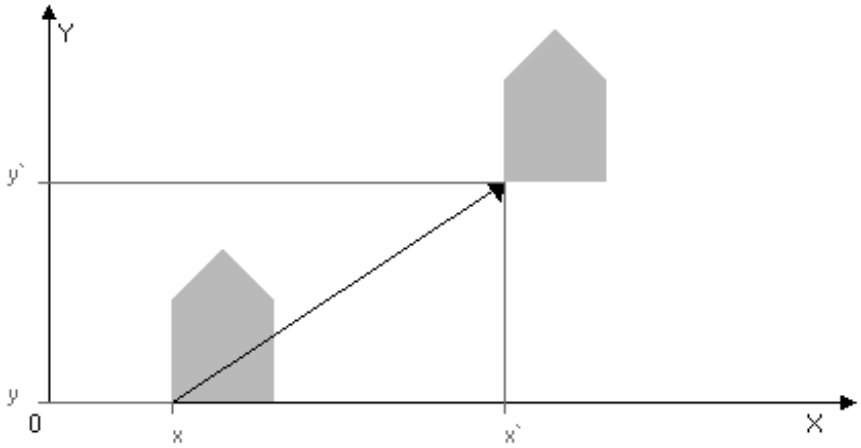
4.3.2.1 Posunutí

Posunutí (translace) představuje přemístění objektu z jedné polohy do druhé. Transformace posunutí je určena vektorem posunutí

$$\vec{p} = (X_T, Y_T) = (X' - X, Y' - Y),$$

kde (*X*, *Y*) jsou souřadnice objektu ve výchozí a (*X'*,*Y'*) v koncové poloze. Transformační matice posunutí je

$$A_M = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ X_t & Y_t & 1 \end{bmatrix}. \tag{72}$$

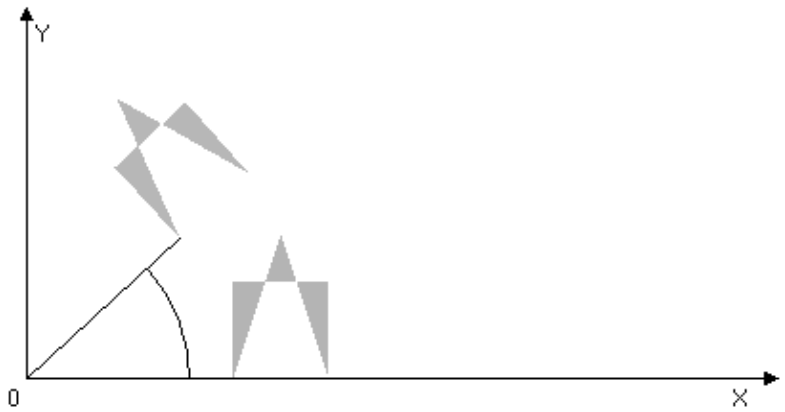


Obr. 75. Posunutí

4.3.2.2 Otáčení

Transformace objektu okolo pevně daného bodu se nazývá otáčením (rotací). Otáčením bodu *P* kolem počátku souřadnic [0, 0] lze získat nový bod o souřadnicích

$$\begin{aligned} X' &= X\cos\alpha - Y\sin\alpha \\ Y' &= X\sin\alpha + Y\cos\alpha \end{aligned}. \tag{73}$$



Obr. 76. Otáčení

Maticové vyjádření transformace otáčení má tvar

$$A_R = \begin{bmatrix} \cos\alpha & \sin\alpha & 0 \\ -\sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 1 \end{bmatrix}. \tag{74}$$

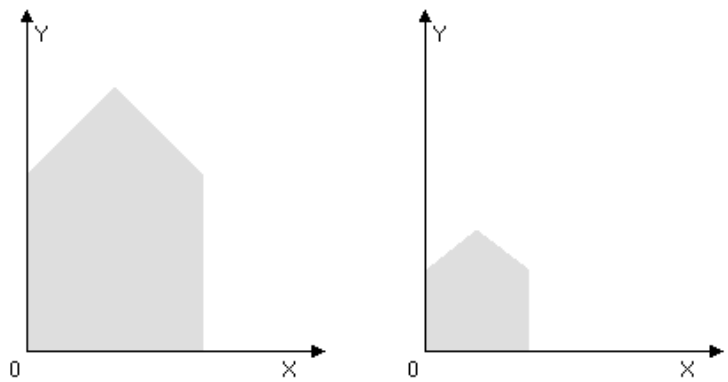
4.3.2.3 Změna měřítka

Změnou měřítka dochází ke změně velikosti objektu ve směru souřadnicových os. Pokud je hodnota koeficientu poměru nové délky k původní větší než 1, objekt se prodlužuje, v opačném případě se zkracuje. Rovnice pro změnu měřítka má tvar

$$\begin{aligned} X' &= S_X X \\ Y' &= S_Y Y \end{aligned}, \tag{75}$$

kde S_X je koeficient změny měřítka ve směru souřadnicové osy x, S_Y je koeficient změny měřítka ve směru osy y. Transformační matice má tvar

$$A_S = \begin{bmatrix} S_X & 0 & 0 \\ 0 & S_Y & 0 \\ 0 & 0 & 1 \end{bmatrix}. \tag{76}$$

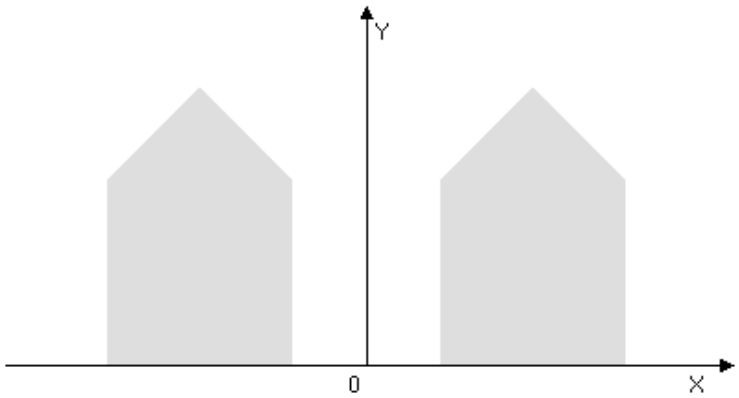


Obr. 77. Změna měřítka

4.3.2.4 Souměrnost

Souměrnost (symetrie) je zvláštním případem změny měřítka. Nastává v okamžiku, kdy absolutní hodnota koeficientu změny měřítka v některé z os je rovna jedné. V rovině lze souměrnost rozdělit na středovou a osovou. Středová souměrnost se provádí podle počátku souřadnic, osová podle jedné z os. Případy, které mohou nastat, jsou uvedeny v následující tabulce:

	S_X	S_Y
souměrnost podle osy *x*	1	-1
souměrnost podle osy *y*	-1	1
středová souměrnost	-1	-1

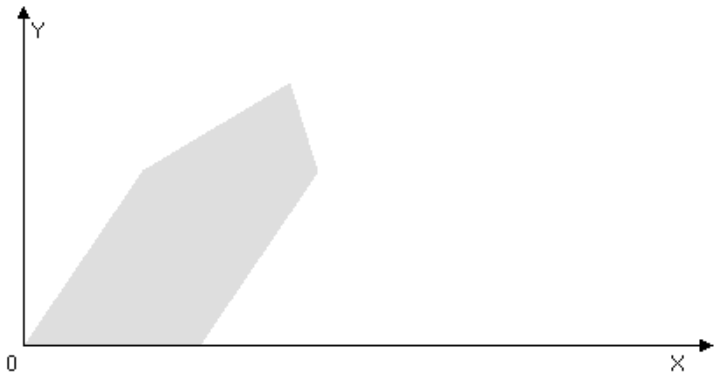


Obr. 78. Souměrnost podle osy *y*

4.3.2.5 Zkosení

Bod *P'* získaný transformací zkosení z bodu *P* má souřadnice

$$\begin{aligned} X' &= X + SH_X Y \\ Y' &= SH_Y X + Y \end{aligned} . \tag{77}$$



Obr. 79. Zkosení ve směru osy *x*.

Koeficienty SH_X a SH_Y udávají míru zkosení ve směru příslušné osy. Odpovídající transformační matice má tvar

$$A_{Sh} = \begin{bmatrix} 1 & SH_Y & 0 \\ SH_X & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}. \tag{78}$$

4.3.2.6 *Skládání transformací*

Při skládání transformací záleží na pořadí, v jakém se transformace provádějí. Je tedy rozdíl, pokud se bod nejprve otočí a posléze posune nebo se nejdříve posune a pak otočí. Transformaci, která vzniká skládáním více transformací, lze vyjádřit jedinou maticí, která vznikne postupným násobením matic odpovídajících jednotlivým transformacím. Samozřejmě je třeba dodržet pořadí násobení matic podle posloupnosti dílčích transformací.

4.3.3 Trojrozměrné geometrické transformace

Podobně jako byly popsány transformace v rovině, lze analogicky odvodit transformace ve třech rozměrech. V počítačové grafice se pro ně používá matice rozměru 4 x 4.

4.3.3.1 *Posunutí*

Transformace posunutí ve 3D je určená vektorem posunutí

$$\vec{p} = (X_T, Y_T, Z_T) = (X' - X, Y' - Y, Z' - Z), \tag{79}$$

kde (X, Y, Z) jsou souřadnice objektu ve výchozí a (X',Y',Z') v koncové poloze. Transformační matice posunutí je

$$A_M = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ X_T & Y_T & Z_T & 1 \end{bmatrix}. \tag{80}$$

4.3.3.2 *Otáčení*

Otáčení ve trojrozměrném prostoru může být jedním z případů otáčení kolem jednotlivých souřadnicových os:

$$A_{Rx} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\alpha & \sin\alpha & 0 \\ 0 & -\sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, A_{Ry} = \begin{bmatrix} \cos\alpha & 0 & \sin\alpha & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\alpha & 0 & \cos\alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix},$$

$$A_{Rz} = \begin{bmatrix} \cos\alpha & \sin\alpha & 0 & 0 \\ -\sin\alpha & \cos\alpha & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{81}$$

4.3.3.3 *Změna měřítka*

Změně měřítka v prostoru odpovídá transformační matice

$$A_{Sc} = \begin{bmatrix} S_X & 0 & 0 & 0 \\ 0 & S_Y & 0 & 0 \\ 0 & 0 & S_Z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \tag{82}$$

kde S_X je koeficient změny měřítka ve směru souřadnicové osy *x*, S_Y je koeficient změny měřítka ve směru osy *y* a S_Z je koeficient změny měřítka ve směru osy *z*.

4.3.3.4 *Souměrnost*

V prostoru je možné souměrnost rozdělit do tří skupin – středovou souměrnost, osovou souměrnost a souměrnost podle roviny. Všechny tyto případy jsou uvedeny v následující tabulce:

	S_X	S_Y	S_Z
souměrnost podle osy *x*	1	-1	-1
souměrnost podle osy *y*	-1	1	-1
souměrnost podle osy *z*	-1	-1	1
souměrnost podle roviny *xy*	1	1	-1
souměrnost podle roviny *xz*	1	-1	1
souměrnost podle roviny *yz*	-1	1	1
středová souměrnost	-1	-1	-1

4.3.3.5 *Zkosení*

Zkosení v prostoru lze rozdělit na tři případy se směru jednotlivých rovin. Ve všech třech případech určují koeficienty SH_X, SH_Y a SH_Z míru zkosení v odpovídajícím směru

$$A_{Shyz} = \begin{bmatrix} 1 & SH_y & SH_z & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, A_{Shxz} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ SH_x & 1 & SH_z & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$A_{Shxy} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ SH_x & SH_y & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \tag{83}$$

SEZNAM POUŽITÉ LITERATURY

[1] DRDLA, J.: *Geometrické modelování křivek a ploch*. 2. vyd. Skripta PřF UP Olomouc, 2001. 94 s.

[2] HUDEC, B. : *Základy počítačové grafiky*. 3. vyd. Skripta ČVUT Praha, 2001. 224 s.

[3] MARTIŠEK, D. : *Matematické principy grafických systémů*. 1. vyd. Littera Brno, 2002. 296 s. ISBN 80-85763-19-2.

[4] PELIKÁN, J. : *Computer graphics- Lectures and seminars*. [on-line]. MFF UK Praha, opraveno 2003. <URL: http://home.pf.jcu.cz/~edpo/povray/povray.html>.

[5] POLÁCH, E. : *Povray 2.2*. [on-line]. PF JCU České Budějovice, opraveno 1999. <URL: http://cgg.ms.mff.cuni.cz/lectures>.

[6] SOBOTA, B. : *Počítačová grafika*. [on-line]. TU Košice, opraveno 2003. <URL: http://hornad.fei.tuke.sk/predmety/pg>.

[7] SOBOTA, B. : *Počítačová grafika a jazyk C*. 1. vyd. Kopp České Budějovice, 1995. 276 s. ISBN 80-85828-52-9.

[8] SOBOTA, B., MILIÁN J. : *Grafické formáty*. 1. vyd. Kopp České Budějovice, 1996. 158 s. ISBN 80-85828-58-8.

[9] SYSEL, M. : *Technické vybavení PC*. 1. vyd. Skripta FT UTB Zlín, 2003. 138 s. ISBN 80-7318-108-8.

[10] ŠTUGEL, P. : *Výučba počítačovej grafiky*. [on-line]. MFF UK Bratislava, opraveno 2003. <URL: http.// pg.netgraphics.sk>.

[11] TIŠNOVSKÝ, P. : *Fraktály*. [on-line]. FIT VUT Brno, opraveno 2000. <URL: http://www.vood.mysteria.cz/fraktaly/uvod.html>.

[12] ŽÁRA, J. a kol. : *Počítačová grafika – principy a algoritmy*. 1. vyd. Grada Praha, 1992. 472 s. ISBN 80-85623-00-5.

[13] ŽÁRA, J., BENEŠ, B. & FELKEL, P. : *Moderní počítačová grafika*. 1. vyd. Computer Press Praha, 1998. 448 s. ISBN 80-7226-049-9.